

Extension Guide

Proactive Risk Manager™ Release 9.0.2.0

17 September 2020



Table of contents

1	Extending and Customizing a Multi-Tenant PRM System	4
1.1	Overview	4
1.2	Guidelines for implementing CSMs and product extensions	4
1.3	Upgrading a PRM system containing CSM(s) and/or product extension(s)	5
1.4	Avoiding potential conflicts on resource IDs with PRM future versions	5
2	Extending the PRM Reference Data Manager	6
2.1	Overview	6
2.2	Extending the PRM_REF_DATA Table & View	6
2.2.1	Extend the number of fields	6
2.3	Validate Presence of New Fields in User Interface	7
2.4	Testing the Reference Data Manager New Fields	8
3	Configuring Reports for Unicode Character Display	11
3.1	Step 1: Update Report Fonts table	11
3.2	Step 2: Update Config Table	12
4	JAAS Login Module Development	13
4.1	Overview	13
4.2	Prerequisites	13
4.3	Step One: Writing the LoginModule	13
4.3.1	Sample LoginModule Code	14
4.4	Step Two: Writing the Principal	18
4.4.1	Sample Principal Code	18
4.5	Step Three: Packaging	19
4.5.1	Packaging the LoginModule and Principal	19
4.6	Step Four: Configuration	19
4.6.1	Java Authentication and Authorization Services Support (JAAS)	19
4.6.2	Configuring JAAS	19
4.6.3	Configuring the Database	20
4.6.4	Configuring JAAS for WebSphere 8.5	22
4.7	JAAS Tomcat Login Module	25
5	Creating Custom Rule Types	25
5.1	Overview	25
5.2	Near Real Time Oracle	27
5.3	Near Real Time MS SQL	28
5.4	Update the "Custom Rule" Procedure	28
5.4.1	Changes in the Custom Rule Procedure if both (NRT and Custom) Procedures are used in the Entry procedures	28
5.4.2	Changes in Custom Rule Procedure if only Custom Rule Procedure is used in Entry Procedures	33
6	Adding Search Fields	36
6.1	Overview	36
6.2	Steps to Add Search Fields	36
6.2.1	Changing the wildcard and field combination restrictions	36
6.3	Use Case – Adding One New Search Field	37

6.4	Performance Impact	37
7	Adding an Action, Auto-Action or Auto-Add-In	41
7.1	Overview	41
7.2	Adding a New Action to an Existing Action Level Review Screen	41
7.2.1	Define Localization (L10N) Properties	41
7.2.2	Add a Permission for the Action	41
7.2.3	Define a New Action Type	42
7.2.4	Verify the Action and Permission Settings	42
7.2.5	Add the Action Level Configurations to the Actions Type Table	43
7.2.6	Define the ActionsText L10N Property Values for the Actions Tab & Actions Report Text	44
7.2.7	Define the ActionsText Values for the Actions Tab and Actions Report Text	44
7.2.8	Add the Toolbar Button	45
7.2.9	Add the Action Dialog Screen	45
7.2.10	Restart the Application Server	46
7.2.11	Assign the Action to the Action Level	46
7.2.12	Assign the Action to One or More Roles	46
7.3	Adding an Auto-Action for Use in a PRM rule	47
7.3.1	Add L10N Properties Text for ActionType and ActionType_Action_Level Tables	47
7.3.2	Add a New Action Type	47
7.3.3	Actions Text for the Action	49
7.4	Adding an Auto Add-In Action	49
7.4.1	Overview	49
7.4.2	Steps to add the Auto Add-In	51
8	Developing Add-Ins for PRM	55
8.1	Introduction	55
8.2	The PRM Environment	55
8.3	PRM User Interface Architecture Overview	56
8.4	The PRM User Interface Classes	57
8.5	The PrmParams Class	57
8.6	The PrmAddin Interface	60
8.7	The PrmDialog Class	60
8.8	The PrmServer Class	62
8.9	PRMAddInSample: PRMDialogNoDB.java	64
8.10	PRMAddInSample: PRMDialogDBUI.java & PRMDialogDBqry.java	65
8.11	Database Access – Servlet Component	77
8.12	Packaging the Add-in	80
8.13	Configuring your Servlet Code	86
8.14	Configuring the web.xml file containing the servlets	86
8.15	Configuring the Database	86
8.15.1	ACT table trigger removal	86
8.15.2	Adding the add-in toolbar button on the review screen	86
8.16	Installing AddIn jar to Tomcat	89
8.17	Installing AddIn jar to WebSphere	89
8.18	Setup Add-Ins in Eclipse	90
8.19	Step by Step Example of Configuring the Add-in Functionality	100
8.19.1	Signing the Code	102
9	Localization Utility	104
9.1	Add a new locale to the Database Configuration	104

9.2	Export a translation properties file	105
9.3	Import a Localization Settings file	106
10	XSD Builder Tool	107
10.1	Prerequisites	107
10.2	Introduction to XSD builder tool	107
10.3	Unzip the distribution	107
10.4	Configure the tool	107
10.5	Run the tool	108
10.6	Integrate the new XSD files in the PRM transaction generator (Developers only)	108
11	Configuring Fields to be Masked in the User Interface	109
11.1	Configure how the field should be masked	109
11.2	Configure the field to be masked	109
12	Custom Real-Time Dispositions	110
12.1	Adding Real-Time Dispositions	110
12.2	Show custom dispositions in Real-Time Rule Performance widget	111

1 Extending and Customizing a Multi-Tenant PRM System

1.1 Overview

This Extension Guide contains the steps to apply several common product extensions that are implemented for customers by the ACI Implementation Services staff and/or by the customer's resources. PRM Release 8.7 and higher supports a multi-tenant architecture for both single and multi-tenant PRM installations (for example, installations containing a single PRM tenant, and installations containing two or more PRM tenants). The PRM database structure consists of a Master database schema, where non-tenant specific data is stored, and a separate real-time database schema (if real-time is licensed) and near real-time database schema for each tenant that is defined in the system, where tenant-specific data is stored. The same database architecture is used regardless of whether the PRM customer is single tenant or multi-tenant. A configuration is set in the Master schema indicating whether the installation is real-time only, or both real-time and near real-time. Every tenant must comply with this configuration; it is not possible to have some tenants installed as real-time and near real-time, and others installed as near real-time only. All tenants in an installation share the same PRM web application. When a PRM customer is multi-tenant and has more than one tenant defined in their PRM installation additional guidelines must be followed when implementing custom software modifications (CSMs) and the product extensions contained in this guide.

When implementing a product extension in the system, the same database metadata changes should be made for each tenant, so that each tenant's database schema has the same metadata structure for the tables, views, and procedures. Further, all tenants share the same PRM web application, so customizations to the core user interface code will affect all tenants.

For completeness, this section will provide guidance on both common CSMs and product extensions. CSMs typically require involvement by ACI Product Development staff, cannot be completed without the ACI Product Development staff's assistance/input, and are not described in this guide. Product extensions typically may be completed without assistance from the ACI Product Development staff and are described in this guide.

1.2 Guidelines for implementing CSMs and product extensions

The following product extensions and CSMs require changes to the database metadata structure and/or core user interface code and cannot vary by tenant, but the same extension may be applied to all tenants:

- JAAS Login Module Development
- Adding New Search Fields
- Localization Utility – modifying the PRMStrings.properties, ReportStrings.properties, & PrmImagesStrings.properties files – the files are shared by all tenants
- Adding an existing transaction or demographic field to an existing record source in which the field was not previously present via the Data Extensibility tool (not described in this guide, but a common CSM which may be implemented by ACI Implementation Services or ACI Product Development)
- Adding new transaction or demographic fields to a record source, the user interface, and the database via the Data Extensibility tool (not described in this guide, but a common CSM which may be implemented by ACI Implementation Services or ACI Product Development)
- Any other CSMs that require changes to the database metadata structure

For example, when adding fields to the Search screen, the same fields should be added to the NRT schema for each tenant.

The following extensions and CSMs do not require changes to the database metadata structure or core user interface code, and the extension may vary by tenant:

- Extending the PRM Reference Data Manager
- Configuration of Reports for Unicode Character Display
- Creating Custom Rule Types
- Adding an Action, Auto-Action, or Auto-Add-In*
- Developing Add-Ins for PRM*
- Localization Utility – adding a new locale to the database & importing/exporting files via the Localization Utility screen
- Configuring Fields to be Masked in the User Interface
- Adding custom look-up tables or views to the tenant's database schema, for use in rules (not described in this guide, but a common product extension)
- Adding indexes to improve rule performance (not described in this guide, but a common product extension)

* Since add-ins require changes to the server code and all tenants use the same web application, the add-in code is available to all tenants. The database changes required to support the add-in should be applied to each tenant schema. However, the add-in may be made available to some, but not all tenants by making the add-in menu or add-in button visible to only certain tenants.

The Data Extensibility Tool and the XSD Builder Tool (which is dependent on the Data Extensibility Tool) may be used in a PRM multi-tenant installation, but the resource using the tools must make the same changes to all tenants, so that the metadata structure is consistent across all tenants.

1.3 Upgrading a PRM system containing CSM(s) and/or product extension(s)

Regardless of whether the customer's PRM installation has a single tenant or multiple tenants, if a customer has CSMs on their PRM system and wishes to upgrade to a newer service pack, minor PRM release, or major PRM release (for example, from 8.6.2 to 8.6.5, from 8.5.1 to 8.7, or from 7.1.10 to 8.7), the customer must first contact ACI to initiate the process to evaluate and if needed, uplift and test any CSMs that need to be carried forward, to determine if changes are needed to the CSMs for them to work properly on the higher release. The standard product upgrade scripts do not take any intentional steps to identify or preserve customizations to the database structure, and there is no guarantee that a CSM will work after upgrading PRM, or that the upgrade scripts will execute properly if the system being upgraded contains CSMs.

Generally, a customer should be able to upgrade to a newer PRM service pack or minor release (for example, from 8.6.2 to 8.6.5, or from 8.5.1 to 8.7) without impacting any existing product extensions or with minimal adjustments required. However, this is not guaranteed and ACI recommends that product extensions be thoroughly tested after upgrading to ensure no impacts, which is a common practice for the ACI Implementation Services team to undertake. For example, the database upgrade scripts do not affect any database object they are not aware of, such as custom views, but the custom object may not work properly after upgrading if a standard object it is based on is modified during the upgrade. If there are issues, the ACI Implementation Services team will need to reapply the product extension on the higher release.

1.4 Avoiding potential conflicts on resource IDs with PRM future versions

As the PRM product evolves, it continuously adds new resources to its factory-provided seed data, in all kinds of tables, starting from L10N_PROPERTY, which is used for strings localization, to other tables that hold action types, permissions, etc., which shall be detailed in a section below.

At the same time, a custom extension will most likely need its resources, having their own IDs in one or more of these tables.

To avoid potential conflicts upon PRM upgrade, between the IDs that are allocated for extensions and IDs that shall be allocated by the PRM development in future releases, a good practice is to allocate the IDs

starting from the maximum integer value (i.e. 2 billion) *downwards*. Since PRM development will always allocate IDs and ID ranges starting from the current maximum upwards, the chance of collision is minimal with this approach.

2 Extending the PRM Reference Data Manager

2.1 Overview

The PRM Reference Data table can be extended, to include multiple types of lists. This includes various black lists, white lists, and sanctions lists. The PRM_REF_DATA table has two fields by default, though additional fields may be needed to accommodate cardholder or other data that may be contained in the lists.

This section explains the process of extending the PRM_REF_DATA table, including the modifications required so the extended fields are presented correctly within the UI. It is highly recommended that this process is initially conducted on a test version of PRM and tested accordingly before making changes to a live system. Note, if replication is being performed between NRT and RT for the Reference Data table, the steps below should be performed on both tables, to ensure the replication process is completed.

2.2 Extending the PRM_REF_DATA Table & View

2.2.1 Extend the number of fields

Extend the number of fields within the PRM_REF_DATA table to the required level. The extension must be made in the NRT and RT databases.

Step 1: Extend PRM_REF_DATA

To extend PRM_REF_DATA, you must connect to the NRT database and call the following script:

- for MSSql database run `extend_prm_ref_data_msq.sql` script
- for Oracle database run `extend_prm_ref_data_ora.sql` script

Before you run this script, you must set variables for:

- `column_name`: the name of the new column in PRM_REF_DATA table
- `column_type`: the type of the new column; must be a valid database type
- `column_desc`: text as shown in the Reference Data Manager dialog
- `is_not_null`: if it is a new column, it must not be null

For each new column added, you must change these variables to the proper value and run the script.

Step 2: Extend RT_PRM_REF_DATA

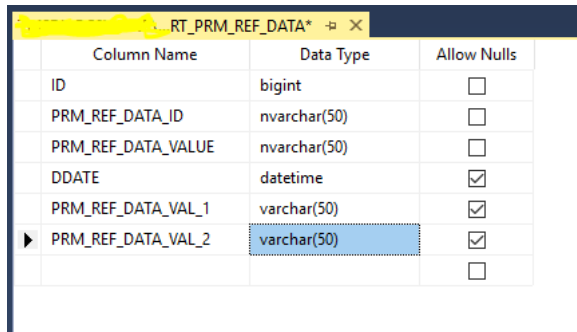
To extend RT_PRM_REF_DATA, you must connect to the RT database. There are 3 possibilities:

1. Use database ALTER command

For example: `ALTER TABLE RT_PRM_REF_DATA
ADD (PRM_REF_DATA_VALUE_2 VARCHAR2(30));`

2. Use database tools:

For example:



Column Name	Data Type	Allow Nulls
ID	bigint	☐
PRM_REF_DATA_ID	nvarchar(50)	☐
PRM_REF_DATA_VALUE	nvarchar(50)	☐
DDATE	datetime	☒
PRM_REF_DATA_VAL_1	varchar(50)	☒
PRM_REF_DATA_VAL_2	varchar(50)	☒
		☐

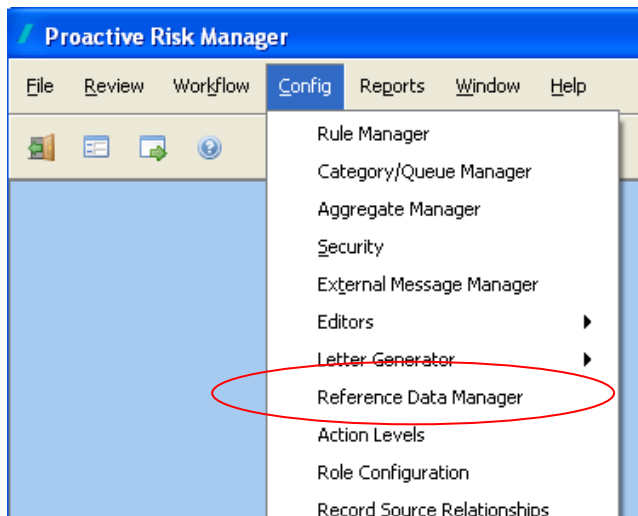
3. Use additional scripts `extend_rt_prm_ref_data_msq.sql` for MSSQL or `extend_rt_prm_ref_data_ora.sql` for Oracle.

In this script, you must replace the tokens with the real server name, the database names on MSSQL or the real database link on Oracle.

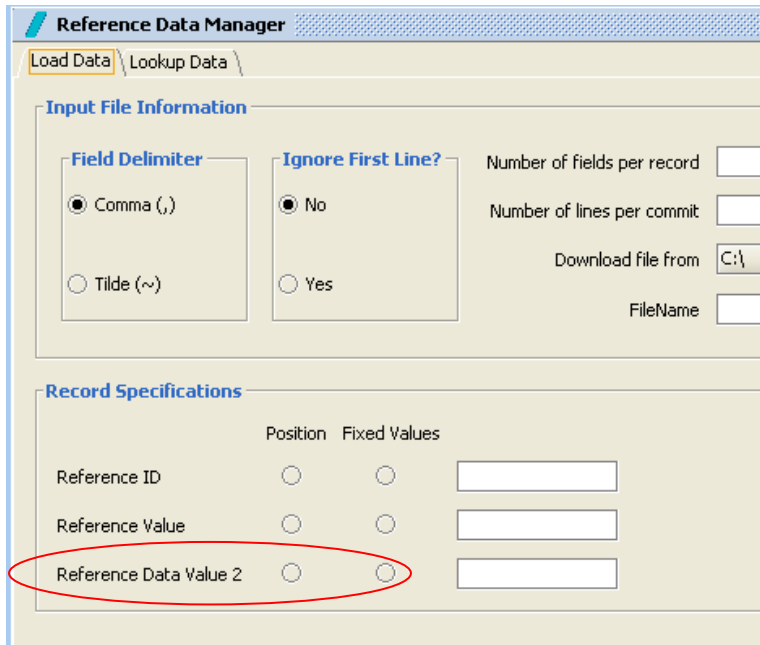
2.3 Validate Presence of New Fields in User Interface

Once the above insert queries have been committed, a check should be made to ensure the new field(s) are present within the UI.

Login to PRM and navigate to the Config drop and click 'Reference Data Manager':



This will present the Reference Data Manager screen. On the 'Load Data' tab, we can see the new field 'Reference Data Value 2':



Reference Data Manager

Load Data | Lookup Data

Input File Information

Field Delimiter

☒ Comma (,)

☐ Tilde (~)

Ignore First Line?

☒ No

☐ Yes

Number of fields per record:

Number of lines per commit:

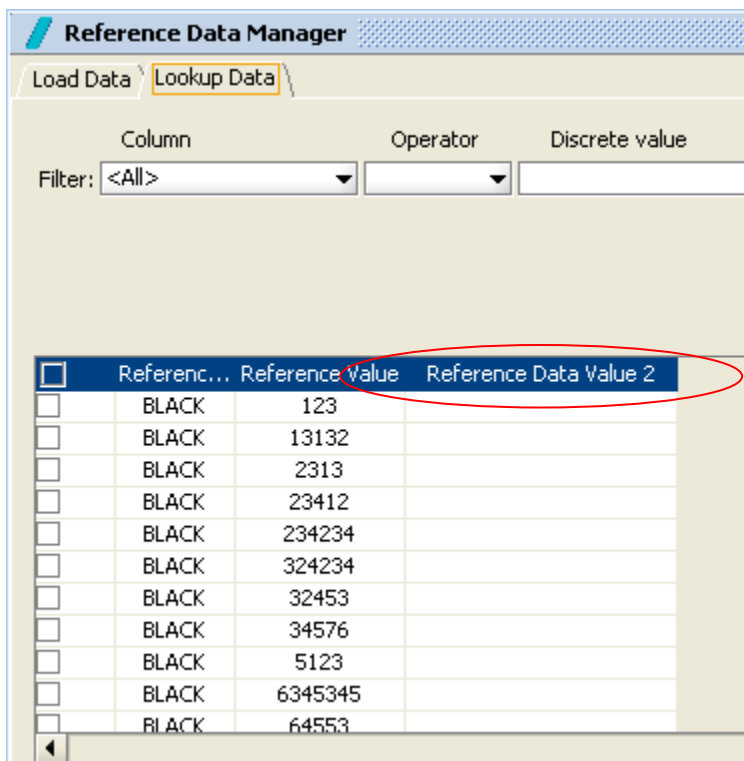
Download file from: C:\

FileName:

Record Specifications

	Position	Fixed Values	
Reference ID	☐	☐	
Reference Value	☐	☐	
Reference Data Value 2	☐	☐	

On the 'Lookup Data' tab, we can see the new field 'Reference Data Value 2' within the grid:



Reference Data Manager

Load Data | Lookup Data

Column: Operator: Discrete value:

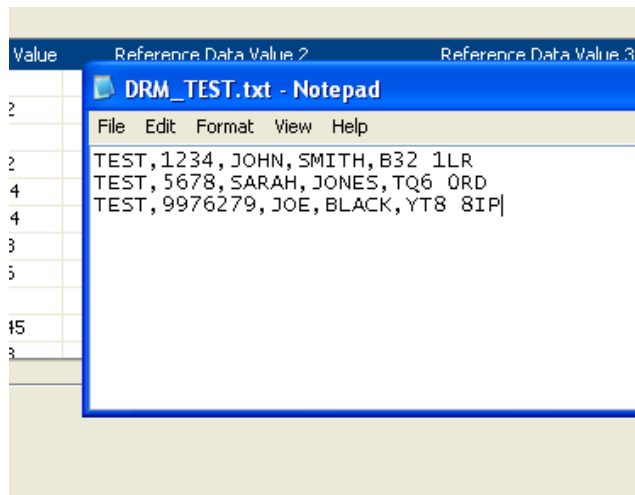
Filter: <All>

	Referenc...	Reference Value	Reference Data Value 2
☐	BLACK	123	
☐	BLACK	13132	
☐	BLACK	2313	
☐	BLACK	23412	
☐	BLACK	234234	
☐	BLACK	324234	
☐	BLACK	32453	
☐	BLACK	34576	
☐	BLACK	5123	
☐	BLACK	6345345	
☐	BLACK	64553	

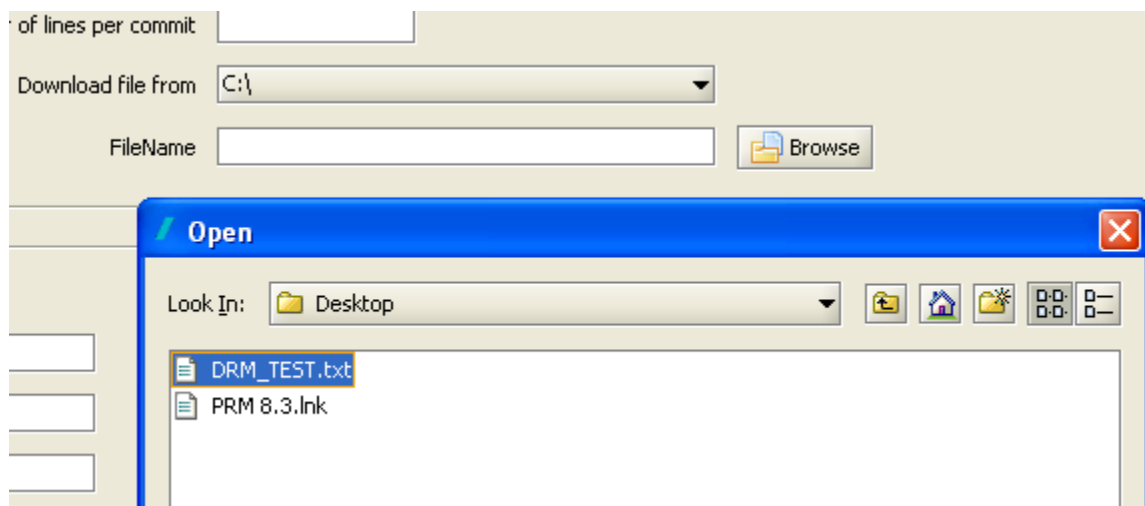
2.4 Testing the Reference Data Manager New Fields

Once we have established that the new field(s) are present in the UI, a test should be made to upload data.

A simple text file can be created like the below:



From the Load Data tab, click **Browse** and navigate to the test files:



Complete the necessary delimiter and record specification options and click **Run**:

Reference Data Manager

Load Data | Lookup Data

Input File Information

Field Delimiter
☒ Comma (,)
☐ Tilde (~)

Ignore First Line?
☒ No
☐ Yes

Number of fields per record: 5
 Number of lines per commit: 3
 Download file from: C:\
 FileName: %entis and Settings%\remote\Desktop\DRM_TEST.txt

Record Specifications

	Position	Fixed Values	
Reference ID	☒	☐	1
Reference Value	☒	☐	2
Reference Data Value 2	☒	☐	3
Reference Data Value 3	☒	☐	4
Reference Data Value 4	☒	☐	5

The results should be displayed:

Reference Data Manager Info

Processing completed.
 Total records processed: 3
 Invalid records found in file: 0
 Duplicate records (not loaded): 0
 Records failed in near real-time database with database error and not loaded in either database: 0
 Records failed in real-time database with database error (loaded successfully in near real-time database): 3

Reference...	Reference...	Reference...	Reference...	Referenc...
TEST	1234	JOHN	SMITH	B32 1LR
TEST	5678	SARAH	JONES	TQ6 0RD
TEST	9976279	JOE	BLACK	YT8 8IP

Save All OK

Check the Lookup data tab to ensure data is present within the new fields:

Reference Value	Reference Data Value 2	Reference Data Value 3	Reference Data Value 4
VIP	223423		
VIP	234234		
VIP	25345235		
VIP	34534636		
VIP	34535		
VIP	5775		
VIP	6756754		
VIP	UAN_010129		
TEST	1234	JOHN	SMITH
TEST	5678	SARAH	JONES
TEST	9976279	JOE	BLACK

3 Configuring Reports for Unicode Character Display

Database changes are required to display Unicode characters within the standard PRM report set. To configure this, please follow the steps outlined below.

3.1 Step 1: Update Report Fonts table

PRM allows the user to override the default report display font (SansSerif) and the default report PDF font (Helvetica). To activate this feature, the user must insert rows into the **REPORTFONTS** table. This table is empty by default. All the reports that do not have entries in the **REPORTFONTS** table will use the default fonts.

Example:

If we want to override the default fonts for the SystemAudit report (with LREPORTID = 15), the following rows should be inserted into the **REPORTFONTS** table to allow the display of Chinese characters:

INSERT INTO reportfonts

(SNAME, BISDEFAULT, SFONTNAME, LSIZE, BISBOLD, BISITALIC, BISUNDERLINE, BISSTRIKETHROUGH, SPDFFONTNAME, SPDFENCODING, BISPDFEMBEDDED, LREPORTID)

VALUES

('SansSerif_Normal', 1, 'SimHei', 9, 0, 0, 0, 0, 'file:///C:/Windows/Fonts/SimHei.TTF', 'Identity-H', 1, 15).

INSERT INTO reportfonts

(SNAME, BISDEFAULT, SFONTNAME, LSIZE, BISBOLD, BISITALIC, BISUNDERLINE, BISSTRIKETHROUGH, SPDFFONTNAME, SPDFENCODING, BISPDFEMBEDDED, LREPORTID)

VALUES

('SansSerif_Bold', 1, 'SimHei', 10, 0, 0, 0, 0, 'file:///C:/Windows/Fonts/SimHei.TTF', 'Identity-H', 1, 15).

The LREPORTID value corresponding to each report is the value in the LID column from the REPORTS table.

Note: The value of the SPDFFONTNAME column must be the path to the font file preceeded by 'file:///'. Also, do not use the name of the font; instead, use the name of the font file.

(In the example above, the name of the font is 'SimHei Regular', but the name of the font file used is 'SimHei'.)

3.2 Step 2: Update Config Table

In the Config Table Editor set the value of **ReportEncodingForPdf** to 1.

4 JAAS Login Module Development

4.1 Overview

This document is intended to help a developer create a custom LoginModule that can be used in conjunction with or separate from the PRM LoginModule.

4.2 Prerequisites

This document assumes that the reader has some knowledge of the following:

Java

JAAS – Java Authentication and Authorization Service

SQL

4.3 Step One: Writing the LoginModule

A custom LoginModule must implement the LoginModule interface.

```
public class PrmSampleLoginModule implements LoginModule
```

To implement this class you must import the standard JAAS packages:

```
import javax.security.*;
```

The LoginModule class must implement the following methods:

initialize() – Initializes the LoginModule

login() - Authenticates the Subject

commit() – Commit/Finalize the authentication process

abort() - Abort the authentication process

logout() - Logout the Subject

Each of these methods will need to be developed based on the needs of the LoginModule.

For more information on these methods or for additional LoginModule information please consult:

[Oracle's LoginModule Developer's Guide](https://docs.oracle.com/javase/6/docs/technotes/guides/security/jaas/JAASLMDevGuide.html)(<https://docs.oracle.com/javase/6/docs/technotes/guides/security/jaas/JAASLMDevGuide.html>)

For additional help: [Oracle's JAAS Reference Guide](https://docs.oracle.com/javase/6/docs/technotes/guides/security/jaas/JAASRefGuide.html)

(<https://docs.oracle.com/javase/6/docs/technotes/guides/security/jaas/JAASRefGuide.html>)

4.3.1 Sample LoginModule Code

This sample LoginModule is an example of how a basic LoginModule might be designed. This LoginModule retrieves a user, a password, and the user's organization from the callbacks and then processes the password against the hard-coded password "prm" for organization "ACI1". For other organization values, the password is processed against the hard-coded password "prm1".

If the user entered organization "ACI1" and a password of "prm" or "PRM", they are authenticated and the login succeeds. If they entered the organization "ACI1" and did not enter the password "prm" or "PRM", then the login fails.

If they entered an organization other than "ACI1", and they entered a password of "prm1" or "PRM1", they are authenticated and the login succeeds. If they entered an organization other than "ACI1" and did not enter the password "prm1" or "PRM1", then the login fails.

Important! The SampleLoginModule should only be used as a basis for LoginModule development and should never be implemented in a live system as is. Additional authentication functionality is required for the SampleLoginModule to be considered secure. Without additional authentication and security, the SampleLoginModule contains vulnerabilities and should never be used in a live system.

```
/*
 * (c) Copyright 2005 ACI Worldwide.
 * All Rights Reserved.
 */

package com.aciworldwide.prm.auth.pam;

import java.util.Map;

import javax.security.auth.Subject;
import javax.security.auth.callback.Callback;
import javax.security.auth.callback.CallbackHandler;
import javax.security.auth.callback.NameCallback;
import javax.security.auth.callback.PasswordCallback;
import javax.security.auth.callback.UnsupportedCallbackException;
import javax.security.auth.login.FailedLoginException;
import javax.security.auth.login.LoginException;
import javax.security.auth.spi.LoginModule;

import com.aciworldwide.prm.auth.OrganizationCallback;

/**
 * <p>
 * This PrmSampleLoginModule authenticates users with a hard-coded password.
 *
 * <p>
 * If testUser successfully authenticates itself, a <code>PrmSamplePrincipal</code> with the user
 * name is added to the
 * Subject.
 */

public class PrmSampleLoginModule implements LoginModule {

    // initial state
    private Subject subject;

    private CallbackHandler callbackHandler;

    private Map sharedState;
```

```

private Map options;

// the authentication status
private boolean succeeded = false;

private boolean commitSucceeded = false;

// username and password
private String username;

private String password;

private String organization;

// user's PRMPrincipal
private PrmSamplePrincipal userPrincipal;

/**
 * Initialize this <code>PRMLoginModule</code>.
 *
 * <p>
 *
 * @param subject
 *         the <code>Subject</code> to be authenticated.
 *
 * <p>
 *
 * @param callbackHandler
 *         a <code>CallbackHandler</code> for communicating with the end user
(prompting for user names and
 *         passwords, for example).
 *
 * <p>
 *
 * @param sharedState
 *         shared <code>LoginModule</code> state.
 *
 * <p>
 *
 * @param options
 *         options specified in the login <code>Configuration</code> for this
particular <code>LoginModule</code>
 *
 *
 */
public void initialize(Subject subject, CallbackHandler callbackHandler, Map sharedState,
Map options) {

    this.subject = subject;
    this.callbackHandler = callbackHandler;
    this.sharedState = sharedState;
    this.options = options;
}

/**
 * Authenticate the user by prompting for a user name and password.
 *
 * <p>
 *
 * @return true in all cases since this <code>LoginModule</code> should not be ignored.
 *
 * @exception FailedLoginException
 *         if the authentication fails.
 *
 * <p>
 *
 * @exception LoginException
 *         if this <code>LoginModule</code> is unable to perform the
authentication.
 */
public boolean login() throws LoginException {

    // prompt for a user name and password
    if (callbackHandler == null)
        throw new LoginException(

```



```

                                "Error: no CallbackHandler available " + "to garner
authentication information from the user");

        Callback[] callbacks = new Callback[3];
        callbacks[0] = new NameCallback("username");
        callbacks[1] = new PasswordCallback("password", false);
        callbacks[2] = new OrganizationCallback();

        try {
            callbackHandler.handle(callbacks);
            username = ((NameCallback) callbacks[0]).getName();
            // ((PasswordCallback) callbacks[1]).clearPassword();
            password = new String(((PasswordCallback) callbacks[1]).getPassword());
            organization = new String(((OrganizationCallback)
callbacks[2]).getOrganization());

            } catch (java.io.IOException ioe) {
                throw new LoginException(ioe.toString());
            } catch (UnsupportedCallbackException uce) {
                throw new LoginException("Error: " + uce.getCallback().toString()
+ " not available to garner authentication information " +
"from the user");
            }

            try {
                if (this.organization.equalsIgnoreCase("ACI1")) {
                    if (this.password.equalsIgnoreCase("prm")) {
                        this.succeeded = true;
                    } else {
                        FailedLoginException fle = new FailedLoginException(
                                "Error: Invalid
Username/Password/Organization Combination. Login Failed!");
                        fle.getCause();
                        throw fle;
                    }
                } else {
                    if (this.password.equalsIgnoreCase("prm1")) {
                        this.succeeded = true;
                    } else {
                        FailedLoginException fle = new FailedLoginException(
                                "Error: Invalid
Username/Password/Organization Combination. Login Failed!");
                        fle.getCause();
                        throw fle;
                    }
                }
            }

            } catch (Exception e) {
                FailedLoginException fle = new FailedLoginException();
                fle.initCause(e);
                throw fle;
            }

            return true;
        }

/**
 * <p>
 * This method is called if the LoginContext's overall authentication succeeded (the
relevant REQUIRED, REQUISITE,
 * SUFFICIENT and OPTIONAL LoginModules succeeded).
 *
 * <p>
 * If this LoginModule's own authentication attempt succeeded (checked by retrieving the
private state saved by the
 * <code>login</code> method), then this method associates a <code>PRMPrincipal</code>
with the <code>Subject</code>
 * located in the <code>LoginModule</code>. If this LoginModule's own authentication
attempted failed, then this
 * method removes any state that was originally saved.

```

```

*
* <p>
*
* @exception LoginException
*         if the commit fails.
*
* @return true if this LoginModule's own login and commit attempts succeeded, or false
otherwise.
*/
public boolean commit() throws LoginException {
    if (succeeded == false) {
        return false;
    } else {
        // add a Principal (authenticated identity)
        // to the Subject

        // assume the user we authenticated is the PRMSamplePrincipal
        this.userPrincipal = new PrmSamplePrincipal(this.username);
        if (!this.subject.getPrincipals().contains(this.userPrincipal)) {
            this.subject.getPrincipals().add(this.userPrincipal);
        }

        // in any case, clean out state
        username = null;
        password = null;

        commitSucceeded = true;

        return true;
    }
}

/**
* <p>
* This method is called if the LoginContext's overall authentication failed. (the
relevant REQUIRED, REQUISITE,
* SUFFICIENT and OPTIONAL LoginModules did not succeed).
*
* <p>
* If this LoginModule's own authentication attempt succeeded (checked by retrieving the
private state saved by the
* <code>login</code> and <code>commit</code> methods), then this method cleans up any
state that was originally
* saved.
*
* <p>
*
* @exception LoginException
*         if the abort fails.
*
* @return false if this LoginModule's own login and/or commit attempts failed, and true
otherwise.
*/
public boolean abort() throws LoginException {
    if (succeeded == false) {
        return false;
    } else if (succeeded == true && commitSucceeded == false) {
        // login succeeded but overall authentication failed
        succeeded = false;
        username = null;
        password = null;
        this.userPrincipal = null;
    } else {
        // overall authentication succeeded and commit succeeded,
        // but someone else's commit failed
        logout();
    }
    return true;
}

/**

```

```

        * Logout the user.
        *
        * <p>
        * This method removes the <code>PRMPrincipal</code> that was added by the
        * <code>commit</code> method.
        *
        * <p>
        *
        * @exception LoginException
        *         if the logout fails.
        *
        * @return true in all cases since this <code>LoginModule</code> should not be ignored.
        */
        public boolean logout() throws LoginException {

            this.subject.getPrincipals().remove(userPrincipal);
            this.succeeded = false;
            this.succeeded = commitSucceeded;
            this.username = null;
            this.password = null;
            this.userPrincipal = null;

            return true;
        }
    }
}

```

4.4 Step Two: Writing the Principal

A Subject is created when JAAS authenticates a user. The purpose of the Subject is to represent an authenticated user. A Subject is made up of Principals. Each Principal represents an identity of that user. For example, a Subject might have a name Principal ("John Doe") and Birthday Principal ("May 20th"), thereby distinguishing this Subject from other Subjects.

A Custom Principal will need to be created to distinguish and authenticate Subjects. A simple sample Principal is shown below.

4.4.1 Sample Principal Code

```

/*
 * (c) Copyright 2006 ACI Worldwide.
 * All Rights Reserved.
 */
package com.aciworldwide.prm.auth.pam;

import java.security.Principal;

public class PrmSamplePrincipal implements Principal
{

```

```
private String name;

public PrmSamplePrincipal(String name)
{
    this.name = name;
}

public String getName()
{
    return name;
}
}
```

4.5 Step Three: Packaging

4.5.1 Packaging the LoginModule and Principal

The LoginModule and Principal need to be packaged together in a JAR file. This JAR file will need to be in a location on the Webserver's classpath. If the class is not found during Login you will need to check the location of the JAR file and make sure the webserver can find it on the classpath.

4.6 Step Four: Configuration

4.6.1 Java Authentication and Authorization Services Support (JAAS)

The PRM User Interface has been enhanced to supports Java Authentication and Authorization Services (JAAS). JAAS uses Pluggable Authentication Modules (PAM) to enable PRM to remain independent of underlying authentication technologies. By implementing a standard JAAS Interface, PRM Users can create a PAM to access their internal authentication system. The Pluggable Authentication Modules can be written by the customer or can be provided by third party commercial Authentication services provider.

4.6.2 Configuring JAAS

PRM 8.0 includes a JAAS LoginModule that uses the existing PRM Authentication Mechanism.

Using the JAAS LoginModule requires some database configuration changes and some configuration changes in each of the Web Application Servers. If the customer has several tenants, these configurations must be done for each of the tenants.

4.6.3 Configuring the Database

4.6.3.1 SystemConfig Table

The **LCProvider** entry in the SystemConfig table (of the underlying tenant database; the same is true for other configuration tables) needs to be set to **com.aciworldwide.prm.auth.PRMJaasDBConfiguration**. This tells PRM which class to use to find JAAS configuration entries.

If **LCProvider** is not set, JAAS is not enabled and will not work. LCProvider must have a value in order for JAAS to function.

The JAASEnabled entry in the SystemConfig table needs to be set to 1 in order for JAAS to function. If JAASEnabled is set to 0, JAAS is not enabled and will not work.

4.6.3.2 AuthConfig Table

The AuthConfig table stores the LoginModule configuration. Each row represents a different LoginModule to be used. This table will need to be configured properly to set up your LoginModule configuration.

In the case of several tenants, each of the tenants will have the same PRMSampleLoginModule configured. In PRMSampleLoginModule class, the organization parameter will be used to identify each tenant.

Here is an example of an SQL insert for the PRMSampleLoginModule. Your SQL statement for your LoginModule will need to be different:

```
INSERT INTO AuthConfig
VALUES ('PRMSample', 'com.aciworldwide.prm.auth.pam.PRMSampleLoginModule', 'Required', NULL);
```

***Bold is required**

AuthConfig		
Field	Type	Description
sName	varchar(50)	A unique name to describe the LoginModule
sModuleClass	varchar(150)	The full class name for the LoginModule
sModuleControlFlag	varchar(15)	The LoginModuleControl flags set the control flag for the login module. The following options are available: OPTIONAL REQUIRED REQUISITE SUFFICIENT Only one control flag can be given. The default flag is REQUIRED if entry is invalid.

sOptions	ntext	Additional options/parameters for the LoginModule can be specified here. These are based on design and are not required. They are optional if your LoginModule needs them. They should be in key/value pairs. There should be an = sign between the keys and values. There should be a space between each entry. Name=John Age=23 Occupation=Programmer The above line would be a valid entry in the sOptions field. It would contain three keys (Name, Age and Occupation) with the values (John, 23, and Programmer).
----------	-------	--

The most important field is the sModuleClass. This has to be the full class name for your LoginModule class. For example: **com.aciworldwide.prm.PrmLoginModule** would be the full name for the class *PrmLoginModule* inside the package *com.aciworldwide.prm*.

4.6.3.3 SystemConfig Table

Important: If your LoginModule is using an external database (outside of PRM) for user passwords, the SystemConfig table needs to be updated.

In the SystemConfig table, the ExternalPasswords row **needs its lCfgValue changed from 0 to 1** if passwords are being handled by an external database.

```
UPDATE SystemConfig SET lCfgValue = '1' WHERE sCfgVariable = 'ExternalPasswords';
```

With ExternalPasswords set to 1, PRM will know that an external database is managing user passwords. User administration dialogs appear differently when this property is set to 1.

The screenshot shows the 'Security' application window with the 'Users' tab selected. On the left, a tree view shows the hierarchy: Users > Team 1 > User 1. The main panel displays the configuration for 'User 1'. The fields are as follows:

- *Name: Analyst19
- *Password: Managed outside of PRM
- *Full Name: (empty field)
- Phone: (empty field)
- E-Mail: (empty field)
- Notes: (empty text area)
- Role: Analyst (dropdown menu)
- Last Login: (empty field)
- Failed Password Attempts: 0

Buttons at the bottom include 'Save', 'Cancel', 'Help', and 'Close'. There is also a 'Clear Failed Password' button next to the 'Failed Password Attempts' field.

If ExternalPasswords is on, the password field will say: *Managed outside of PRM* to let the administrator know that passwords can not be changed through the PRM interface and must be managed through the external system.

4.6.4 Configuring JAAS for WebSphere 8.5

The following procedural changes are required for a WebSphere install:

Log into the Websphere admin console and click on the “Security” link.

Next, click on the “Global Security” link.

The screenshot shows the WebSphere Admin Console interface. On the left is a navigation tree with a 'View: All tasks' dropdown. The 'Security' node is expanded, and 'Global security' is highlighted with a red box. The main content area is titled 'Global security' and includes a 'Close page' link. It contains several sections: 'Administrative security' with a checked 'Enable administrative security' checkbox and links for 'Administrative user roles', 'Administrative group roles', and 'Administrative authentication'; 'Application security' with an unchecked 'Enable application security' checkbox; 'Java 2 security' with an unchecked 'Use Java 2 security' checkbox and checked 'Warn if applications are granted custom permissions' and 'Restrict access to resource authentication data' checkboxes; and 'User account repository' with a 'Current realm definition' field set to 'Federated repositories' and a list of 'Available realm definitions' also showing 'Federated repositories'. On the right, the 'Authentication' section is expanded, showing 'LTPA' as the selected mechanism, with a note that 'Kerberos and LTPA' is currently disabled. Below this, 'Application logins' is highlighted with a red box, along with links for 'System logins' and 'J2C authentication data'. Other links like 'Security domains', 'External authorization providers', and 'Custom properties' are at the bottom.

Expand the **Java Authentication and Authorization Service** node from the Authentication group (on the right side).

Next, click the **Application Logins** Link.

Global security ?

Global security > JAAS - Application logins

Defines login configurations that are used by Java(TM) Authentication and Authorization Service (JAAS). You cannot remove the default login configurations because doing so might cause applications to fail.

⊞ Preferences

New Delete

⊞ ⊞ ⊞ ⊞

Select Alias ⌵

You can administer the following resources:

☐	ClientContainer
☐	DefaultPrincipalMapping
☐	KerberosMapping
☐	TrustedConnectionMapping
☐	WSKRB5Login
☐	WSLogin

Total 6

Click **New** to create a new Login Configuration.

Global security ?

Global security > JAAS - Application logins > New

Defines login configurations that are used by Java(TM) Authentication and Authorization Service (JAAS). You cannot remove the default login configurations because doing so might cause applications to fail.

General Properties

* Alias
PRM

JAAS login modules

New Delete

Select	Module Class Name ⌵	Authentication Strategy ⌵
None		

Total 0

Apply OK Reset Cancel

Enter PRM (all caps) as the configuration alias and click **Apply**.

Global security

Messages

⚠

Changes have been made to your local configuration. You can:

- [Save](#) directly to the master configuration.
- [Review](#) changes before saving or discarding.

⚠

The server may need to be restarted for these changes to take effect.

Global security > JAAS - Application logins > PRM

Defines login configurations that are used by Java(TM) Authentication and Authorization Service (JAAS). You cannot remove the default login configurations because doing so might cause applications to fail.

General Properties

* Alias

PRM

JAAS login modules

NewDelete

SelectModule Class NameAuthentication Strategy

None

Total 0

ApplyOKResetCancel

Click the **New** to add a LoginModule.

Global security

Messages

- ⚠ Changes have been made to your local configuration. You can:
 - [Save](#) directly to the master configuration.
 - [Review](#) changes before saving or discarding.
- ⚠ The server may need to be restarted for these changes to take effect.

Global security > JAAS - Application logins > PRM > New

Each entry in the login configuration must contain at least one login module. However, you can define more than one login module for a login configuration. If you define more than one login module for a login configuration, they are processed in the order that they are defined.

General Properties

* Module class name
com.aciworldwide.prm.auth.pam.PRMLoginModule

☐ Use login module proxy

Authentication strategy
REQUIRED

Custom properties

Select	Name	Value
☐		

Apply OK Reset Cancel

New Delete

Enter full class name as the Module class name.

The Authentication strategy should be set to REQUIRED for a default PRM LoginModule configuration.

Click **OK** and save the Websphere Master Configuration.

Restart the server.

4.7 JAAS Tomcat Login Module

For instructions on how to configure JAAS in Tomcat 7.0 and 8.0, reference:

<https://tomcat.apache.org/tomcat-7.0-doc/realm-howto.html#JAASRealm>
<https://tomcat.apache.org/tomcat-8.0-doc/realm-howto.html#JAASRealm>

5 Creating Custom Rule Types

5.1 Overview

Rules, queues, and aggregates are stored in the *RULEQUE* table. The *RULEDEF* table defines the types of rules, queues and aggregates that are available within the PRM system.

The **LID** value from this table indicates all rule types (Rule, Queue, Aggregate) and subtypes (Standard, Profile, Real-Time).

Standard Rule (LID=0)

Profile Rule(LID=2)

Real time Rule(LID=3)

Standard Queue(LID=1)

Profile Queue(LID=5)

Standard Aggregate(LID=4)

Profile Aggregate(LID=6)

Real time Aggregate(LID=7)

The **LType** column from the RULEDEF table indicates rule type (Rule, Queue, or Aggregate). These values are also stored in the RULETYPES table.

The **LSubType** column from the RULEDEF table indicates rule subtype (Standard, Profile, Real-Time). These values are also stored in the RULESUBTYPES table.

When the rule, queue, and aggregate are syntax checked/deployed, data from the **TEMPLATES** table is used. The TEMPLATES table contains templates used to create the procedure. Each of these templates contains place holders which are replaced from Java.

To add a new custome rule type you must take into consideration the following steps:

1. Insert into the **RULESUBTYPES** table the new subtype ID and name. E.g. insert into rulesubtypes values (3,'Poc')
2. Insert into the **RULEDEF** table new rule and queue def IDs. The LSUBTYPE column for these two new records has to contain the subtype created at step 1 as value. For example,
 - a. insert into ruledef values (8,0,3,'Rule Poc',0,1,9,NULL,1,NULL,'PP_POC_RULES_')
 - b. insert into ruledef values (9,1,3,'Que Poc',0,0,1,NULL,1,NULL,'PP_POC_QUEUES_')
 - c. insert into ruledef values (10,2,3,'Aggregate Poc',0,0,1,NULL,1,NULL,'PP_POC_AGGREGATE_')
3. The **SYSTEMCONFIG** table will to be updated.
 - a. Update SCFGVALUE for the "QueueRQDeflds" SCFGVARIABLE with ruledef id for the new queue
 - b. Update SCFGVALUE for the "RuleRQDeflds" SCFGVARIABLE with ruledef id for the new rule
 - c. Also the following updates need to be performed if the user creates a new aggreagte type

- d. Update SCFGVALUE for the "AggregateRQDefIds" SCFGVARIABLE with ruledef id for the new aggregate
 - e. Update SCFGVALUE for the "AggregateMaps" SCFGVARIABLE with mapping between new aggregate ruledef id and new ruledef id.
4. Update the **TEMPLATES** and the **RULEDEFTEMPLATELINK** tables with the templates that will be used for the new rule and queue.
 - a. The RULEDEFTEMPLATELINK contains the link between ruledef id and templates used for that ruledef id.
 - b. The following sql can be used in order to get templates used for a specific ruledef id:


```
SELECT l.TemplateId,t.nType, t.sName,l.IOrderId,t.sData FROM templates t, ruledeftemplatelink l where t.ITemplateId=l.ITemplateId AND l.Iruledefid = ? ORDER BY t.nType,l.IOrderId
```
 - c. User can use the NRT templates or the Profile templates or the RT templates for the new rule and queue.
 - d. The user can perform any changes in these templates but he can not insert any new replace holder (tags).
 - e. The call of QUEUE procedure from the new rule's template footer has to be removed if user will not insert any templates for the new queue.
5. The **RULESUBTYPETEMPLATE** table was created in order to keep the connection between the rule/queue/aggregate subtype and template type (NRT, Profile, RT) used for it. The value for the TEMPLATESUBTYPEID column can be:
 - 0 - NRT templates
 - 1 - Profile templates
 - 2 - RT templates
6. The **RQVARDEFASSIGN**, **RQVARIABLES**, and **RQ_FIELDS** tables need to be updated with seed data for the new ruledef ids, otherwise no data will be displayed in the Transaction, Demographic and Variables dropdowns in the Rule Editor for the new rule type.

5.2 Near Real Time Oracle

Run create_custom_rule_types_oracle.sql script from additional_scripts.
 (additional_scripts.zip\Model_Upgrade\Upgrade\Additional_scripts\ORACLE\9000_9020\NRT\CREATE_CUSTOM_RULE_TYPES\create_custom_rule_types_oracle.sql)

5.3 Near Real Time MS SQL

Run `create_custom_rule_types_msq.sql` script from `additional_scripts`.

```
(additional_scripts.zip\Model_Upgrade\Upgrade\Additional_scripts\  
MSSQL\9000_9020\NRT\CREATE_CUSTOM_RULE_TYPES\create_custom_rule_types_msq.sql  
)
```

5.4 Update the “Custom Rule” Procedure

Depending on the use of Custom Rule Procedure in the entry procedures, some changes should be performed in the Custom Procedure for Rule Score and insert in the ALERTS, DETAIL_ALERTED_RULE and PROFILE_ALERTED_RULE tables.

There are few scenarios that should be considered:

5.4.1 Changes in the Custom Rule Procedure if both (NRT and Custom) Procedures are used in the Entry procedures

Before proceeding, set the order of execution of NRT and Custom rule in entry procedures. If NRT rules will be triggered first and then Custom Rule, Custom Rule templates need to be changed, otherwise NRT rule procedure.

Updates for Rule Score if Custom Rule uses as starting point NRT templates (otherwise no changes is needed)

Standard, Rule Score is initialized when Rule procedure is triggered. To avoid this behaviour, in Custom Rule procedure template (in TEMPALTES table) you have to make following changes

Change procedure's parameter **RULE_SCORE** from OUT to IN OUT (Rule Procedure Header)

Not initialize **RULE_SCORE** parameter to 0 (Rule Procedure Header 2)

Updates for insert in DETAIL_ALERTED_RULE if for Custom Rule uses as starting point NRT templates (Rule Procedure Footer 3). Change **insert** into `auto_action_pending` statement with **merge** statement. For example, in Oracle -

```
INSERT INTO auto_action_pending  
(action_level_id,  
action_level_key,  
rule_id,  
dd_partition_date,  
nd_key_hash,  
record_source_id,  
sd_tiebreaker,  
sm_fiid,  
sm_bin,  
alert_date)  
SELECT v_action_level_id,  
v_action_level_key,  
v_AutoActVar(v_index).tbAutoActionId,  
v_AutoActVar(v_index).tbRuleId,  
v_dd_partition_date,
```

```

v_nd_key_hash,
v_record_source_id,
v_sd_tiebreaker,
v_auto_sm_fiid,
v_auto_sm_bin,
v_Current_DateTime
FROM dual;

```

will be replaced with

```

merge into auto_action_pending a
using (SELECT v_action_level_id ACTION_LEVEL_ID,
v_action_level_key ACTION_LEVEL_KEY,
v_AutoActVar(v_index).tbAutoActionId Action_id,
V_autoactvar(v_index).tbruleid rule_id,
v_dd_partition_date DD_PARTITION_DATE,
v_nd_key_hash ND_KEY_HASH,
v_record_source_id RECORD_SOURCE_ID,
v_sd_tiebreaker SD_TIEBREAKER,
v_auto_sm_fiid sm_fiid,
v_auto_sm_bin sm_bin,
v_Current_DateTime alert_date
FROM dual
) s
On (a.ACTION_LEVEL_ID=s.ACTION_LEVEL_ID AND
a.ACTION_LEVEL_KEY=s.ACTION_LEVEL_KEY and
a.ACTION_ID=s.ACTION_ID AND

```

Change **insert** into detail_alerted_rule statement with **merge** statement. For example, in Oracle:

```

INSERT INTO detail_alerted_rule
(dd_partition_date,
nd_key_hash,
record_source_id,
sd_tiebreaker,
action_level_id,
action_level_key,
sd_rules,
alert_date,
alert_color,
nd_manual_act_stat,
bd_alertedscoring,
bd_alertedrule,
bd_iced,
ns_rulehits,
applicable_ques,
applicable_categories)
VALUES (v_dd_partition_date,
v_nd_key_hash,
v_record_source_id,
v_sd_tiebreaker,
v_action_level_id,
v_action_level_key,
v_rules,
v_current_datetime,

```

```

v_alertcolor,
v_manualact,
v_scorealerted,
v_brulealerted,
v_ice,
v_rulealert,
v_lrqaid || v_alerts,
v_alert_category_id );

```

Will be changed to

```

MERGE INTO detail_alerted_rule dar
using ( select v_dd_partition_date dd_partition_date,
             v_nd_key_hash nd_key_hash,
             v_record_source_id record_source_id,
             v_sd_tiebreaker sd_tiebreaker,
             v_action_level_id action_level_id,
             v_action_level_key action_level_key,
             v_rules sd_rules,
             v_current_datetime alert_date,
             v_alertcolor alert_color,
             v_manualact nd_manual_act_stat,
             v_scorealerted bd_alertedscoring,
             v_brulealerted bd_alertedrule,
             v_ice bd_iced,
             v_rulealert ns_rulehits,
             v_lrqaid||v_alerts applicable_ques,
             v_alert_category_id applicable_categories
             from dual
         ) s
on ( dar.dd_partition_date = s.dd_partition_date
    and dar.nd_key_hash=s.nd_key_hash
    and dar.record_source_id=s.record_source_id
    and dar.sd_tiebreaker=s.sd_tiebreaker
    and dar.action_level_id=s.action_level_id )
when not matched then
INSERT (dd_partition_date,
        nd_key_hash,
        record_source_id,
        sd_tiebreaker,
        action_level_id,
        action_level_key,
        sd_rules,
        alert_date,
        alert_color,
        nd_manual_act_stat,
        bd_alertedscoring,
        bd_alertedrule,
        bd_iced,
        ns_rulehits,
        applicable_ques,
        applicable_categories)
VALUES (v_dd_partition_date,
        v_nd_key_hash,
        v_record_source_id,

```

```

v_sd_tiebreaker,
v_action_level_id,
v_action_level_key,
v_rules,
v_current_datetime,
v_alertcolor,
v_manualact,
v_scorealerted,
v_brulealerted,
v_ice,
v_rulealert,
v_lrqaid||v_alerts,
v_alert_category_id )

```

when matched then

```

update set action_level_key=s.action_level_key,
sd_rules=sd_rules||s.sd_rules,
alert_date=s.alert_date,
alert_color=s.alert_color,
nd_manual_act_stat=s.nd_manual_act_stat,
bd_alertedscoring=bd_alertedscoring+
    s.bd_alertedscoring,
bd_alertedrule=(case when bd_alertedrule=0
    then s.bd_alertedrule
    else bd_alertedrule end),
bd_iced=(case when bd_iced=0
    then s.bd_iced
    else bd_iced end),
ns_rulehits=ns_rulehits+s.ns_rulehits,
applicable_ques=v_LRQAID||s.applicable_ques,
applicable_categories=applicable_categories
    ||s.applicable_categories ;

```

For MS SQL Server change MERGE statement accordingly

Updates for insert in ALERTS, PROFILE_ALERTED_RULE if for Custom Rule uses as starting point PROFILE templates. Update steps are similar with steps described in “Updates for insert in DETAIL_ALERTED_RULE if for Custom Rule uses as starting point NRT templates” paragraph. It is needed to replace “insert into auto_action_pending” statement with merge statement (Rule Procedure Footer 3).

For example, in Oracle:

```

INSERT INTO auto_action_pending
(action_level_id,
action_level_key,
rule_id,
dd_partition_date,
nd_key_hash,
record_source_id,
sd_tiebreaker,
sm_fiid,
sm_bin,
alert_date)
SELECT v_action_level_id,

```



```

v_action_level_key,
v_AutoActVar(v_index).tbAutoActionId,
v_AutoActVar(v_index).tbRuleId,
v_dd_partition_date,
v_nd_key_hash,
v_record_source_id,
v_sd_tiebreaker,
v_auto_sm_fiid,
v_auto_sm_bin,
v_Current_DateTime
FROM dual;

```

will be replaced with:

```

merge into auto_action_pending a
using (SELECT v_action_level_id ACTION_LEVEL_ID,
v_action_level_key ACTION_LEVEL_KEY,
v_AutoActVar(v_index).tbAutoActionId Action_id,
V_autoactvar(v_index).tbruleid rule_id,
v_dd_partition_date DD_PARTITION_DATE,
v_nd_key_hash ND_KEY_HASH,
v_record_source_id RECORD_SOURCE_ID,
v_sd_tiebreaker SD_TIEBREAKER,
v_auto_sm_fiid sm_fiid,
v_auto_sm_bin sm_bin,
v_Current_DateTime alert_date
FROM dual
) s
On (a.ACTION_LEVEL_ID=s.ACTION_LEVEL_ID AND
a.ACTION_LEVEL_KEY=s.ACTION_LEVEL_KEY and
a.ACTION_ID=s.ACTION_ID AND
a.RULE_ID=s.RULE_ID AND
a.DD_PARTITION_DATE=s.DD_PARTITION_DATE AND
a.ND_KEY_HASH=s.ND_KEY_HASH AND
a.RECORD_SOURCE_ID=s.RECORD_SOURCE_ID AND
a.SD_TIEBREAKER=s.SD_TIEBREAKER
)
when not matched then

```

```

insert (action_level_id,
action_level_key,
action_id,
rule_id,
dd_partition_date,
nd_key_hash,
record_source_id,
sd_tiebreaker,
sm_fiid,
sm_bin,
alert_date)
VALUES (s.action_level_id,
s.action_level_key,
s.action_id,
s.rule_id,
s.dd_partition_date,

```

```

s.nd_key_hash,
s.record_source_id,
s.sd_tiebreaker,
s.sm_fiid,
s.sm_bin,
s.alert_date)
when matched then
  update set sm_fiid=s.sm_fiid,
            sm_bin=s.sm_bin,
            alert_date=s.alert_date ;

```

Change insert into

5.4.2 Changes in Custom Rule Procedure if only Custom Rule Procedure is used in Entry Procedures

5.4.2.1 *Updates for Rule Score if Custom Rule uses as starting point NRT templates -*

No change is needed

5.4.2.2 *Updates for insert in ALERTS, DETAIL_ALERTED_RULE if for Custom Rule uses as starting point NRT templates*

No change is needed.

5.4.2.3 *Updates for insert in ALERTS, PROFILE_ALERTED_RULE if for Custom Rule uses as starting point PROFILE templates*

No change is needed.

5.4.2.4 *Update the entry procedures to use the “Custom Rule” procedure*

5.4.2.4.1 Changes in Custom Rule Procedure if both (NRT and Custom) procedures are used in the entry procedures

5.4.2.4.2 In PP_NRT_ENTRY_XXX entry procedure, identify all action level assigned and clone variables <ACTION_LEVEL>_RULE_NAME to <ACTION_LEVEL>RULE_CUST_NAME

for example, in Oracle -

```
v_Card_rule_name VARCHAR2(30);
```

will be cloned as

```
v_Card_rule_cust_name VARCHAR2(30);
```

Assign rules procedure name into variable

For example, in Oracle - Change

```
IF v_Card_IsAssigned = 1 AND v_Card_portfolio IS NOT NULL THEN
```

```

        v_Card_rule_name := 'PP_NRT_RULES_' || TRIM (v_Card_portfolio) ;
        v_Card_NRT_RULE_SCR := 0;
    END IF;

```

with

```

    IF v_Card_IsAssigned = 1 AND v_Card_portfolio IS NOT NULL THEN
        v_Card_rule_name := 'PP_NRT_RULES_' || TRIM (v_Card_portfolio) ;
        v_Card_rule_cust_name := 'PP_POC_RULES_' || TRIM (v_Card_portfolio) ;

        v_Card_NRT_RULE_SCR := 0;
    END IF;

```

In Run near real time Rules paragraph, after calling NRT rule procedure call CUST rule procedure. Take in consideration that for Customer rule NRT_RULE_SCR variable is IN OUT parameter (value of rule score is passed from NRT RULE to CUST _RULE), change code accordingly.

For example, in Oracle –

after

```

EXECUTE IMMEDIATE 'Begin ' || v_Card_rule_name || '('

```

Call custom rule procedure:

```

EXECUTE IMMEDIATE 'Begin ' || v_Card_rule_cust_name || '('

```

Identify in call procedure statement RULE_SCORE and change it

ex: Change

```

        OUT v_Card_NRT_RULE_SCR, /* 8 */

```

To:

```

        IN OUT v_Card_NRT_RULE_SCR, /* 8 */

```

5.4.2.5 *Changes in Custom Rule Procedure if only Custom Rule procedure is used in entry procedures:*

In entry procedure change for all rule name variable initialization name of rule procedure

For example, in Oracle: update entry script replacing 'PP_NRT_RULES_' with 'PP_POC_RULES_'

Replace

```

    IF v_Card_IsAssigned = 1 AND v_Card_portfolio IS NOT NULL THEN
        v_Card_rule_name := 'PP_NRT_RULES_' || TRIM (v_Card_portfolio) ;
        v_Card_NRT_RULE_SCR := 0;
    END IF;

```

with

```

    IF v_Card_IsAssigned = 1 AND v_Card_portfolio IS NOT NULL THEN
        v_Card_rule_name := 'PP_POC_RULES_' || TRIM (v_Card_portfolio) ;

```

```
        v_Card_NRT_RULE_SCR := 0;  
    END IF;
```

6 Adding Search Fields

6.1 Overview

This section describes the modifications necessary for adding new search fields to a PRM action level. A use case is also included, exemplifying the addition of one new search field.

6.2 Steps to Add Search Fields

Search screen form seed:

Add the label informations:

- *insert into PRM_UI_COMPONENT*

Add the new field:

- *insert into PRM_UI_COMPONENT*

Add the PRM_FIELD record:

- *insert into PRM_FIELD*
- *insert into field_action_level*
- *insert into PRM_UI_FIELD*

Add the new search field restrictions:

- *insert into PRM_UI_FIELD_SEARCH*

Update to the database view used to populate the search screen grid:

- *drop view PV_<ACTION_LEVEL_NAME>_MASTER*
- *create view PV_<ACTION_LEVEL_NAME>_MASTER*

Search screen result grid seed:

Add the new search field to the grid:

- *insert into PRM_UI_COMPONENT*
- *insert into PRM_FIELD*
- *insert into PRM_UI_FIELD*

NOTE: Please check section 5.5.3 for a detailed example on how to add a new search field.

6.2.1 Changing the wildcard and field combination restrictions

It is enough to adjust the corresponding row (of an existing search field) in the PRM_UI_FIELD_SEARCH table in order to change the wildcard and field combination restrictions for the search field.

PRM_UI_FIELD_SEARCH columns meaning:

MAXCHAR - This column contains the maximum number of characters that the user can enter in the input fields from the Search screens.

SEARCH_ASSOCIATION - This column specifies if a field that can be used in the Search screen (to search for action level keys) has an associated field. If a field has an associated field and the user searches for all action level keys with a given value for the original field, then the UI will also display all action level keys which have the same searched value for the associated field. For example, the SM_PRI_FIRST_NAM field has SM_SEC_FIRST_NAM as an associated field, and when the user searches for all accounts for which "First name" is "John" (that is, SM_PRI_FIRST_NAM = 'John'), the UI will also return all accounts for which "Secondary First Name" is 'John' (that is, SM_SEC_FIRST_NAM = 'John').

VALIDATION_DEPENDENCY - This column contains a name of the database field which will also be required to be provided so the search can be executed.

VALIDATION_WILDCARD - This column contains a numeric value which will be the minimum number of characters needed to perform the search.

6.3 Use Case – Adding One New Search Field

Use case description: Add a new search field (SM_PRI_EMAIL_ADDR) to the Card action level search screen. The following search restrictions should be created:

- validation dependency: SM_USER_HOME_PHN
- validation wildcard: 4 characters
- search association: none
- maximum length of the UI input text search field: 20 characters

Solution: There are three main tasks to be performed in order to update the search screen as per the above request. These tasks are listed below. Each task is followed by the name of an example script that can be used to perform the task. The example scripts listed below are included in the PRM 8.7.0.0 – Documentation.zip file in a folder named ExampleSearchScreenExtension.

Search screen form seed example script:

exampleSearchScreenExtension_add_seed_search_screen_form.sql

Update to the database views example script:

exampleSearchScreenExtension_update_view_search_screen_grid.sql

Search screen result grid seed example script:

exampleSearchScreenExtension_add_seed_search_screen_result_grid.sql

Note: The example script files listed above can be modified in order to add other search fields. The example scripts were developed specifically for SQL Server, and should be reviewed and adapted, if necessary, before use with Oracle.

6.4 Performance Impact

Observations:

- The Date UI search fields are practically unusable – the search will match only the exact Timestamp passed in the UI.

- The Currency and Number UI search fields are passed to the DB as Strings; beside the Date UI fields, all the rest are passed as Strings to the DB. This will cause performance impact described below.
- All the UI search fields are passed to the DB for the actual search using the “LIKE” operator, regardless of the UI search field types. Here is an example of the query generated by the PRM:

```
SELECT TOP 50 pv_CARD_MASTER.*
FROM pv_Card_MASTER
WHERE sm_user_home_phn LIKE ?
      AND sm_user_mob_phn LIKE ?
      AND dm_user_pri_dob LIKE ?
      AND mm_auth_lmt LIKE ?
      AND nm_crd_num_iss LIKE ?
      AND IPortfolioId IN (?)
ORDER BY SM_PRI_LAST_NAM_NOT_NULL ASC , SM_PAN ASC
```

Note that not all the fields used in the query are varchar, and neither all the passed values are containing wildcards.

Restrictions:

If you add a numeric (money) or date (datetime) field in search screen, the query performance can't be improved by adding these columns to an index, because the index will be skipped, and scanning the entire table.

This behavior is caused:

- When a numeric value is sent (to query command) as string, it force the database to perform an implicit conversion (cast), by adding TO_CHAR function on numeric column.
- When a timestamp value is sent (to query command) as string, it force the database to perform an implicit conversion (cast), by adding INTERNAL_FUNCTION function on datetime column.
- When a string value is sent (to query command), if wildcard character are used before searched value (for example: '%abc'), the database optimizer will choose to scan the entire table instead of index scan.

Improvement (query performance tuning):

The query performance can be improved by creating index on searched columns.

Attention: It is recommended to add only string columns in search screen, because of previous restrictions from numeric and datetime columns (and wildcard string columns).

Increasing the number of indexes or increasing the number of columns to an index, will decrease the performance at the demographic data load (insert/update/delete).

For example, if one or two columns are added in search screen, you can add them to an existing index. This requires a DBA operation to remove the old index and create back with the additional columns.

Attention: Do not alter the primary key index (by adding new columns), since it's used by demographic data load (and will affect the performance).

Example of changing existing index by adding new columns filter (used from Search screen):
SM_SEC_FIRST_NAM, SM_SEC_LAST_NAM to existing columns index (SM_PRI_LAST_NAM, SM_PRI_FIRST_NAM).

Oracle:

```
DROP INDEX CARD_MASTER_IX1
```

```
CREATE INDEX CARD_MASTER_IX1 ON CARD_MASTER (SM_PRI_LAST_NAM,  
SM_PRI_FIRST_NAM, SM_SEC_FIRST_NAM, SM_SEC_LAST_NAM) TABLESPACE  
<%TS_CARDMASTERINDEX%>
```

Mssql:

```
DROP INDEX CARD_MASTER_IX1
```

```
CREATE INDEX [CARD_MASTER_IX1] ON [CARD_MASTER] ([SM_PRI_LAST_NAM],  
[SM_PRI_FIRST_NAM], [SM_SEC_FIRST_NAM], [SM_SEC_LAST_NAM]) ON  
[<%TS_CARD_MASTER_IDX%>]
```

If there are multiple columns (in search screen), you can group them in a separate new index (used only by Search screen), where the column position in this index it's important and based on the number of usage of each column and in combination with other columns in the search condition.

Example of new index creation for new columns filter (added in Search screen): SM_SEC_FIRST_NAM, SM_SEC_LAST_NAM.

Oracle:

```
CREATE INDEX CARD_MASTER_IX10 ON CARD_MASTER (SM_SEC_FIRST_NAM,  
SM_SEC_LAST_NAM) TABLESPACE <%TS_CARDMASTERINDEX%>
```

Mssql:

```
CREATE INDEX [CARD_MASTER_IX10] ON [CARD_MASTER] ([SM_SEC_FIRST_NAM],  
[SM_SEC_LAST_NAM]) ON [<%TS_CARD_MASTER_IDX%>]
```

Fine tuning:

On MsSql, if some new search columns (added with UI seed in Search screen) are used only to be displayed in UI, and not to search on them, the new index can only include these columns instead of adding (to be built on them).

For example:

```
DROP INDEX CARD_MASTER_IX1
```

```
CREATE INDEX [CARD_MASTER_IX1] ON [CARD_MASTER] ([SM_PRI_LAST_NAM],  
[SM_PRI_FIRST_NAM]) INCLUDE ([SM_SEC_FIRST_NAM], [SM_SEC_LAST_NAM]) ON  
[<%TS_CARD_MASTER_IDX%>]
```

On MsSql and Oracle, if some searched columns are usually sorted in descending order, the index creation can include this order when it's built.

Default column order on index creation is ASC (ascending), even if it's not specified.

For example, on MsSql:

```
DROP INDEX CARD_MASTER_IX1
```

```
CREATE INDEX [CARD_MASTER_IX1] ON [CARD_MASTER] ([SM_PRI_LAST_NAM],  
[SM_PRI_FIRST_NAM]) INCLUDE([SM_SEC_FIRST_NAM] DESC, [SM_SEC_LAST_NAM] DESC) ON  
[<%TS_CARD_MASTER_IDX%>]
```

Attention: When the index is created on column order, it's important to understand that if UI (search screen) requires opposite order than the column order (from index), the query performance will be decreased. For example, if the column order (in the index) is descending, and UI search requires data in ascending order, the query will take longer to perform.

7 Adding an Action, Auto-Action or Auto-Add-In

7.1 Overview

This section describes the modifications necessary to add an action to a PRM action level review screen, and an auto action or auto add-in to PRM rules.

7.2 Adding a New Action to an Existing Action Level Review Screen

Review screen actions are related to toolbar buttons on the review screen that perform a function related to the open alert/review or other action level keys within the transactions for the open action level alert/review. Some of the standard actions a user can perform via the toolbar buttons on the action level alert/review screen are: block, watch, ice, call, remind, forward, and print. Customers may also add their own actions and action buttons to the alert/review screen.

7.2.1 Define Localization (L10N) Properties

Define L10N properties for:

the sText for the action type (ACTIONSTYPE.STEXT)

the new action's permission name (PRM_PERMISSION.DISPLAY_NAME)

the new action's permission description (PRM_PERMISSION.DESCRPTION)

Note that new entries have to be added to both L10N_PROPERTY and L10N_PROPERTY_ADDITIONAL_INFO tables.

	LOCALE_ID	L10N_PROPERTY_ID	L10N_VALUE
31709	en_US	800015	Action Required 3.3
31710	en_US	800016	Action Required 4.1
31711	en_US	800017	Action Required 4.2
31712	en_US	800018	Action Required 4.3
31713	en_US	800019	New action
31714	en_US	900001	Custom Information
31715	en_US	900002	Custom Information Action Type
31716	en_US	900003	Custom Information

In this example, 900001, 900002 and 900003 are used.

Note: Remember the good practice for ID allocation documented in section [1.4](#).

7.2.2 Add a Permission for the Action

Define a new permission for the action in the PRM_PERMISSIONS table as shown below. The PRM_PERMISSION.DISPLAY_NAME and PRM_PERMISSION.DESCRPTION are L10N properties

whose text was defined in the L10N_PROPERTY table in step M1.1. The PERMISSION_GRP_ID has to be 4 (i.e. the 'Toolbar Button' permission group). The ASGN_BY_DFLT has to be 0.

	PERMISSION_ID	DISPLAY_NAME	DESCRIPTION	PERMISSION_GRP_ID	ASGN_BY_DFLT
85	93	255093	260093	3	0
86	96	255096	260096	3	2
87	97	255097	260097	9	2
88	98	255098	260098	9	2
89	100	255100	260100	5	2
90	101	255101	260101	5	2
91	102	255102	260102	5	2
92	201	900003	900002	4	0

Note: Remember the good practice for ID allocation documented in section [1.4](#).

7.2.3 Define a New Action Type

Define a new action in the ActionType table. In this example, the action is called "CustomInformation". The ACTIONTYPE.STEXT column uses the L10N_PROPERTY.L10N_PROPERTY_ID 900001 created in the previous step. Auto actions are disabled (0) by default.

	LTYPE	NAME	STEXT	AUTO_ACTION	AUTO_ACTION_PROC	PERMISSION_ID	MULTIPLE_ACTION_LEVELS_ENABLED
27	28	ResponseRequired	145029	0	NULL	73	0
28	29	CrossReference	145030	0	NULL	43	0
29	50	Card Auto Add in	800000	2	NULL	NULL	0
30	51	Employee Auto Add in	800001	2	NULL	NULL	0
31	52	UAN Auto Add in 1	800002	2	NULL	NULL	0
32	53	UAN Auto Add in 2	800003	2	NULL	NULL	0
33	200	CustomInformation	900001	0	NULL	201	0

Note: Remember the good practice for ID allocation documented in section [1.4](#).

7.2.4 Verify the Action and Permission Settings

Verify the Action and Permission Settings using the following SQL:

```
select a.lType, a.Name, p.PERMISSION_ID, coalesce(L1.L10N_VALUE, (SELECT
NA.L10N_VALUE FROM L10N_PROPERTY NA
WHERE NA.L10N_PROPERTY_ID = 0 AND NA.LOCALE_ID = 'en_US')) AS
DISPLAY_NAME,
coalesce(L2.L10N_VALUE, (SELECT NA.L10N_VALUE
FROM L10N_PROPERTY NA
WHERE NA.L10N_PROPERTY_ID = 0 AND NA.LOCALE_ID
= 'en_US')) AS DESCRIPTION,
pg.PERMISSION_GRP_ID,
```

```

        coalesce(L3.L10N_VALUE, (SELECT NA.L10N_VALUE
FROM L10N_PROPERTY NA
        WHERE NA.L10N_PROPERTY_ID = 0 AND NA.LOCALE_ID
= 'en_US')) AS PERMISSION_GRP_DISPLAY_NAME

from prn_permission p

INNER JOIN ACTIONSTYPE a

    ON p.PERMISSION_ID = a.PERMISSION_ID

INNER JOIN PRM_PERMISSION_GRP pg

    ON p.PERMISSION_GRP_ID = pg.PERMISSION_GRP_ID

INNER JOIN L10N_PROPERTY L1

    ON p.DISPLAY_NAME = L1.L10N_PROPERTY_ID

    AND L1.LOCALE_ID = 'en_US'

INNER JOIN L10N_PROPERTY L2

    ON p.DESCRPTION = L2.L10N_PROPERTY_ID

    AND L2.LOCALE_ID = 'en_US'

INNER JOIN L10N_PROPERTY L3

    ON pg.DISPLAY_NAME = L3.L10N_PROPERTY_ID

    AND L2.LOCALE_ID = 'en_US'

where p.PERMISSION_ID = 201

```

Results		Messages						
	IType	Name	PERMISSION_ID	DISPLAY_NAME	DESCRIPTION	PERMISSION_GRP_ID	PERMISSION_GRP_DISPLAY_NAME	
1	200	CustomInformation	201	Custom Information	Custom Information Action Type	4	Toolbar Button	

7.2.5 Add the Action Level Configurations to the Actions Type Table

Add the action level configuration to the ACTIONSTYPE_ACTION_LEVEL table. Our example will define the action for the Card action level and it will use the same L10N_PROPERTY.L10N_PROPERTY_ID used for the STTEXT column in the ActionType table (see M1.3). The AUTO_UPDATES_ALLOWED column should be set to 0 to indicate that the action may only be updated once, or to 1 to indicate that it should update each time the rule fires. The ACTIONSTYPE_ACTION_LEVEL.SHOWSTATE column indicates whether the MasterDisp table will be updated and if they review screen button will show the current state (0 = do not show state and 1 = show the state). The

ACTIONSTYPE_ACTION_LEVEL.SHOWSTATUS column indicates whether the action's toolbar button will indicate status (0 = do not show status and 1 = show status).

	ACTION_TYPE_ID	ACTION_LEVEL_ID	STEXT	SHOWSTATE	SHOWSTATUS	TITLEBARSTRING	HASAUTOTEXT	UPDATES_ALLOWED	AUTO_UPDATES_ALLOWED
129	28	3	145029	0	0	140029	0	0	1
130	28	4	145029	0	0	140029	0	0	1
131	28	5	145029	0	0	140029	0	0	1
132	50	1	800000	0	0	800000	0	NULL	NULL
133	51	5	800001	0	0	800001	0	NULL	NULL
134	52	3	800002	0	0	800002	0	NULL	NULL
135	53	3	800003	0	0	800003	0	NULL	NULL
136	200	1	900001	0	0	900001	0	1	0

7.2.6 Define the ActionsText L10N Property Values for the Actions Tab & Actions Report Text

Define the text that will appear in the actions tab for the different states and statuses for the action in the L10N_PROPERTY table for the ACTIONSTEXT table. This text will also be used for action reports.

Note that new entries have to be added to both L10N_PROPERTY and L10N_PROPERTY_ADDITIONAL_INFO tables.

Note: Remember the good practice for ID allocation documented in section [1.4](#).

	LOCALE_ID	L10N_PROPERTY_ID	L10N_VALUE
3...	en_US	800018	Action Required 4.3
3...	en_US	800019	New action
3...	en_US	900001	Custom Information
3...	en_US	900002	Custom Information Action Type
3...	en_US	900003	Custom Information
3...	en_US	900004	Custom Information Req
3...	en_US	900005	Custom Information Sbm
3...	en_US	900006	Custom Information Failed

7.2.7 Define the ActionsText Values for the Actions Tab and Actions Report Text

Define the text that will be displayed for the action in the Actions tab on the Action Level's review screen in the ACTIONSTEXT table. This text will also appear on action reports. Please note that internationalize text will need to be added for the ACTIONSTEXT.STEXT column in the same manner as was done in the section "Adding a New Action to an Existing Action Level Review Screen".

One entry is needed in the ACTIONSTEXT table for each ACTION_LEVEL_ID, LTYPE (action type), LSTATE, LSTATUS combination with a unique LID for each potential state and status that the action may have.

	LID	LTYPE	LSTATE	LSTATUS	SNAME	STEXT	SIMAGENAME	ACTION_LEVEL_ID	REVIEW_SCREEN
561	241	28	1	1	ResponseRequiredReq	150241	NULL	3	0
562	242	28	1	0	ResponseRequired	150242	NULL	3	0
563	243	28	1	2	ResponseRequiredFail	150243	NULL	3	0
564	244	28	1	3	ResponseRequiredSbmt	150244	NULL	3	0
565	2000	200	1	0	CustomerInformation	900001	NULL	1	0
566	2001	200	1	1	CustomerInformationReq	900004	NULL	1	0
567	2002	200	1	2	CustomerInformationFailed	900006	NULL	1	0
568	2003	200	1	3	CustomerInformationSbmt	900005	NULL	1	0

7.2.8 Add the Toolbar Button

Define a new Toolbar Button for the Action Level Review screen by adding an entry in the ToolbarButtons table.

Note: Remember the good practice for ID allocation documented in section [1.4](#).

	ID	NAME	CAPTION	ICON	HOTKEY	DISPLAYMNEMONIC	TOOLTIP	ALTTOOLTIP
33	49	linkedAccounts	NULL	linkedAccoun...	0	NULL	//Displays linked ac...	NULL
34	59	ResponseRequired	NULL	HeldTransacti...	NULL	NULL	//ResponseRequired	NULL
35	60	EditButton	NULL	editButton.gif	69	NULL	//E dit	NULL
36	61	AddButton	NULL	addButton.gif	65	NULL	//Add	NULL
37	62	DeleteButton	NULL	deleteButton.gif	88	NULL	//Delete	NULL
38	63	ALMemoButton	NULL	actionLevelM...	76	NULL	//Action Level Memo	NULL
39	64	CustomActionButton	NULL	customAction...	NULL	NULL	//Custom Action	NULL

FORMTYPE	ACTIONTYPE	READONLYENABLED	VISIBLE	ENABLED	SELECTABLE	DEFAULTORDER	SERVLETNAME	LISTENER	CLASSNAME	TYPE	PERMISSION_ID
0	NULL	0	1	1	0	3		NULL	NULL	3	156
0	21	1	1	1	0	6		NULL	NULL	1	46
0	200	0	1	1	1	100	DoAction	NULL	com.aciworl...	1	201

Make sure the file specified in the ICON column is accessible in the classpath of the PRMClient in the following package: `com.aciworldwide.prm.images`.

The CLASSNAME column should contain the full name of a class implementing the `com.aciworldwide.prm.interfaces.PrmAddin` interface. When the toolbar button is pressed the `execute()` method of this class will be called.

7.2.9 Add the Action Dialog Screen

Create a Swing dialog for the action based on the `ActionDialog` class in `PrmClient`, similar to `Memo`. Or, they can create their own dialog. If creating a new dialog without using `Action Dialog`, it should extend

PrmDialog. Add the new class to the PRM class path so that it may be found. (See the PRM installation guide for further details.)

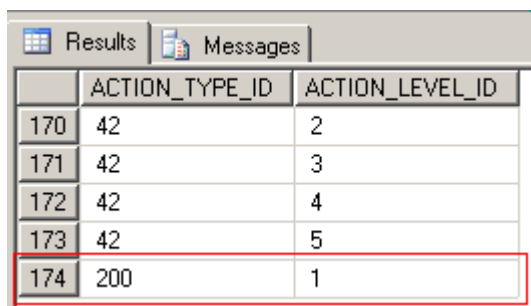
7.2.10 Restart the Application Server

The application server must be restarted, to apply the changes made in the previous steps.

7.2.11 Assign the Action to the Action Level

You need to assign the newly created action type to one or more action levels. The toolbar button of the action will be visible only on the review screens of the action levels to which the action is assigned.

If you want to assign the newly created action to one of the default action levels, a new row should be inserted in the ASSGND_ACTIONSTYPE_ACTIONLEVEL table representing the association between the action type and the action level. In our example we assign our new action type with the Card action level:



	ACTION_TYPE_ID	ACTION_LEVEL_ID
170	42	2
171	42	3
172	42	4
173	42	5
174	200	1

If you want to assign the newly created action to a custom action level you can either insert a new row in the ASSGND_ACTIONSTYPE_ACTIONLEVEL table (as described above) or go to the Action Level Configuration Screen using an administrator account and make the change in the “Assigned Actions” section.

7.2.12 Assign the Action to One or More Roles

You need to assign the newly created action type to one or more roles. Only the users having one of these roles will see the action toolbar button as enabled. For the other users the toolbar button will be disabled.

If you want to assign the newly created action to one of the default roles, a new row should be inserted in the PRMROLEPERMS table representing the association between the role, the action permission and the action level. In our example we assign our new action permission to the Team Administrator role and for the Card action level:

	ROLEID	PERMISSION_ID	ACTION_LEVEL_ID	REVIEW_SCREEN_TYPE
788	2	157	4	0
789	0	157	5	0
790	1	157	5	0
791	2	157	5	0
792	2	201	1	0

If you want to assign the newly created action to a custom role you can either insert a new row in the PRMROLEPERMS table (as described above) or go to the Role Configuration screen using an administrator account and assign the action permission to the desired role.

7.3 Adding an Auto-Action for Use in a PRM rule

The Auto Action functionality for 8.0 has been expanded to include other actions in addition to Auto Block. Memo, Watch, Call, and Ice will also be supported. Customers may define their own auto action. The image below shows how tabs are added when an auto action is added. The user may define a maximum of 3 auto actions (and/or auto addins – see the next sections) for a rule.

The image shows three sequential screenshots of the 'Auto Action' dialog box. The first screenshot shows the 'Block' tab with fields for 'Auto Action' (Block), 'User' (AutoUser1), 'Aggregate' (none), and 'Block Code' (Stolen). The second screenshot shows the 'Watch' tab with fields for 'Auto Action' (Watch), 'User' (AutoUser1), 'Aggregate' (empty), and 'Block Code' (empty). A red arrow points to the 'Aggregate' field. The third screenshot shows the 'Ice' tab with fields for 'Auto Action' (Ice), 'User' (AutoUser1), 'Aggregate' (empty), and 'Ice until' (5/12/04 10:18:36 AM). A red arrow points to the 'Aggregate' field. Each screenshot has 'OK', 'Cancel', and 'Help' buttons at the bottom right.

7.3.1 Add L10N Properties Text for ActionType and ActionType_Action_Level Tables

Following the same methods as defined in Section 7.2.1, add the text for the action that will be shown in the Actions tab on the Action Level review screen and the action reports.

7.3.2 Add a New Action Type

Add a new action type for the action in the ActionType table using the instructions from 6.2.3 with the exception that a permission ID is not required if the action is not used on the review screen. Auto actions are invoked by the system during rule processing and do not require a permission to run. Also, the auto

action requires that the ACTIONSTYPE_ACTION_LEVEL.UPDATES_ALLOWED and ACTIONSTYPE_ACTION_LEVEL.AUTO_UPDATES_ALLOWED columns be set.

The ACTIONSTYPE.AUTO_ACTION column should be set to 1 to indicate that the action is available for auto action (by default it is set to 0).

ACTIONSTYPE.AUTO_ACTION column has the following values:

- 0 = no - Not enabled for auto-actions
- 1 = yes - Enabled for auto-actions
- 2 = Auto Addin - Enabled for auto add-in actions

The ACTIONSTYPE_ACTION_LEVEL.AUTO_UPDATES_ALLOWED indicates whether or not records in the MasterDisp table for actions with state can be updated by Auto-Action rules performing the same action on the same account. Valid values are as follows:

- 0=Records in the MasterDisp table can not be updated by Auto-Action rules performing the same action that has state or the same state. If EMM is enabled, the record in the EMD table will only be updated if the Auto-Action state is different than the state of the record in the EMD table and the record in the EMD table has a status=0 (i.e. 'completed').
- 1= Records in the MasterDisp table can be updated by Auto-Action rules performing the same action that maintains state for the same state. If EMM is enabled, a record may be inserted into the EMD table regardless of the state or status of existing EMD records.

ACTIONSTYPE_ACTION_LEVEL. UPDATES_ALLOWED — When an action recorded in the ACTIONSTYPE_ACTION_LEVEL table has state (ACTIONSTYPE_ACTION_LEVEL.SHOWSTATE = 1), this indicates whether or not a manual action may update the MASTERDISP table when the update is for the same state (e.g., updating a block code on an already blocked action).

If UPDATES_ALLOWED = 1 then a manual action with state will successfully execute even if the update is for the same state. Upon a successful action execution the MASTERDISP table will be updated and if External Message Manager is enabled, then new entries will be inserted into the EXTERNAL_MESSAGE_DATA table also.

If UPDATES_ALLOWED = 0 then a manual action with state will be successfully executed only if the new state is different than the current state or, if they are equal, the status of the action is 0 (i.e. 'completed'). Upon a successful action execution the MASTERDISP table will be updated and if External Message Manager is enabled, then new entries will be inserted into the EXTERNAL_MESSAGE_DATA table also.

AUTO_ADD_IN_CONFIG.CONFIG_TYPE_ID has the following values:

- 0 = Action Type
- 1 = Required Value 1
- 2 = Required Value 2
- 3 = Required Value 3

- 4 = Required Value 4

See the section below for more details about AUTO_ADD_IN_CONFIG.

When an auto-action record is created, a record is inserted into the RuleQue table for the rule and a record is inserted into the AUTOACTIONSRULE table for each of the up to three potential auto-actions that can be defined for a rule. The ACTION_ORDER column of the AUTOACTIONSRULE table indicates the whether the auto-action is the first, second or third auto-action defined for the rule.

For following table indicates which columns in the AUTOACTIONSRULE table are required for each type of action in addition to the AUTOACTIONSRULE .LID, AUTOACTIONSRULE .LRULEID, AUTOACTIONSRULE .LTEAMID and AUTOACTIONSRULE .ACTION_ORDER values.

Block	x	x	x	x
Watch	x	x		x
Ice	x	x		x
Call	x			x
Memo	x			x
Case Create	x			x
Add-In Action	x			x

If an auto-action rule fires for a transaction, the rule ID is stored in a local variable in the rules procedure. After all of the deployed rules have been evaluated, the rules procedure inserts a record into the AUTO_ACTION_PENDING table for each of the auto-action rules that fired. The PP_AUTO-ACTION procedure is called at the end of the rules procedure if at least one auto-action rule fired.

7.3.3 Actions Text for the Action

Define the text for the auto action based on the instructions in sections 7.2.6 and 7.2.7.

7.4 Adding an Auto Add-In Action

7.4.1 Overview

An Auto Add-in action is a form of auto action that intended for use with a host system such as a case management system or other 3rd party system. It allows communication with the host system via the External Message Manager functionality while allowing configurable entry fields to be passed to the host system. The screen below shows an example of a possible ACM case management system screen.

The AUTO_ADD_IN_CONFIG table contains the auto add-in information that will be used during the auto add-in rules creation.

Auto Action

Block: ACM

Auto Action: ACM

User: AutoUser1

Aggregate: none

Action Type: ACM MOTO

Action Required 1: Debit

Action Required 2: MOTO

Action Required 3: 3

Action Required 4: 5

Enter action description

Buttons: OK, Cancel, Help, Add Auto Action, Remove Auto Action

Auto Action

Block: ACM

Auto Action: ACM

User: AutoUser1

Aggregate: none

Action Type: ACM MOTO

Action Required 1: Debit

Action Required 2: MOTO

Action Required 3: 3

Action Required 4: 5

Enter action description

Buttons: OK, Cancel, Help, Add Auto Action, Remove Auto Action

The configurable information on the screen (circled in red above), includes the Action Type, and up to 4 Action Required fields. If all four fields have not configured, then only those configured will show on the tab.

AUTO_ADD_IN_CONFIG.CONFIG_TYPE_ID has the following values:

- 0 = Action Type
- 1 = Required Value 1
- 2 = Required Value 2
- 3 = Required Value 3
- 4 = Required Value 4

AUTO_ADD_IN_CONFIG.ACTION_TYPE_ID — This column contains the action type. It is linked to the LTYPE column in the ActionsTYPE table.

The dropdown boxes of the Auto Action pop-up screen would be populated with data from the AUTO_ADD_IN_CONFIG table. The "Action Type" would be stored in the AUTOACTIONSRULE.AUTO_ADD_IN_TYPE column.

The four "Action Required" selections correspond to records in the AUTO_ADD_IN_CONFIG table. The user selections for the "Action Required 1", "Action Required 2", "Action Required 3" and "Action Required 4" drop down boxes will be stored in the AUTO_ADD_IN_RULE_CONFIG associative table. Each drop down box is associated with an Action_Type_ID in the AUTO_ADD_IN_CONFIG table and each ACTION_TYPE_ID can have multiple records in the AUTO_ADD_IN_CONFIG table. Each record represents a value in the action type drop down box.

The UI should concatenate the four AUTO_ADD_IN_CONFIG.HOST_VALUE values from the four "Action Required" selections chosen by the user and place this concatenated value in the AUTOACTIONSRULE.SHOST_VALUE column.

7.4.2 Steps to add the Auto Add-In

The customer will be able to provide the entries for the dropdowns for the auto action. They can define the values for the Action Type dropdown, as well as up to four entries for the Action Required dropdown. If they have not configured all four fields, then only those configured will show on the tab. Because auto add-ins are intended to communicate with other systems, they will be presumed to be External Message Manager enabled. When an auto add-in rule fires, it will update MasterDisp and Act, but will also update the EXTERNAL_MESSAGE_DATA table. The contents of the Auto Add-in Action Type combo box will be placed in the EXTERNAL_MESSAGE_DATA.sReasonCode column. The contents of the Action Required dropdowns will be concatenated into a comma separated list and put in the EXTERNAL_MESSAGE_DATA.SHOST_DATA column. The action description will be placed in EXTERNAL_MESSAGE_DATA.sNotes. The other columns in the EXTERNAL_MESSAGE_DATA table, shown below will be filled in with the transaction data that fired the rule where data is available in the transaction (columns known not to be needed excluded):

```

MESSAGE_ID
ACTION_LEVEL_ID
LPORTFOLIOID
ACTION_TYPE_ID
ACTION_LEVEL_KEY
LSTATE

```

Reflects the state after the rule has fired

LSTATUS	and any auto actions applied. Reflects the state after the rule has fired and any auto actions applied.
DD_DATE	
ND_KEY_HASH	
ND_RECORD_SOURCE_ID	
SD_TIEBREAKER	
DTREQUESTED	Insert Date/Time
DTMOSTRECENTTXN	
DTENDDATE	
LREVID	
LERRID	
SEMSG	
SNOTE	Auto Add-in action description
SHOST_DATA	Action Request 1, Action Request 2, Action Request 3, Action Request 4 (up to 4)
SREASON_CODE	(Auto Addi-in) Action Type
SD_SEQ_NUM	
SD_MBR_NUM	
SM_FIID	
SM_BIN	

7.4.2.1 Add L10N Properties Text for the ActionType and ACTIONSTYPE_ACTION_LEVEL Tables

Follow the same methods as defined in 7.2.1 to add the text for the action that will be shown in the Actions tab on the Action Level review screen and the action reports.

7.4.2.2 Add a New Action Type

Add a new action type for the action in the ActionType table using the instructions from 7.2.3 with the exception that a permission ID is not required if the action is not used on the review screen. Auto actions are invoked by the system during rule processing and do not require a permission to run. Also, the auto add-in action requires two columns to be set: ACTIONSTYPE.AUTO_ACTION and ACTIONSTYPE_ACTION_LEVEL.AUTO_UPDATES_ALLOWED.

The ACTIONSTYPE.AUTO_ACTION column should be set to 2 to indicate that the action is available for auto Add-In.

ACTIONSTYPE.AUTO_ACTION column has the following values:

0 = no	- Not enabled for auto-actions
1 = yes	- Enabled for auto-actions
2 = Auto Addin	- Enabled for auto add-in actions

The ACTIONSTYPE_ACTION_LEVEL.AUTO_UPDATES_ALLOWED column should be set to 0 if the action can only update once (if the rule fires again, it will be ignored and there will be no updates to Act, MasterDisp or EXTERNAL_MESSAGE_DATA. If set to 1, entries will be put in those tables each time the rule fires.

The ACTIONSTYPE_ACTION_LEVEL.ShowState value indicates for auto actions or add-ins whether the MasterDisp table should be updated. A value of 1 indicates that it should be updated and a value of 0

indicates that it should not. The auto action is always considered to be invoking the on state of the action in standard rule processing.

AUTO_ADD_IN_CONFIG.CONFIG_TYPE_ID has the following values:

- 0 = Action Type
- 1 = Required Value 1
- 2 = Required Value 2
- 3 = Required Value 3
- 4 = Required Value 4

The ACTIONSTYPE.AUTO_ACTION_PROC column should contain the name of the stored procedure to be used for the Auto Add-in.

When an auto-action record is created, a record is inserted into the RuleQue table for the rule and a record is inserted into the AUTOACTIONSRULE table for each of the up to three potential auto-actions that can be defined for a rule. The ACTION_ORDER column of the AUTOACTIONSRULE table indicates the whether the auto-action is the first, second or third auto-action defined for the rule.

For following table indicates which columns in the AUTOACTIONSRULE table are required for each type of action in addition to the LID, LRULEID, LTEAMID and ACTION_ORDER values.

Block	x	x	x	x
Watch	x	x		x
Ice	x	x		x
Call	x			x
Memo	x			x
Case Create	x			x
Add-In Action	x			x

If an auto-action rule fires for a transaction, the rule ID is stored in a local variable in the rules procedure. After all of the deployed rules have been evaluated, the rules procedure inserts a record into the AUTO_ACTION_PENDING table for each of the auto-action rules that fired. The PP_AUTO-ACTION procedure is called at the end of the rules procedure if at least one auto-action rule fired.

7.4.2.3 Add Actions Text for the Action

Define the text for the auto action based on the instructions in the previous sections.

7.4.2.4 Add Entries to the AUTO_ADD_IN_CONFIG table

Add entries to AUTO_ADD_IN_CONFIG to define the Action Type and Required Fields for the auto Add-in. The columns of the table are:

AUTO_ADD_IN_CONFIG_ID	INTEGER	NOT NULL
ACTION_TYPE_ID	INTEGER	NOT NULL
CONFIG_TYPE_ID	INTEGER	NOT NULL
L10N_PROPERTY_ID	INTEGER	NOT NULL
DESCRIPTION	NVARCHAR(50)	NULL,
HOST_VALUE	NVARCHAR(256)	NULL

7.4.2.5 Rule Template Modifications

Modifications may be needed to the TEMPLATE_AUTOACTION rule template when a new action is added. If the templates are changed, the rules must be redeployed.

Auto Action processing will be performed in pp_Auto_Actions procedure as similar to Standard actions. AutoAction template will capture the RuleId and ActionId and use these values to create necessary entries into Auto_Action_Pending table. When an Auto Add-In action is matched for a Rule, pp_Auto_Action procedure will call the PROC defined for the Auto Add-In actions. No other updates are included in pp_Auto_Actions table (Act or MasterDisp or External_Message_Data). All the required processing should be part of the Custom Proc defined by the customer for the corresponding Action. Following are the standard parameters used to call the customer defined proc –

@DD_PARTITION_DATE, @ND_KEY_HASH, @RECORD_SOURCE_ID, @SD_TIEBREAKER,
@ACTION_LEVEL_ID, @ACTION_LEVEL_KEY, @PORTFOLIO_ID, @DD_DATE,
@ACTION_TYPE_ID, @EMM_ENABLED, @SHOWSTATE, @AUTO_UPDATES_ALLOWED,
@RULE_ID, @SM_BIN, @SM_FIID, @ALERT_DATE, @LTEAMID, @LREVID, @SHOST_VALUE,
@LOCALE_ID, @ACTION_DESCRIPTION, @ACTION_DATETIME, @ACTION_ORDER,
@AUTO_ADDIN_TYPE

If any additional changes are required, it may be advisable to update the pp_Auto_Actions procedure for a specific action so that Standard actions and processing will not be impacted. Please review the updates in this procedure and add custom code for additional updates, if any required.

8 Developing Add-Ins for PRM

8.1 Introduction

The PRM analysis review system is a JAVA-based client-server application built around a relational database using a combination of JAVA SWING and Servlet technology. The PRM User Interface provides the analysis and review capabilities of the PRM system. As a developer, you can expand the functionality of the analysis review system with PRM Add-In components. Installation and configuration of these components is administered on the Web Server and in structures contained in the PRM database.

In order to develop an Add-In you must have the following resources available:

- A development environment with Java SDK, installed and access to a web server (e.g. Tomcat)
- Access to a development copy of PRM containing both UI components (ui-client) and server-side components (ui-server).
- A copy of the JAR file containing the main portions of the PRM UI. This file is called ui-client.jar. Note: this file needs to be removed for the final build.
- Permission to update database structures in the test PRM database you will be using to develop your add-in.
- Java Web Start available at: <http://java.sun.com/products/javawebstart/download-windows.html>

See the PRM *Hardware and Software Requirements Guide* for supported versions.

Additional items that will be helpful in learning how to build a PRM Add-in include:

- Documentation on JNLP technology and Java Web Start:
<https://docs.oracle.com/javase/tutorial/deployment/webstart/>
- Familiarity with JAR files and how to create and deploy them:
<https://docs.oracle.com/javase/tutorial/deployment/jar/basicsindex.html>

8.2 The PRM Environment

PRM functionality is provided by a combination of user interface components written in JAVA using SWING technology, Servlets contained in a Servlet container that access and manipulate data, and the PRM database that houses the data and stores the end-user interaction with the data.

Your responsibilities as an add-in developer will be to write JAVA code using the PRM framework to extend the end-user experience. You will accomplish this by writing user interface code using JAVA SWING, your own servlets based upon base PRM classes, and accessing either data within the PRM database, or external data outside of PRM that your end users require.

Implementing your add-in will involve the following:

Developing your backend Servlets that access and package the data that your users require. Or, if the design of your add-in requires that you capture end-user data, your Servlet will be responsible for storing it in data structures you create in PRM, or depositing that data in external systems.

Developing the UI code using JAVA SWING to present your data, or capture your end-user data as required.

Defining your message format using XML data structures to ship your data to and from your UI and Servlet backend.

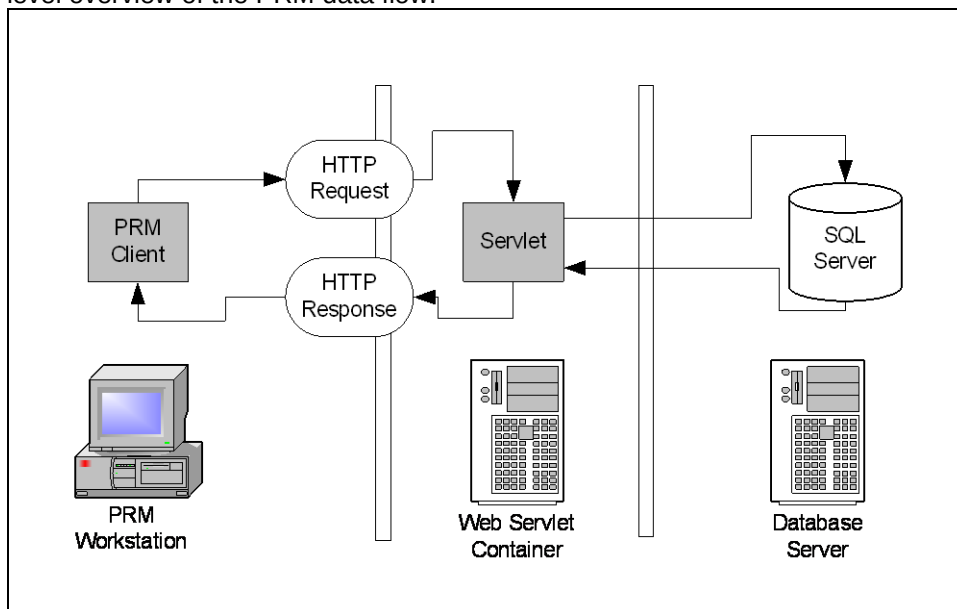
Packaging your UI code into JAR files for distribution to your client workstations.

Modifying the PRM JNLP file(s) to inform Web Start to bring your code (JAR file) down to the workstation automatically.

Modifying configuration structures in the PRM database to make the PRM UI aware of your add-in.

8.3 PRM User Interface Architecture Overview

The PRM architecture is based upon an n-tier design. The application is designed to separate presentation, business logic, and data into discreet layers. Each layer communicates through a defined interface with the layer below isolating changes from one layer to another. The following picture is a high-level overview of the PRM data flow.



The PRM client communicates with the web server by composing HTTP requests in the same way that a browser would. The HTTP request contains normal headers, but the body of the request is an XML data structure that provides information regarding the type of request that the client requires.

The web server resolves the request to the correct servlet which then extracts the XML payload and performs the necessary logic to satisfy the request. In most cases, the request is to get some data that is necessary to render the UI. The servlet can use a database connection to interrogate the database server for the information that it needs to satisfy the request, and perform any other required business logic that might be necessary to transform the data.

Finally, the servlet responds with an HTTP response to the PRM UI with its payload another XML data structure that the client can use in the presentation of the data to the end user.

Built into the PRM framework is the ability to encrypt the PRM message as well as to compress the message. By using the base PRM classes, your add-in will get this functionality with no additional coding requirements.

8.4 The PRM User Interface Classes

There are five interfaces and base classes that you will use when constructing your add-in. The following table lists the currently supported interfaces and base classes:

PrmAddin	Interface	Client side: Add In interface with PRMClient
PrmDialog	Class	Client side: PrmDialog extends JDialog implements PrmAddin
PrmServlet	Class	Server side: Base PRM Servlet class. All PRM servlets extend the PrmServlet class. Contains utility functions common to all servlets.
PrmParams	Class	PrmParams implements Map This class allows to pass objects as parameter to the various methods.
ObjectLibrary	Class	ObjectLibrary extends java.lang.Object Stores references to objects that are used throughout the program. The objectlibrary contains a list of name/value pairs that are available to your add-in without having to write servlet code.

8.5 The PrmParams Class

PrmParams implements Map.

This class allows the program to pass objects as parameter to the various methods. It also creates an instance of the class objectlibrary, which stores references to objects that are used throughout the program.

By using PrmParams you get access to all objects set in the objectlibrary as well as any other objects that have been set by user.

The following chart lists the procedures, methods, and properties of the PrmParams class.

getAsBoolean	method	java.lang.Object key	gets object as Boolean
getAsBoolean	method	java.lang.Object key, boolean defaultValue	gets object as Boolean
getAsInt	method	java.lang.Object key	gets object as Integer
getAsInt	method	java.lang.Object key, boolean defaultValue	gets object as Integer

getAsString	method	java.lang.Object key	gets object as String
getAsString	method	java.lang.Object key, boolean defaultValue	gets object as String
getObjLib()	method		gets objectlibrary
getObjLib()	method	ObjectLibrary library	Set object in the objectlibrary
toString()	method	java.lang.Object key	translates object into string.

AccountInfoTabLayout	getAccountInfoTabLayout()
java.util.Vector	getAccountToolBarButtons()
java.util.Hashtable	getActionDecoratorIcons()
java.util.Hashtable	getActionIcons() Returns the myActionIcons_.
AutoText	getAutoText() Returns the myAutoText_.
java.lang.String	getBaseHelpUrl()
ConfigReader	getConfigReader()
ConfigTable	getConfigTable()
java.awt.Cursor	getDefaultCursor()
java.awt.Cursor	getHourGlassCursor()
javax.swing.ImageIcon	getImage(java.lang.String imageName, PrmLogger logger)
MainScreen	getMainScreen()

MessageHandler	getMessageHandler() Returns the Exception handler myPrmException_.
PrmResourceBundle	getMessages()
PrmTableLayouts	getPrmTableLayouts()
RecentAccounts	getRecentAccounts()
User	getUser()
UserConnection	getUserConnection()
XMLProcessor	getXMLProcessor()
void	setUserConnection(UserConnection userConnection)
void	setXMLProcessor(XMLProcessor xmlProcessor)
void	setAccountInfoTabLayout(AccountInfoTabLayout newLayout)
void	setAccountToolBarButtons(java.util.Vector myAccountToolBarButtons)
void	setActionDecoratorIcons(java.util.Hashtable hashtable)
void	setActionIcons(java.util.Hashtable myActionIcons) Sets the myActionIcons_.
void	setAutoText(AutoText myAutoText) Sets the myAutoText_.
void	setBaseHelpUrl(java.lang.String string)
void	setConfigReader(ConfigReader configReader)
void	setConfigTable(ConfigTable config)
void	setMainScreen(MainScreen mainScreen)
void	setMessageHandler(MessageHandler messageHandler) Sets the Exception handler myPrmException_.

void	setMessages(PrmResourceBundle resource)
void	setPrmTableLayouts(PrmTableLayouts newObj)
void	setRecentAccounts(RecentAccounts newObj)
void	setUser(User user)

8.6 The PrmAddin Interface

The following chart lists the procedures, methods, and properties of the PrmAddin interface.

load	method	PrmParams	Load is called when object is added to menu. Handle to menu and menu item are in params
execute	method	PrmParams	Execute is called when add-in is executed, either button click or menu select. If button on review screen, current accountkey and user object in the objectslibrary are passed in. If from menu click, menuid and name are passed in.
getParam	method	PrmParams	Returns a param as string. Current addin must support Version - version # of addin IsPersistent - should object be maintained between calls.

8.7 The PrmDialog Class

Use this class to create a dialog window. (see *The Java Tutorial* section [How to Make Dialogs](#))

PrmDialog extends JDialog which implements PrmAddin. Then you create a new class that extends PrmDialog.

The following fields are set in the PrmDialog class.

protected java.awt.Cursor	defaultCursor_
protected java.util.Locale	locale
protected MainScreen	mainScreen
protected PrmResourceBundle	messages_
protected MessageHandler	msgHandler
protected ObjectLibrary	objLib_
protected PrmLogger	prmLog
protected UserConnection	userCon
protected java.awt.Cursor	waitCursor_
protected XMLProcessor	xmlProc

load	method	PrmParams	Load is called when object is added to menu. Handle to menu and menu item are in params.
execute	method	PrmParams	Execute is called when add-in is executed, either button click or menu select. If button on review screen, current accountkey and user object are passed in. If from menu click, menuid and name are passed in.
getParam	method	PrmParams	Returns a param as string.
Init	method	PrmParams params	

8.8 The PrmServer Class

Base PRM Servlet class.

PrmServer extends HttpServlet. Then you create a new class that extends PrmServer.

All PRM servlets extend the PrmServlet class.

Contains utility functions common to all servlets.

Example of class using PrmServer as parent class is

PrmSampleSql **extends** PrmServlet

Example of how to query the database.

The following chart lists the procedures, methods, and properties of the PrmServer interface.

init	method	ServletConfig	initial method setting logger.
setContentType	method	HttpServletRequest HttpServletResponse	Sets the response ContentType to text/xml. The body of HTTP response is an XML document.
doPost	method	HttpServletRequest HttpServletResponse	
doGet	method	HttpServletResponse	
getXMLWriter	method	HttpServletResponse,	Returns an XML writer based on the response's output stream
WriteResponse	method	HttpServletResponse, PrmResponse	Writes the current response document to the Servlet output stream
getUser	method	String SessionId	Returns a user out of the user's table stored in the web app's context
getSystemFields	method		Returns list of the system_fields, a copy of the sytemfields table. Used to map PRM fields to specific database fields by client.
getSystemField	method	String sName	Returns a systemfield name=value mapping.
getConfigItem	method	String name	Example: AccountKey = SM Key Returns a Config_Item, that wraps a row from the config and system config tables.

getFieldConfigItem	method	String name	Returns a FieldConfig_Item, that wraps a row from the fieldconfig table.
getConfigVariableString	method	String name	Returns the string value of a config item
getConfigVariableLong	method	String name	Returns the long value of a config item.
getSqlNow	method		Returns the database specific sql command for the current datetime.
log_Headers	method		Log's the HTTP request headers to the current log.
log_pushContext	method	String sName	This pushes a string value onto the current log's context stack. This associates a context (current user for example) with log messages.
log_removeContext	method		Pops the current context off the stack.
log_Debug	method	Object arg	Log a debug level message.
log_Debug	method	Object arg, Throwable t	Log a debug level message.
log_Info	method	Object arg	Log info message.
log_Info	method	Object arg, Throwable t	Log info message.
log_Warn	method	Object arg	Log warning message.
log_Warn	method	Object arg, Throwable t	Log warning message.
log_Error	method	Object arg	Log error message.
log_Error	method	Object arg, Throwable t	Log error message.
log_Fatal	method	Object arg	Log fatal message.
log_Fatal	method	Object arg, Throwable t	Log fatal message.

getSessionId	method	HttpServletRequest request	Returns the sessionId for the current request. It is added to the header to allow the filtering servlet to stop requests that don't have a valid sessionId.
UserHasAccount	method	User user, String account	Checks that a user has an account open, if not, throws AccountNotOpenException.
UserOnTeam	method	User user, int TeamId	Checks that a user has access to a team.
getServletName	method		Gets Servlet Name.

8.9 PRMAddInSample: PRMDialogNoDB.java

The first sample provided is a toolbar button addin that when clicked displays the current account number and the name of the user logged onto PRM. This addin makes use of the PRM objectLibrary which contains a list of name value pairs that are available to your add-in without having to write servlet code.

The following is a generalized walkthrough of the logic within this sample:

- The PRMDialogNoDB class consist of 3 methods (execute, load, getParam). Only execute is being used in the current version.
- Prm Objectlibrary.
- PRMDialogNoDB implements PrmAddin.
- The execute method takes PrmParams as parameter, which enables the method to access the Prm Objectlibrary.
 - get Account number → String accountkey = params.getAsString("AccountKey");
 - get User → User user = (User) params.get("User");
- Add Account number and User to a string → Message.
- Create new window using Jdialog, which include JoptionPane with the Message string on it.

```

/*
 * Sample review screen add-in.
 * Add-in will be added to the account review toolbar.
 * The prmInit function will be called on button click.
 *
 * Copyright ACIWORLDWIDE 2003
 */
package com.aciworldwide.prm.PRMAddInSample;

import com.aciworldwide.prm.general.PrmParams;
import com.aciworldwide.prm.general.User;
import com.aciworldwide.prm.interfaces.PrmAddin;
import com.aciworldwide.prm.util.PrmUtilities;

import javax.swing.JDesktopPane;
import javax.swing.JDialog;
import javax.swing.JOptionPane;
import javax.swing.JRootPane;

public class PRMDialogNoDB
    implements PrmAddin

```

```

{
    public String getParam(PrmParams params)
        throws Exception
    {
        return null;
    }

    public void execute(PrmParams params)
        throws Exception
    {
        // Get current account key
        String accountkey = params.getAsString("AccountKey");

        //Get current user
        User user = (User) params.get("User");

        //Create message
        String message = "Current account is " + accountkey + ". Current user is " +
            user.getUsername();

        // Create Popup
        JDesktopPane desktop = PrmUtilities.getMainScreen().getDesktop();

        // Display message
        JOptionPane pane = new JOptionPane(message);

        // Create New JDialog
        JDialog dialog = pane.createDialog(desktop, this.getClass().getName());

        //Set dialog to current look and feel
        dialog.getRootPane().setWindowDecorationStyle(JRootPane.INFORMATION_DIALOG);

        // set screen title
        dialog.setTitle("PRM Dialog No DB Sample");

        // show screen
        dialog.show();
    }

    public void load(PrmParams params)
        throws Exception
    {
    }
}

```

8.10 PRMAddInSample: PRMDialogDBUI.java &PRMDialogDBqry.java

This sample shows you how to access data using UI and Servlet components. This sample also makes use of the xmlProc to ship data between the UI component and the Servlet.

PRMDialogDBUI.java

The following is a generalized walkthrough of the logic within UI component:

- The PRMDialogDBUI class is basically a copy of PRMDialogNoDB class as it also consist of 3 methods (execute, load, getParam). Only execute is being used in the current version.
- Only the bolded code is being used, where it creates an instance of the PRMDialogDBSQL class:
PRMDialogDBqry prmDialog = new PRMDialogDBqry(dialog, "PRMDialogDBUI");

```

package com.aciworldwide.prm.PRMAAddInSample;

import com.aciworldwide.prm.general.PrmParams;

```

```

import com.aciworldwide.prm.general.User;
import com.aciworldwide.prm.interfaces.PrmAddin;
import com.aciworldwide.prm.forms.configure.edit.*;
import com.aciworldwide.prm.util.PrmUtilities;

import javax.swing.JDesktopPane;
import javax.swing.JDialog;
import javax.swing.JOptionPane;

public class PRMDialogDBUI
    implements PrmAddin
{
    public String getParam(PrmParams params)
        throws Exception
    {
        return null;
    }

    /* Called when add-in is executed (button is clicked)
     * PrmParams extends a HashMap. It contains a reference to the Prm
     * ObjectLibrary, and a collection of name/value pairs.
     *
     * @see
     * com.aciworldwide.prm.interfaces.PrmPlugin#prmInit(com.aciworldwide.prm.general.PrmParams)
     */
    public void execute(PrmParams params)
        throws Exception
    {
        // Get current account key
        String accountkey = params.getAsString("AccountKey");

        // Get current user
        User user = (User) params.get("User");

        // Create message -- not used
        String message = "Current account is " + accountkey + "\nCurrent user is " +
user.getUsername();

        // Create Popup
        JDesktopPane desktop = PrmUtilities.getMainScreen().getDesktop();

        // Display message
        JOptionPane pane = new JOptionPane(message);

        // Create New JDialog
        JDialog dialog = pane.createDialog(desktop, this.getClass().getName());

        // Create New prmDialog
        PRMDialogDBqry prmDialog = new PRMDialogDBqry(dialog, "PRMDialogDBUI");

        // Set screen location
        prmDialog.setLocation(200, 200);
        params.put("width", new Integer(600));
        params.put("height", new Integer(400));

        // Execute prmDialog
        prmDialog.execute(params);
    }

    public void load(PrmParams params)
        throws Exception
    {
    }
}

```

PRMDialogDBqry	Constructor	JFrame owner, String dialogTitle	Constructor method that set up basic settings.
PRMDialogDBqry	Constructor	Dialog owner, String title	Constructor method that set up basic settings.
execute	Method	PrmParams params	Init the base class, set title, draws the screen and initiate the PrmServlet class used.
callServlet	Method	String accountKey	Called by execute, sets the parameters used and calls instance of XMLProcessor which is a utility class that handles the communication with the servlets. The servlet used here is PRMAddInSampleServlet. It handles encryption/compression of the message. It takes a PrmParams object which is a hashmap of name/value pairs
drawScreen	Method		Create the screen and text area. Called in execute.
processResponse	Method	Document respDoc	Iterates through the result set that was passed in.
updateConfig	Method	String configVar, String sCfgVal, int sCfgVal	Demonstrates the use of the ObjectLibrary. How to read & change values .
buildButtonPanel	Method		Creates the Button panel.
buildContentPanel	Method		Creates the main content panel.
buildTopPanel	Method		Creates the top panel.

```

package com.aciworldwide.prm.PRMAAddInSample;

import com.aciworldwide.prm.forms.PrmDialog;
import com.aciworldwide.prm.forms.configure.edit.ConfigEditorForm;
import
com.aciworldwide.prm.forms.configure.edit.ConfigEditorForm.MyEventListener;
import
com.aciworldwide.prm.forms.configure.edit.ConfigEditorForm.TableMouseListener;
import
com.aciworldwide.prm.forms.configure.edit.ConfigEditorForm.TypesComboBoxListener;
import com.aciworldwide.prm.forms.configure.edit.EditTextDialog;
import com.aciworldwide.prm.forms.prmtable.PrmTable;
import com.aciworldwide.prm.forms.prmtable.PrmTableModel;
import com.aciworldwide.prm.forms.prmtable.PrmTableUtilities;
import com.aciworldwide.prm.general.*;
import com.aciworldwide.prm.util.PrmUtilities;
import org.dom4j.Document;
import org.dom4j.Node;
import java.awt.BorderLayout;
import java.awt.Container;

```

```

import java.awt.Dialog;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.HeadlessException;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;

import java.util.Enumeration;
import java.util.HashMap;
import java.util.Hashtable;
import java.util.Iterator;
import java.util.List;
import java.util.Vector;

import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;

public class PRMDialogDBqry
    extends PrmDialog
{
    private static final String screenTitle = "PRMDialogDBqry";
    private static int        screenWidth = 600;
    private static int        screenHeight = 400;
    public static final int AddIn_ADD = 0;
    public static final int AddIn_EDIT = 1;
    public static final int AddIn_DELETE = 2;
    public static final int AddIn_SELECT = 3;
    public static final int AddIn_SP = 4;
    public JComboBox          typeComboBox = null;
    protected Dimension       buttonSize = null;
    protected Hashtable       typesById = null;
    protected JButton         addButton = null;
    protected JButton clearButton = null;
    protected JButton deleteButton = null;
    protected JButton editButton = null;
    protected JButton exitButton = null;
    protected JButton selButton = null;
    protected JButton spButton = null;
    protected JFrame frame;
    protected JLabel          lblAccKey = null;
    protected JLabel          lblBox1 = null;
    protected JLabel          lblBox2 = null;
    protected JLabel          lblBox3 = null;
    protected JLabel          lblBox4 = null;
    protected JLabel          lblBox5 = null;
    protected JLabel          lblDB = null;
    protected JTextField       txtBox1 = null;
    protected JTextField       txtBox2 = null;
    protected JTextField       txtBox4 = null;
    protected JTextField       txtBox5 = null;
    protected PrmLogger        myPrmLogger ;
    protected PrmTable         table = null;
    protected PrmTableModel    model = null;
    protected String          ak = null;
    protected String          configEditorTitle = null;
    protected String          errorTitle = null;
    protected String          populateTableResponseNode = null;
    protected String          populateTableServlet = null;

```

```

protected String      servletResponseNode = null;
protected String      textEditorTitle = null;
protected String      updateServlet = null;
protected Vector      categoryTypes = null;
protected boolean     buildTopPanel = false;
protected int         numberOfColumns;
private JPanel        panel;
private JPanel        panel2;
private JScrollPane   responseScroll;
private JTextArea     responseText;
public ObjectLibrary  myObjLib;
public PrmParams      myParams;
private static boolean modal=true;

public PRMDialogDBqry(JFrame owner, String dialogTitle)
{
    super(owner, dialogTitle, modal);

    setResizable(true);
    setModal(true);
    setDefaultCloseOperation(DISPOSE_ON_CLOSE);
}

public PRMDialogDBqry(Dialog owner, String title)
    throws HeadlessException
{
    super(owner, title, modal);

    setResizable(true);
    setModal(true);
    setDefaultCloseOperation(DISPOSE_ON_CLOSE);
}

public Integer getIdByValue(String value)
{
    Integer id = null;

    boolean found = false;

    for (Enumeration e = typesById.keys(); e.hasMoreElements() && !found;)
    {
        Integer thisId = (Integer) e.nextElement();
        String val = typesById.get(thisId).toString().trim();

        if (value.equals(val))
        {
            id = thisId;
            found = true;
        }
    }

    return id;
}

public void execute(PrmParams params) throws Exception
{
    //Init the base class first
    super.execute(params);
    myObjLib = this.objLib_;
    myParams = params;

    // Get account number
    ak = params.getAsString("AccountKey");

    // Set screen title
    setTitle((screenTitle + " -- Account: " + ak)); // + " (Detail table)");

    // Set screen size
    screenWidth = params.getAsInt("width");
    screenHeight = params.getAsInt("height");
}

```

```

        //setSize(scrWidth, scrHeight);
        setSize(screenWidth, screenHeight);

        myPrmLogger_ = new PrmLogger(ConfigEditorForm.class.getName());

        setResizable(true);

        buttonSize = new Dimension(80, 25);

        PrmUtilities.setPositionCenterScreen(this, 700, 450);

        // Draw screen
        drawScreen();
    }

    private JPanel buildButtonPanel()
    {
        JPanel buttonPanel = new JPanel(new BorderLayout());
        buttonPanel.setSize(new Dimension(100, 400));

        JPanel topButtonPanel = new JPanel();
        JPanel bottomButtonPanel = new JPanel();

        String buttonString = null;

        // Calculate button size
        String[] sButtons;
        sButtons = new String[6];

        PrmResourceBundle messages_ = PrmUtilities.getMessages();
        sButtons[0] = messages_.getString("AddIn_Insert_Button");
        sButtons[1] = messages_.getString("AddIn_Update_Button");
        sButtons[2] = messages_.getString("AddIn_Delete_Button");
        sButtons[3] = messages_.getString("AddIn_Exit_Button");
        sButtons[4] = messages_.getString("AddIn_Select_Button");
        sButtons[5] = messages_.getString("AddIn_Clear_Button");

        int iLen = -1;

        for (int i = 0; i < sButtons.length; i++)
        {
            if (sButtons[i].length() > iLen)
            {
                iLen = sButtons[i].length();
            }
        }

        if (iLen > 5)
        {
            buttonSize = new Dimension((80 + iLen + 5), 25);
        }

        //Create buttons
        // ADD
        addButton = new JButton();
        addButton.setName("addButton");
        addButton.setPreferredSize(buttonSize);
        buttonString = sButtons[0];
        addButton.setText(buttonString);
        addButton.setMnemonic(KeyEvent.VK_A);

        int index = buttonString.toLowerCase().indexOf("a");

        if (index != -1)
        {
            addButton.setDisplayedMnemonicIndex(index);
        }

        //      add listener
        addButton.addActionListener(new MyEventListener());
    }

```

```

// EDIT
editButton = new javax.swing.JButton();
editButton.setName("editButton");
editButton.setPreferredSize(buttonSize);
buttonString = sButtons[1];
editButton.setText(buttonString);
editButton.setMnemonic(KeyEvent.VK_E);
index = buttonString.toLowerCase().indexOf("e");

if (index != -1)
{
    editButton.setDisplayedMnemonicIndex(index);
}

//      add listener
editButton.addActionListener(new MyEventListener());

// DELETE
deleteButton = new javax.swing.JButton();
deleteButton.setName("deleteButton");
deleteButton.setPreferredSize(buttonSize);
deleteButton.setMnemonic(KeyEvent.VK_D);
buttonString = sButtons[2];
deleteButton.setText(buttonString);

index = buttonString.toLowerCase().indexOf("d");

if (index != -1)
{
    deleteButton.setDisplayedMnemonicIndex(index);
}

//      add listener
deleteButton.addActionListener(new MyEventListener());

// SELECT
selButton = new JButton();
selButton.setName("selButton");
selButton.setPreferredSize(buttonSize);
buttonString = sButtons[4];
selButton.setText(buttonString);
selButton.setMnemonic(KeyEvent.VK_S);

index = buttonString.toLowerCase().indexOf("s");

if (index != -1)
{
    selButton.setDisplayedMnemonicIndex(index);
}

//      add listener
selButton.addActionListener(new MyEventListener());

// clear
clearButton = new javax.swing.JButton();
clearButton.setName("clearButton");
clearButton.setPreferredSize(buttonSize);
clearButton.setMnemonic(KeyEvent.VK_C);
buttonString = sButtons[5];
clearButton.setText(buttonString);

index = buttonString.toLowerCase().indexOf("c");

if (index != -1)
{
    clearButton.setDisplayedMnemonicIndex(index);
}

// add listener
clearButton.addActionListener(new MyEventListener());

```



```

        exitButton = new javax.swing.JButton();
        exitButton.setName("exitButton");
        exitButton.setPreferredSize(buttonSize);
        exitButton.setMnemonic(KeyEvent.VK_X);
        buttonString = sButtons[3];
        exitButton.setText(buttonString);
        index = buttonString.toLowerCase().indexOf("x");

        if (index != -1)
        {
            exitButton.setDisplayedMnemonicIndex(index);
        }

        //make the ok button the default button
        getRootPane().setDefaultButton(addButton);

        //add listener
        exitButton.addActionListener(new MyEventListener());

        //    PrmParams params = new PrmParams(myObjectLibrary_);
        //    helpButton.addActionListener(new HelpLauncher(params));
        //Create button panel
        topButtonPanel.setLayout(new GridBagLayout());
        bottomButtonPanel.setLayout(new GridBagLayout());

        GridBagConstraints c = new GridBagConstraints();

        c.anchor = GridBagConstraints.NORTH;
        c.fill = GridBagConstraints.NONE;
        c.insets.right = 10;
        c.insets.left = 10;

        c.insets.bottom = 5;
        c.insets.top = 20;
        c.gridx = 1;
        c.gridy = 1;
        c.gridwidth = 1;
        c.gridheight = 1;

        topButtonPanel.add(addButton, c);

        c.insets.top = 0;
        c.gridy = 2;
        topButtonPanel.add(editButton, c);

        c.gridy = 3;
        topButtonPanel.add(deleteButton, c);

        c.gridy = 4;
        topButtonPanel.add(selButton, c);

        c.gridy = 5;
        topButtonPanel.add(clearButton, c);

        c.gridy = 2;
        bottomButtonPanel.add(exitButton, c);

        buttonPanel.add(topButtonPanel, BorderLayout.NORTH);
        buttonPanel.add(bottomButtonPanel, BorderLayout.SOUTH);

        return buttonPanel;
    }

    /**
     * Build ContentPanel
     */
    private JPanel buildContentPanel()
    {
        panel = new JPanel();

```

```

        panel.setName("JFrameContentPane"); //$NON-NLS-1$
        panel.setLayout(new BorderLayout());

        panel.setName("MyPanel");
        panel.setLayout(new BorderLayout());

        responseText = new JTextArea();
        responseText.setLineWrap(true);
        responseText.setBackground(new java.awt.Color(252, 252, 203));
// (java.awt.Color.yellow);
        responseText.setForeground(java.awt.Color.blue);
        responseScroll = new JScrollPane(responseText);

responseScroll.setVerticalScrollBarPolicy(JScrollPane.VERTICAL_SCROLLBAR_ALWAYS);
        responseScroll.setPreferredSize(new Dimension(screenHeight,
screenWidth));

        getContentPane().add(responseScroll, BorderLayout.CENTER);

        return panel;
    }

    /*****
private JPanel buildTopPanel()
{
    // TOP PANEL
    JPanel topPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
    topPanel.setBounds(1, 1, 750, 125);
    topPanel.setPreferredSize(new Dimension(600, 125));

    // LEFT PANEL within TOP PANEL
    JPanel lPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
    lPanel.setBounds(0, 0, 250, 125);
    lPanel.setPreferredSize(new Dimension(250, 125));

    // MIDDLE PANEL within TOP PANEL
    JPanel mPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
    mPanel.setBounds(0, 0, 250, 125);
    mPanel.setPreferredSize(new Dimension(250, 125));

    // RIGHT PANEL within TOP PANEL
    JPanel rPanel = new JPanel(new FlowLayout(FlowLayout.LEFT));
    rPanel.setBounds(0, 0, 250, 125);
    rPanel.setPreferredSize(new Dimension(250, 125));

    // AccountKey label
    lblAccKey = new JLabel();
    lblAccKey.setPreferredSize(new Dimension(200, 20));
    lblAccKey.setForeground(java.awt.Color.BLUE);
    lblAccKey.setText("AccountKey: " + ak);

    // Table label
    lblDB = new JLabel();
    lblDB.setPreferredSize(new Dimension(200, 20));
    lblDB.setText("");

    // TEXTBOX 1
    lblBox1 = new JLabel();
    lblBox1.setPreferredSize(new Dimension(75, 20));
    lblBox1.setText("dtActDateKey: ");
    txtBox1 = new JTextField();
    txtBox1.setPreferredSize(new Dimension(150, 20));
    txtBox1.setEditable(true);
    txtBox1.addActionListener(new TypesComboBoxListener());
    txtBox1.updateUI();
    txtBox1.setText("yyyy-MM-dd HH:mm:ss.SSS");
    txtBox1.setToolTipText("You must enter a valid dtActDateKey for UPDATES &
DELETES");

    // TEXTBOX 2
    lblBox2 = new JLabel();

```

```

lblBox2.setPreferredSize(new Dimension(75, 20));
lblBox2.setText("ActionID: ");
txtBox2 = new JTextField();
txtBox2.setPreferredSize(new Dimension(150, 20));
txtBox2.setLocation(0, 30);
txtBox2.setText("0");

// TEXTBOX 4
lblBox4 = new JLabel();
lblBox4.setPreferredSize(new Dimension(75, 20));
lblBox4.setText("Notes1: ");
txtBox4 = new JTextField();
txtBox4.setPreferredSize(new Dimension(150, 20));
txtBox4.setLocation(0, 30);
txtBox4.setText("enter notes");

// TEXTBOX 5
lblBox5 = new JLabel();
lblBox5.setPreferredSize(new Dimension(75, 20));
lblBox5.setText("Notes2: ");
txtBox5 = new JTextField();
txtBox5.setPreferredSize(new Dimension(150, 20));
txtBox5.setLocation(0, 30);
txtBox5.setText("enter notes");

GridBagConstraints c = new GridBagConstraints();

c.anchor = GridBagConstraints.NORTH;
c.fill = GridBagConstraints.NONE;
c.insets.right = 10;
c.insets.left = 10;

c.insets.bottom = 0; //5
c.insets.top = 0; //20
c.gridx = 1;
c.gridy = 1;
c.gridwidth = 1;
c.gridheight = 1;

// ROW 1
c.gridx = 1;
c.insets.right = 0;
c.insets.left = 0;
lPanel.add(lblBox1, c);
mPanel.add(lblBox4, c);
c.gridx = 2;
lPanel.add(txtBox1, c);
mPanel.add(txtBox4, c);

// ROW 2
c.insets.top = 0; //20
c.gridx = 1;
c.gridy = 2;
lPanel.add(lblBox2, c);
mPanel.add(lblBox5, c);
c.gridx = 2;
lPanel.add(txtBox2, c);
mPanel.add(txtBox5, c);

// ROW 4
c.gridx = 1;
c.gridy = 4;
lPanel.add(lblAccKey, c);
c.gridx = 2;
c.gridy = 4;
mPanel.add(lblDB, c);

topPanel.add(lPanel, BorderLayout.EAST);
topPanel.add(mPanel, BorderLayout.EAST);
topPanel.add(rPanel, BorderLayout.EAST);

```

```

        return topPanel;
    }

    /**
     * Calls a servlet that extends PrmServlet
     */
    private Document callServlet(String accountKey, int ActionType)
    {
        Document responseDoc = null;

        try
        {
            //XmlProc is a utility class that handles the communication with
            //the servlets. It handles encryption/compression of the message
            //It takes a PrmParams object which is a hashmap of name/value pairs
            HashMap params = new HashMap();
            params.put("ActionType", new Integer(ActionType).toString());
            params.put("AccountKey", accountKey);
            params.put("sDate", txtBox1.getText());
            params.put("ActionID", txtBox2.getText());
            params.put("Notes1", txtBox4.getText());
            params.put("Notes2", txtBox5.getText());

            params.put("WhereClause", " = '" + accountKey + "'");
            xmlProc.submitRequest("PRMAddInSampleServlet", params);
            //xmlProc.submitRequest("PrmSampleSql", params, false);

            responseDoc = xmlProc.getResponseDoc();
        }
        catch (Exception e)
        {
            // Let the user know we had problems at the server
            msgHandler.displayException(e, prmLog);
        }

        return responseDoc;
    }

    /**
     * Create the screen and text area
     */
    private void drawScreen()
    {
        Container cp = this.getContentPane();
        cp.setLayout(new BorderLayout());

        cp.add(buildTopPanel(), BorderLayout.NORTH);
        cp.add(buildButtonPanel(), BorderLayout.EAST);
        cp.add(buildContentPanel(), BorderLayout.WEST);

        this.setVisible(true);
    }

    /**
     * @param respDoc
     */
    private void processResponse(Document respDoc)
    {
        //Iterate nodes of response doc
        //KSW - Replaced RowData with Item
        List rows = respDoc.selectNodes("//MethodResponse/Results/AddIn");
        //$NON-NLS-1$

        //responseText.insert("", 30);
        responseText.setText("");

        for (Iterator rowIterator = rows.iterator(); rowIterator.hasNext();)
        {
            // Get a rule type node

```

```

        Node rowNode = (Node) rowIterator.next();

        responseText.insert(rowNode.asXML() + "\n",
responseText.getText().length());
    }
    // COPY THE RESULT TO THE CLIPBOARD
    responseText.selectAll();
    responseText.copy();
}

public class TableMouseListener
    extends MouseAdapter
{
    public void mouseClicked(MouseEvent e)
    {
        int row = table.rowAtPoint(e.getPoint()); //what row did they click
    }
}

public class MyEventListener
    implements ActionListener
{
    public MyEventListener()
    {
    }

    public void actionPerformed(ActionEvent e)
    {
        // Get the name of the button
        JButton sourceButton = (JButton) e.getSource();

        if (sourceButton == exitButton) //$NON-NLS-1$
        {
            // setVisible(false);
            startShutDown();
            dispose();
        }
        else if (sourceButton == addButton)
        {
            responseText.setText("");

            //Get xml response document from server
            Document respDoc = callServlet(ak, AddIn_ADD);
            lblDB.setForeground(java.awt.Color.blue);
            lblDB.setText("Table: ACT");

            //Load the text area with the response
            processResponse(respDoc);
        }
        else if (sourceButton == editButton)
        {
            responseText.setText("");

            //Get xml response document from server
            Document respDoc = callServlet(ak, AddIn_EDIT);
            lblDB.setForeground(java.awt.Color.green);
            lblDB.setText("Table: ACT");

            //Load the text area with the response
            processResponse(respDoc);
        }
        else if (sourceButton == deleteButton)
        {
            responseText.setText("");

            //Get xml response document from server
            Document respDoc = callServlet(ak, AddIn_DELETE);
            lblDB.setForeground(java.awt.Color.red);
            lblDB.setText("Table: ACT");

            //Load the text area with the response

```

```

        processResponse(respDoc);
    }
    else if (sourceButton == selButton)
    {
        responseText.setText("");

        //Get xml response document from server
        Document respDoc = callServlet(ak, AddIn_SELECT);
        lblDB.setForeground(java.awt.Color.blue);
        lblDB.setText("Table: ACT");

        //Load the text area with the response
        processResponse(respDoc);
    }
    else if (sourceButton == clearButton)
    {
        responseText.setText("");
    }
    else
    {
        JOptionPane.showMessageDialog(frame, sourceButton.getName() + "
is not working!");
    }
}

public class TypesComboBoxListener
    implements ActionListener
{
    public void actionPerformed(ActionEvent e)
    {
        JComboBox typesCombo = (JComboBox) e.getSource();

        if (typesCombo.getSelectedIndex() != -1)
        {
            String selectedCategory =
typesCombo.getSelectedItem().toString();
            Integer id = getIdByValue(selectedCategory);

            try
            {
                //populateTable(id);
                table.updateUI();
            }
            catch (Exception e1)
            {
                objLib_.getExceptionHandler().displayException(e1, prnLog);
            }
        }
    }
}
}

```

8.11 Database Access – Servlet Component

This sample shows you how to access data using UI and Servlet components. This sample also makes use of the xmlProc to ship data between the UI component and the Servlet.

The following is a generalized walkthrough of the logic within Servlet component:

PrmSampleSql.java

- The PrmSampleSql class extends PrmServlet.
- The doGet method establish connection with the database, takes the parameters from PrmParams and executes the query.

```

package com.aciworldwide.prm.servlet;

import com.aciworldwide.prm.PrmRequest;
import com.aciworldwide.prm.PrmResponse;
import com.aciworldwide.prm.User;
import com.aciworldwide.prm.exceptions.PermissionDeniedException;
import com.aciworldwide.prm.exceptions.PrmException;
import com.aciworldwide.prm.util.ReviewScreenType;
import com.aciworldwide.prm.utils.handler.PrmElementListHandler;
import com.aciworldwide.prm.utils.wrapper.PrmQueryRunner;

import java.lang.Integer;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.sql.DataSource;

public class PRMAddInSampleServlet extends PrmServlet<PrmRequest> {
    static final String VERSION = "1.0";
    public static final int AddIn_ADD = 0;
    public static final int AddIn_EDIT = 1;
    public static final int AddIn_DELETE = 2;
    public static final int AddIn_SELECT = 3;
    Integer ActionID;
    Integer sUID;
    PrmQueryRunner qr;
    String AccKey;
    String Notes1;
    String Notes2;
    String sDate;
    int actionType;

    public void getInfo(PrmResponse doc_response)
        throws SQLException
    {
        int MaxRows = 5;
        String sSQL = //TOP 5
            "SELECT ACTION_LEVEL_KEY,dtActDateKey,tDDateKey,nRevId,lActionId,l10n.l10n_value as
action,lState,lStatus,sActAction,sActNotes,sActNotes2 "
            + "FROM Act a "
            + "JOIN ACTIONSTYPE at ON a.lActionId = at.ltype "
            + "JOIN L10N_PROPERTY l10n ON at.stext=l10n.l10n_property_id AND
l10n.locale_id='en_US' "
            + "WHERE ACTION_LEVEL_KEY=? AND ACTION_LEVEL_ID=1 "
            + "ORDER BY dtActDateKey DESC";

        PrmElementListHandler listhandle = new PrmElementListHandler();
        listhandle.setNodeName("AddIn");

        ArrayList<Object> params = new ArrayList<>();
        params.add(AccKey);

        List<?> nodes = (List<?>) qr.query(sSQL, params.toArray(), listhandle, MaxRows);
        doc_response.addElements(nodes);
    }

    public void doAdd()
        throws SQLException
    {
        String sSQL =
            "INSERT INTO Act (ACTION_LEVEL_KEY,
ACTION_LEVEL_ID,dtActDateKey,tDDateKey,nRevId,lActionId,sActAction,sActNotes,sActNotes2) "
            + "SELECT ?,1,GETDATE(),MAX(SD_DateKey),?,?,(SELECT stext FROM actionstype WHERE
ltype=?),?,?,? FROM DETAIL "
            + "WHERE SD_PAN=?";
    }

```

```

        ArrayList<Object> params = new ArrayList<>();
        params.add(AccKey);
        params.add(sUID);
        params.add(ActionID);
        params.add(ActionID);
        params.add(Notes1);
        params.add(Notes2);
        params.add(AccKey);

        qr.update(sSQL, params.toArray());
    }

    private void doUpdate()
        throws SQLException
    {
        String sSQL =
            "UPDATE ACT SET lActionId=?, "
            + "sActAction=(select stext from actionstype where ltype=?),sActNotes=?,sActNotes2=? "
            + "WHERE ACTION_LEVEL_KEY=? AND ACTION_LEVEL_ID=1 AND dtActDatekey=?";

        ArrayList<Object> params = new ArrayList<>();
        params.add(ActionID);
        params.add(ActionID);
        params.add(Notes1);
        params.add(Notes2);
        params.add(AccKey);
        params.add(sDate);

        qr.update(sSQL, params.toArray());
    }

    public void doDelete()
        throws SQLException
    {
        String sSQL = "DELETE ACT WHERE ACTION_LEVEL_KEY=? AND ACTION_LEVEL_ID=1 AND dtActDatekey=? ";

        ArrayList<String> params = new ArrayList<>();
        params.add(AccKey);
        params.add(sDate);

        qr.update(sSQL, params.toArray());
    }

    public PrmResponse doGet(User user, PrmRequest prmRequest)
    {
        PrmResponse doc_response = new PrmResponse(getServletName(), VERSION);

        DataSource ds = getNrtDataSource(user.getTenantId());

        try
        {
            qr = new PrmQueryRunner(ds);
            sUID = user.getId();

            // Required Parameters
            AccKey = prmRequest.getParameter("AccountKey");
            sDate = prmRequest.getParameter("sDate");

            // Not Required Parameters
            String sAID = prmRequest.getParameter("ActionID");
            ActionID = Integer.valueOf(sAID);
            Notes1 = prmRequest.getParameter("Notes1");
            Notes2 = prmRequest.getParameter("Notes2");

            actionType = Integer.parseInt(prmRequest.getRequiredParameter("ActionType")); //$NON-NLS-1$

            switch (actionType)
            {
                case AddIn_ADD:
                    doAdd();
                    getInfo(doc_response);
            }
        }
    }

```



```

        break;

        case AddIn_EDIT:
            doUpdate();
            getInfo(doc_response);
            break;

        case AddIn_DELETE:
            doDelete();
            getInfo(doc_response);
            break;

        case AddIn_SELECT:
            getInfo(doc_response);
            break;

    }
}
catch (Exception e)
{
    doc_response.setStatus(new PrmException(e.getMessage()));
}
return doc_response;
}

public void init(ServletConfig config)
    throws ServletException
{
    super.init(config);
}

public void checkPermission(User user, PrmRequest prmRequest) throws PrmException {
    if (!user.hasPermission(ReviewScreenType.ActionLevel, 1, 2111222111)) {
        throw new PermissionDeniedException();
    }
}
}
}

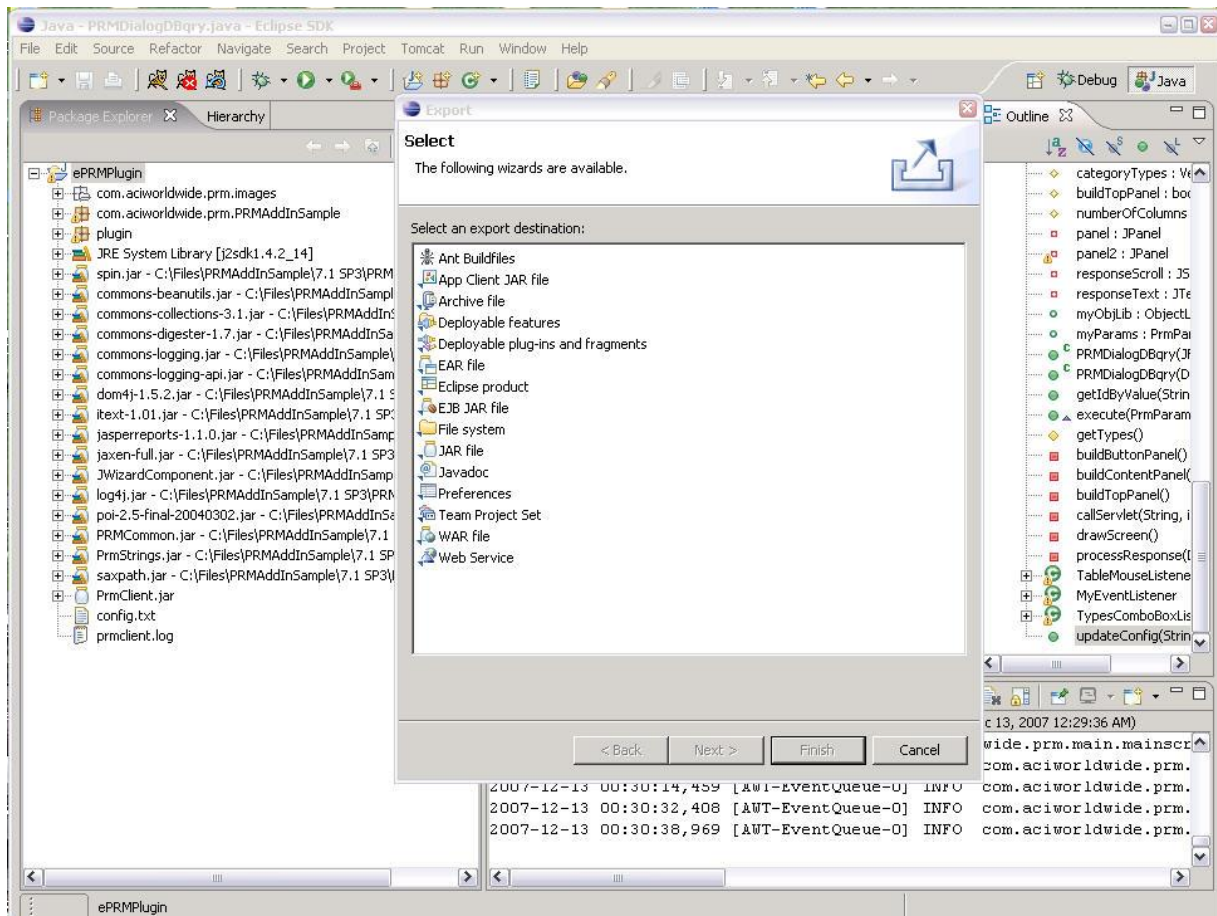
```

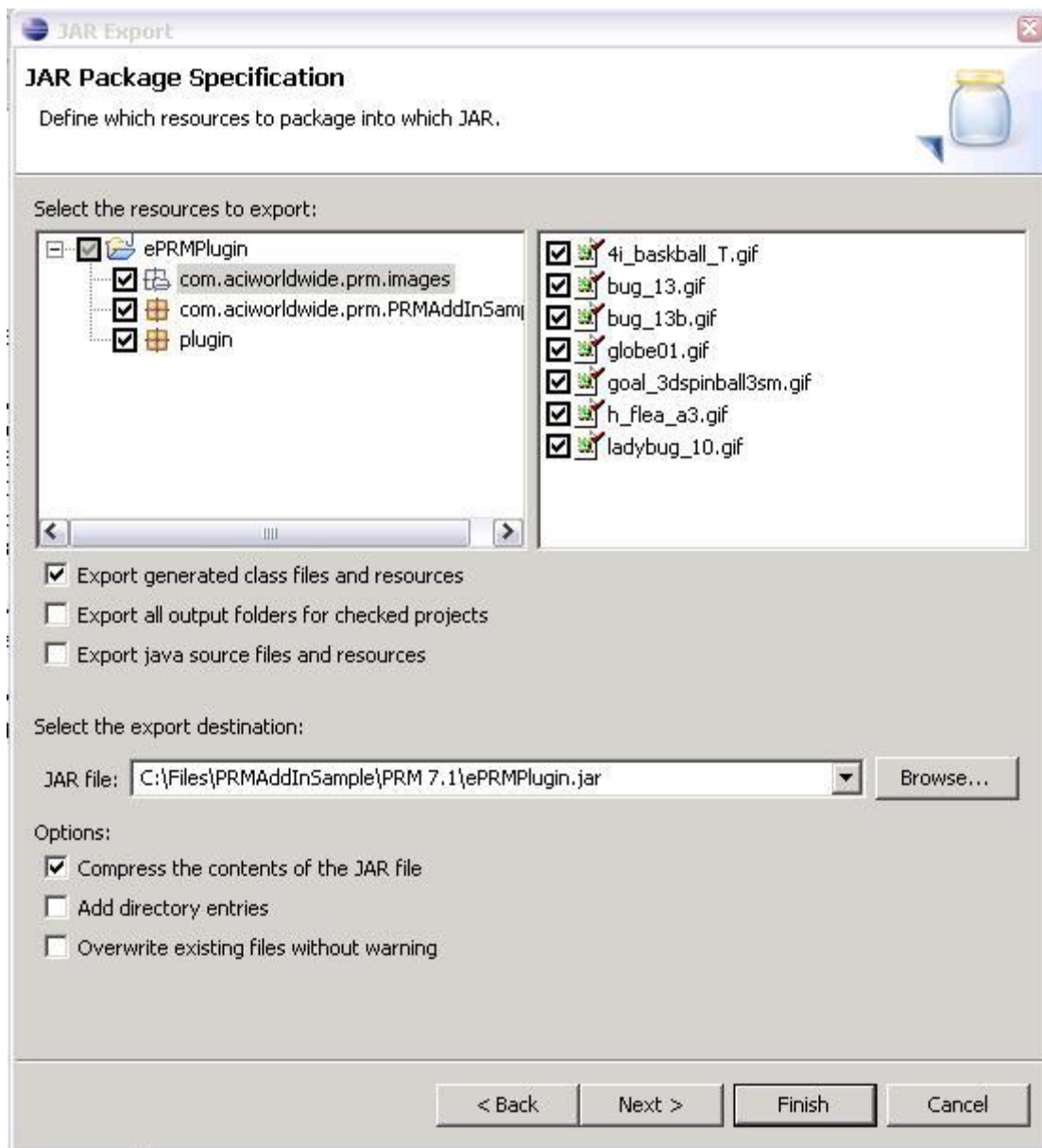
8.12 Packaging the Add-in

You must JAR your add-in in order to be included within the Web Start distribution process. Specific information can be found using the link provided in the documentation, however generally you should jar your Add-In according to the following procedure:

Using Eclipse:

1. Select the project in the Package Explorer.
2. Right click and select Export.
3. Select JAR File and follow online directions.





Using the Command Line Utility: *(step 2 & 3 are not needed and might complicate the installation of the JAR file)*

1. `jar cf myaddin.jar myaddin.class myPRMDialogDBSQL.class`
2. While not required you can create a key in the keystore (or use one you already have). You will be prompted for information like first and last name. At a minimum, you should fill in this information.
`Keytool -genkey -keystore myKeys -alias jdc`
3. Sign the JAR. Be sure to remember your password from the previous step. `Jarsigner -keystore myKeys SamplePlugin.jar jdc`

Configuring the JNLP

The next step is to include your addin (jar file) with PRM's JNLP file to allow Web Start to load your addin onto the client workstation along with the rest of the PRM code. You accomplish this by in two steps, 1) by modifying the JNLP resource file supplied with PRM to include a reference to your own JNLP file and 2) writing your own JNLP file that includes your custom add-in.

You add a reference to your JNLP in the main PRM JNLP file as an extension in the references section. In the example below the PRMAddIn.jnlp file has been included by adding this reference:

<extension href="PRMAddIn.jnlp"/>

```
<?xml version="1.0" encoding="utf-8"?>

<!-- JNLP File for PRM Application -->

<jnlp
  spec="1.0+"
  codebase="https://localhost:8443/PrmClient"
  href="https://localhost:8443/PrmClient/PrmClient.jnlp">

  <information>
    <title>PRM 9.0.2.0</title>
    <vendor>ACI Worldwide</vendor>
    <homepage href="PrmWelcome.htm"/>
    <description>PRM Application</description>
    <description kind="short">ACI Worldwide's fraud management product.</description>
    <icon href="prm_about_logo.gif"/>
    <offline-allowed/>
  </information>

  <security>
    <all-permissions/>
  </security>
```

```

<resources>

    <j2se version="1.8" />

    <jar href="lib/ui-client-9.0.2-SNAPSHOT.jar" />

    <jar href="lib/ui-common-9.0.2-SNAPSHOT.jar" />

    <jar href="lib/commons-beanutils-1.9.4.jar"/>

    <jar href="lib/commons-codec-1.13.jar"/>

    <jar href="lib/commons-collections-3.2.2.jar"/>

    <jar href="lib/commons-digester-2.1.jar"/>

    <jar href="lib/commons-lang3-3.8.1.jar"/>

    <jar href="lib/commons-logging-1.1.1.jar"/>

    <jar href="lib/dom4j-2.1.3.jar"/>

    <jar href="lib/itext-2.1.7.js6.jar"/>

    <jar href="lib/jasperreports-6.7.1.jar"/>

    <jar href="lib/jaxen-1.1.6.jar"/>

    <jar href="lib/javahelp-2.0.05.jar"/>

    <jar href="lib/log4j-api-2.13.3.jar"/>

    <jar href="lib/log4j-core-2.13.3.jar"/>

    <jar href="lib/poi-3.17.jar"/>

    <jar href="lib/jgoodies-common-1.8.1.jar"/>

    <jar href="lib/jgoodies-looks-2.7.0.jar"/>

    <jar href="lib/jackson-annotations-2.10.0.jar"/>

    <jar href="lib/jackson-core-2.10.0.jar"/>

    <jar href="lib/jackson-databind-2.10.0.jar"/>

    <property name="jnlp.aci.prm.Url"
value="https://localhost:8443/PrmServer/servlet"/>

    <property name="jnlp.aci.prm.CountryOverride" value="US" />

    <property name="jnlp.aci.prm.LanguageOverride" value="en" />

    <property name="jnlp.aci.prm.log4jURL"
value="https://localhost:8443/PrmClient/log4j2.xml" />

```

```

        <property name="jnlp.aci.prm.lookAndFeel"
value="com.jgoodies.looks.plastic.Plastic3DLookAndFeel" />

        <property name="jnlp.aci.prm.showStackTraceInErrors" value="false" />

        <extension href="PRMAddIn.jnlp"/>

    </resources>

    <application-desc main-class="com.aciworldwide.prm.main.PrmMain" />

</jnlp>

```

The second step is to write a new JNLP that includes your specific jar file. Below is a sample:

```

<?xml version="1.0" encoding="utf-8"?>

<!-- JNLP File for PRM Add-In -->

<jnlp
spec="1.0+"

codebase="https://localhost:8443/PrmClient">

<information>

    <title>PRM Add-In</title>

    <vendor>ACI Worldwide.</vendor>

</information>

<security>

    <all-permissions/>

</security>

<resources>

    <jar href=" lib/SamplePlugin.jar" />

</resources>

    <component-desc/>

</jnlp>

```

8.13 Configuring your Servlet Code

If your addin includes Servlet code you will need to include your class files with the existing PRM class files in the correct PRM context on the Web Server. Typically, PRM Servlets are installed in the web context called PRMServer.

8.14 Configuring the web.xml file containing the servlets

All the new servlets should be registered to web.xml (PRMServer package) file in order to be able to be run.

See the following entries as example:

```
<servlet>

    <servlet-name>GetTableData</servlet-name>

    <servlet-class>com.aciworldwide.prm.servlet.GetTableData</servlet-class>

</servlet>

<servlet-mapping>

    <servlet-name>GetTableData</servlet-name>

    <url-pattern>/servlet/GetTableData</url-pattern>

</servlet-mapping>
```

8.15 Configuring the Database

8.15.1 ACT table trigger removal

The trigger from ACT table was removed. That trigger only updates TD_DATEKEY column; so all the add-ins which relies on this trigger, should be updated.

8.15.2 Adding the add-in toolbar button on the review screen

Since the add-in to be added is a button on toolbar from review screen a new entry should be added into ToolbarButtons table. Also a new entry should be added to in ACTIONSTYPE table.

The rows in the ACTIONSTYPE table are actions that are going to be executed in case of each action levels.

In order to enable or disable an action type for an action level the action type has to be found in the ACTIONSTYPE_ACTION_LEVEL table where one defines an action for an action level and also in the ASSIGNED_ACTIONSTYPE_ACTION_LEVEL table where one can enable an action type (if it is there) or disable an action (if it is not there) by action level.

On the other hand many users can execute actions for action levels so if the customer would like to enable/disable an action by user he can do that by user's role.

A user's role has a set of Security Permissions which is defined in PRMROLEPERMS table. There one has to add an entry with role id, permission id and the action level id.

There are 2 types of permissions one for actions who have to be granted by action level and the other ones who can secure with out the notion of action level.

In case of Actions the definition of the required permission is found in the ACTIONSTYPE table in the PermissionId column.

In order to grant role permission for an action for an action level, an entry has to be added to the PRMROLEPERMS table with all the values filled as needed.

For more information about adding a new action to review screen toolbar please refer to section 2 of this document.

Example for adding ToolbarButtons for PRMAddIn

The PRM UI needs to know that your addin is part of the application. PRM uses a table called toolbarbuttons to know how to configure its toolbar. The following table describes the structure of the toolbarbuttons table:

Variable	Datatype	Description
Id	integer	Unique Row ID
Name	varchar (20)	Name of Button
Caption	varchar (40)	Caption (not used for display)
Icon	varchar (50)	Icon used (located in: PRMClient\bin\com\aciworldwide\prm\images
Hotkey	integer	Used for shortcut
DisplayMnemonic	integer	Null
ToolTip	varchar (200)	Set in PRMStrings, override use "/" in front of text
AltToolTip	varchar (200)	Example: //Sample Plugin
FormType	integer	0
ActionType	integer	corresponding IType in ActionsType table
ReadOnlyEnabled	bit	0=False, 1=True
Visible	bit	0=False, 1=True
Enabled	bit	0=False, 1=True
Selectable	bit	0=False, 1=True
DefaultOrder	bit	Display Order
ServletName	varchar (50)	*Servlet used, Example: DoAction, DoPlugin
Listener	varchar (200)	Class called, Example: com.aciworldwide.prm.sampleplugin.Sample1
ClassName	varchar (255)	Null
Type	integer	0=Main Screen, 1=Transaction Screen,2=Fraud

To include your addin on the toolbar you will need to add a row to the table.

8.16 Installing AddIn jar to Tomcat

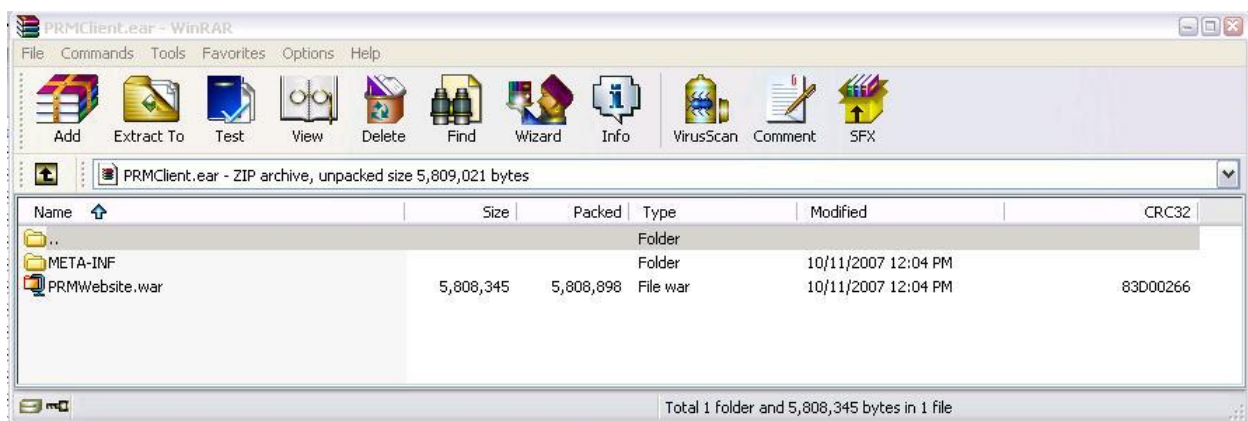
1. Copy new jar file to PRMClient\lib.

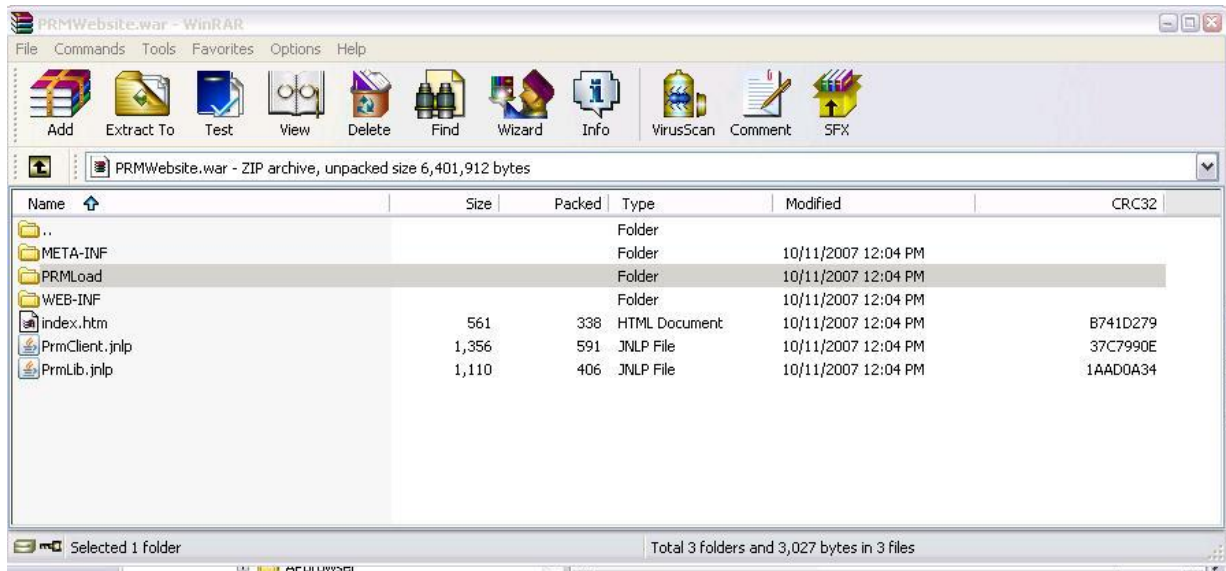
eg. \\prv2prm01\www\client\8.7\PRMClient\lib

2. Update PRMAddIn.jnlp to include the new jar file.
3. Update database.
4. Restart server.
5. Start application.

8.17 Installing AddIn jar to WebSphere

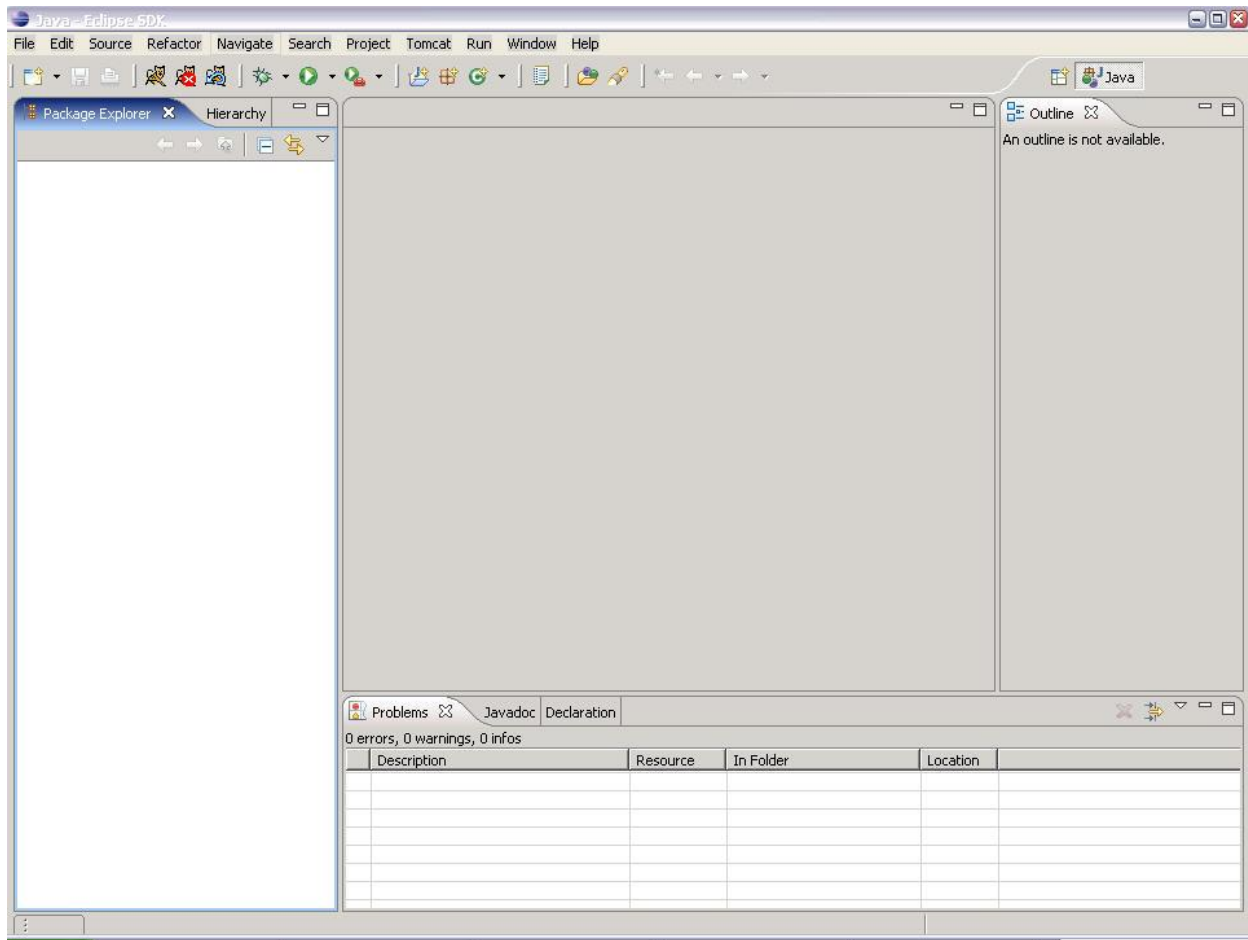
1. Extract PRMClient.war from PRMClient.ear using winzip or winrar.
2. Copy the new jar file to \lib directory in PRMClient.war.
3. Edit the PRMAddIn.jnlp with a text editor and add the new jar file.
4. Copy the new PRMAddIn.jnlp into PRMClient.war.
5. Extract and edit the PRMClient.jnlp file with the reference to the PRMAddIn.jnlp file (see pages 47-48 of this document)
6. Copy the new PRMClient.jnlp into PRMClient.war
5. Copy the new PRMClient.war into PRMClient.ear.
6. Update database.
7. Install WebSphere. Restart server.
9. Start application.



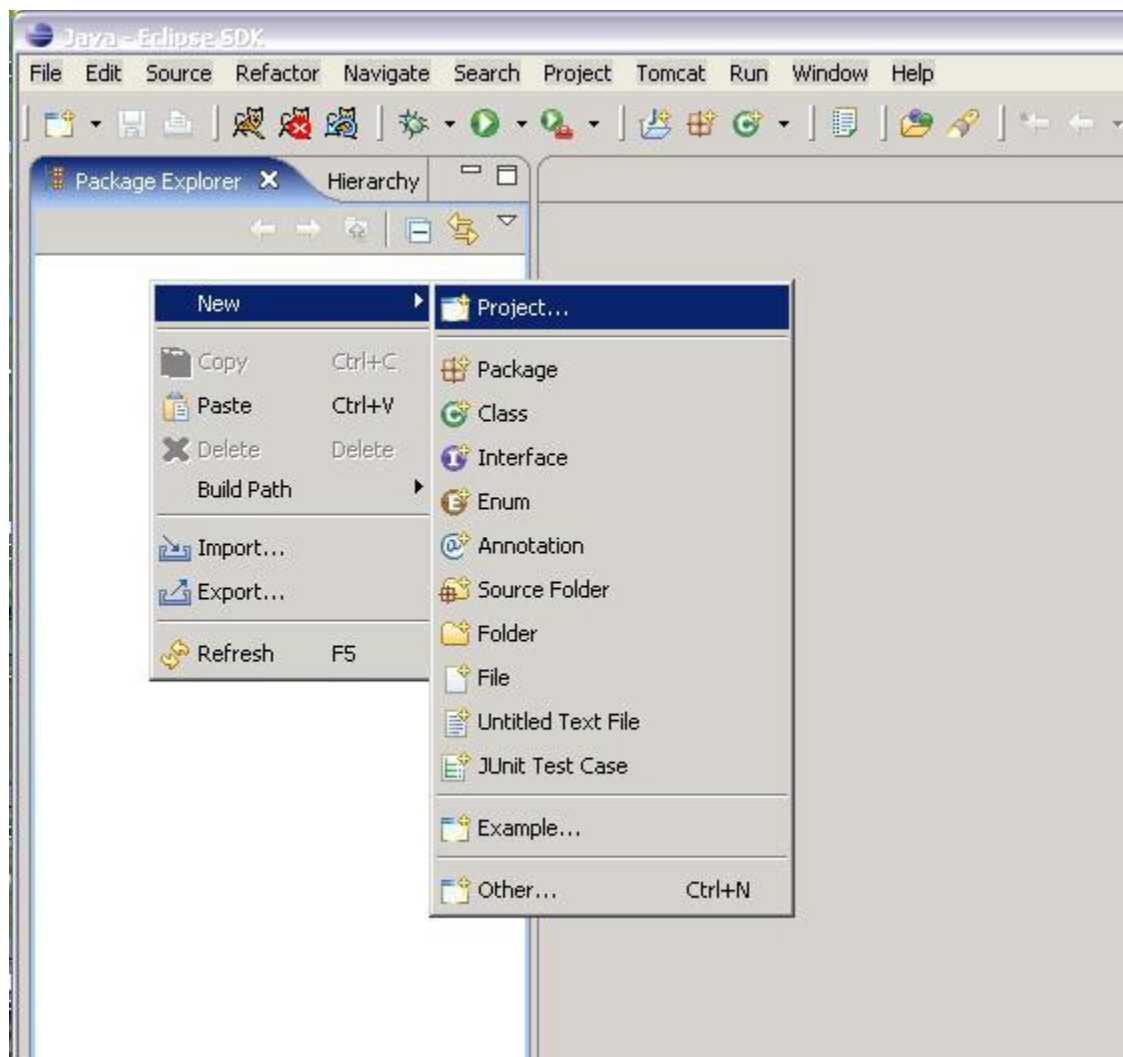


8.18 Setup Add-Ins in Eclipse

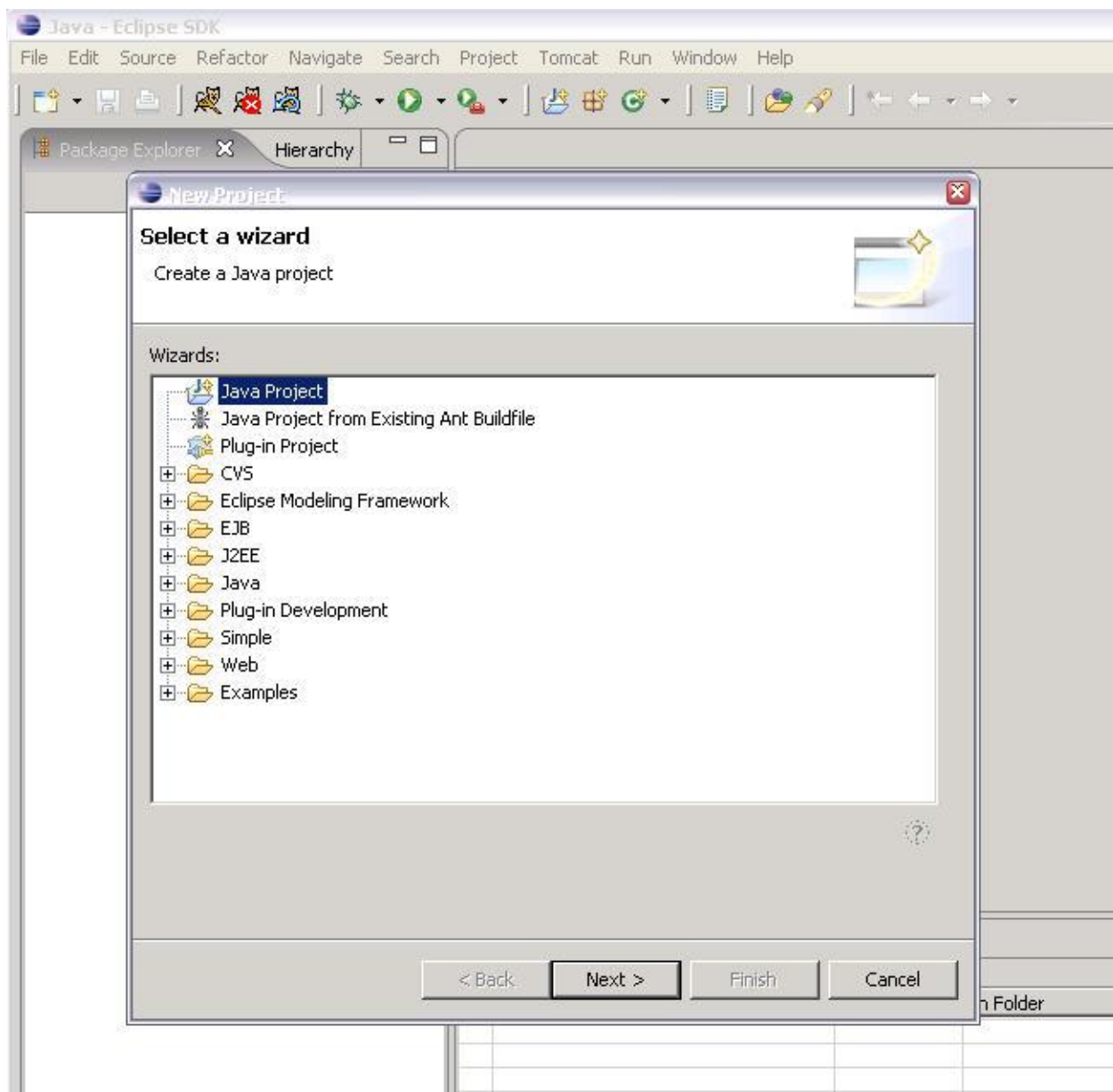
1. Java perspective in Eclipse



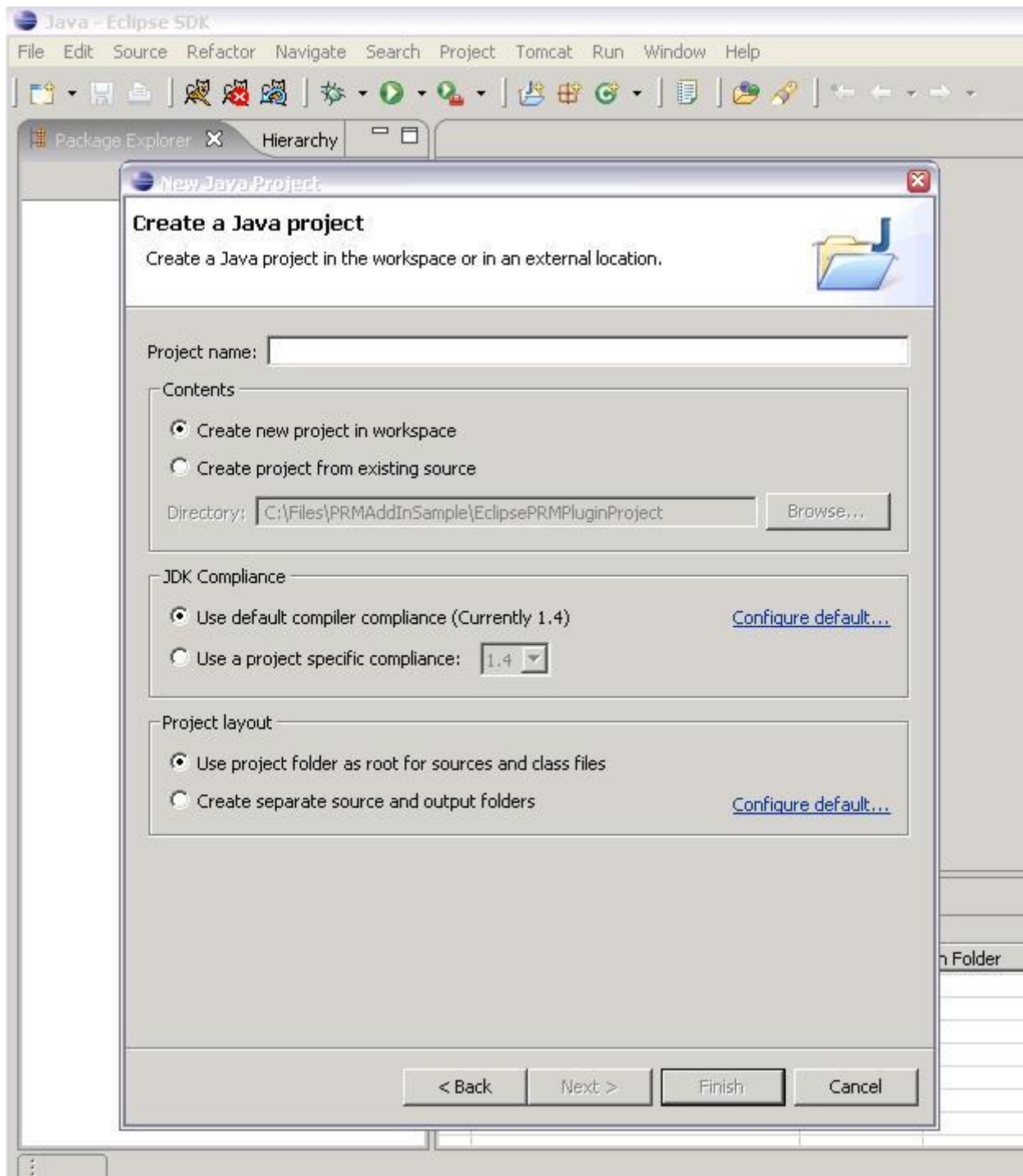
2. Add new Project



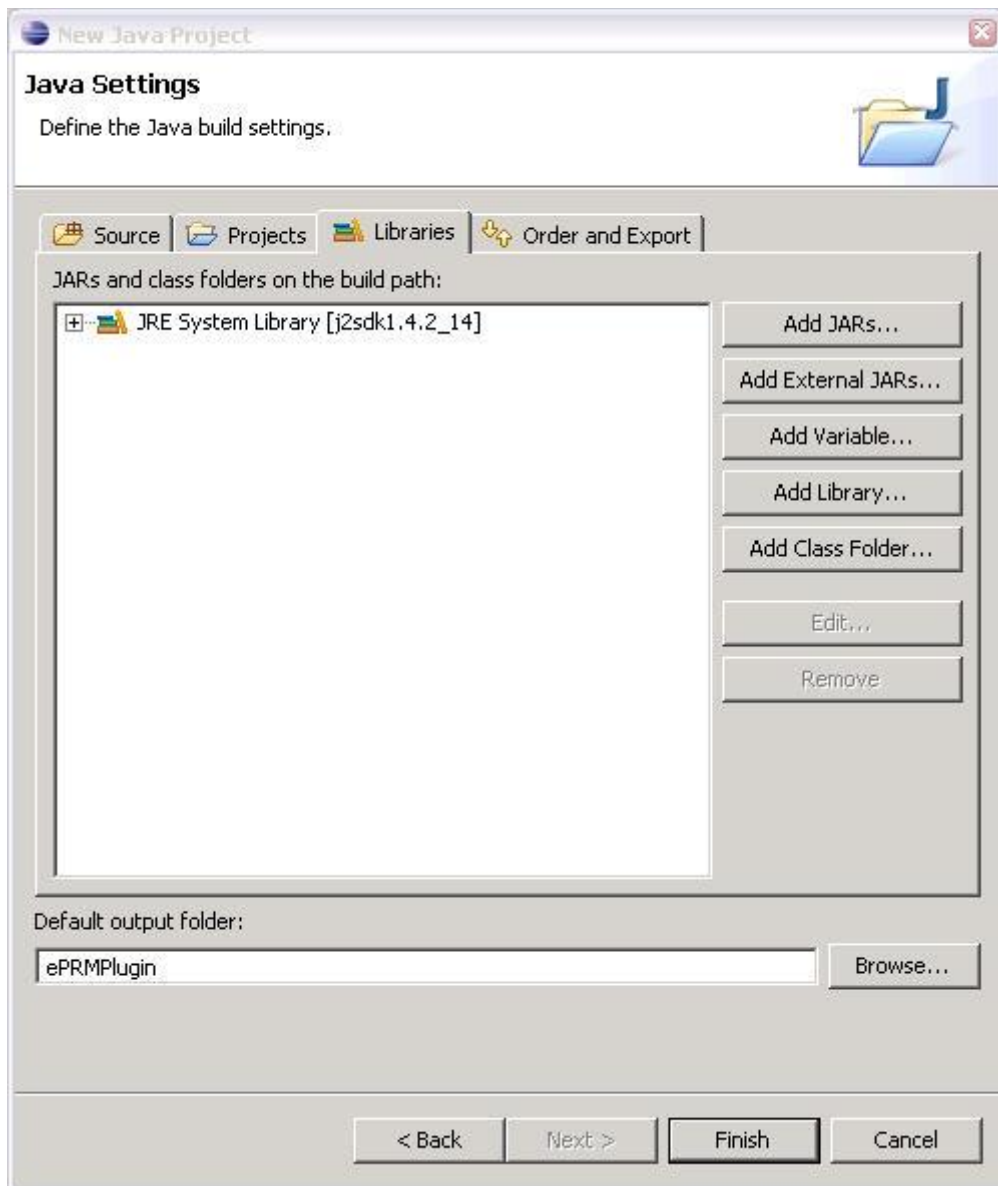
3. Create a Java Project



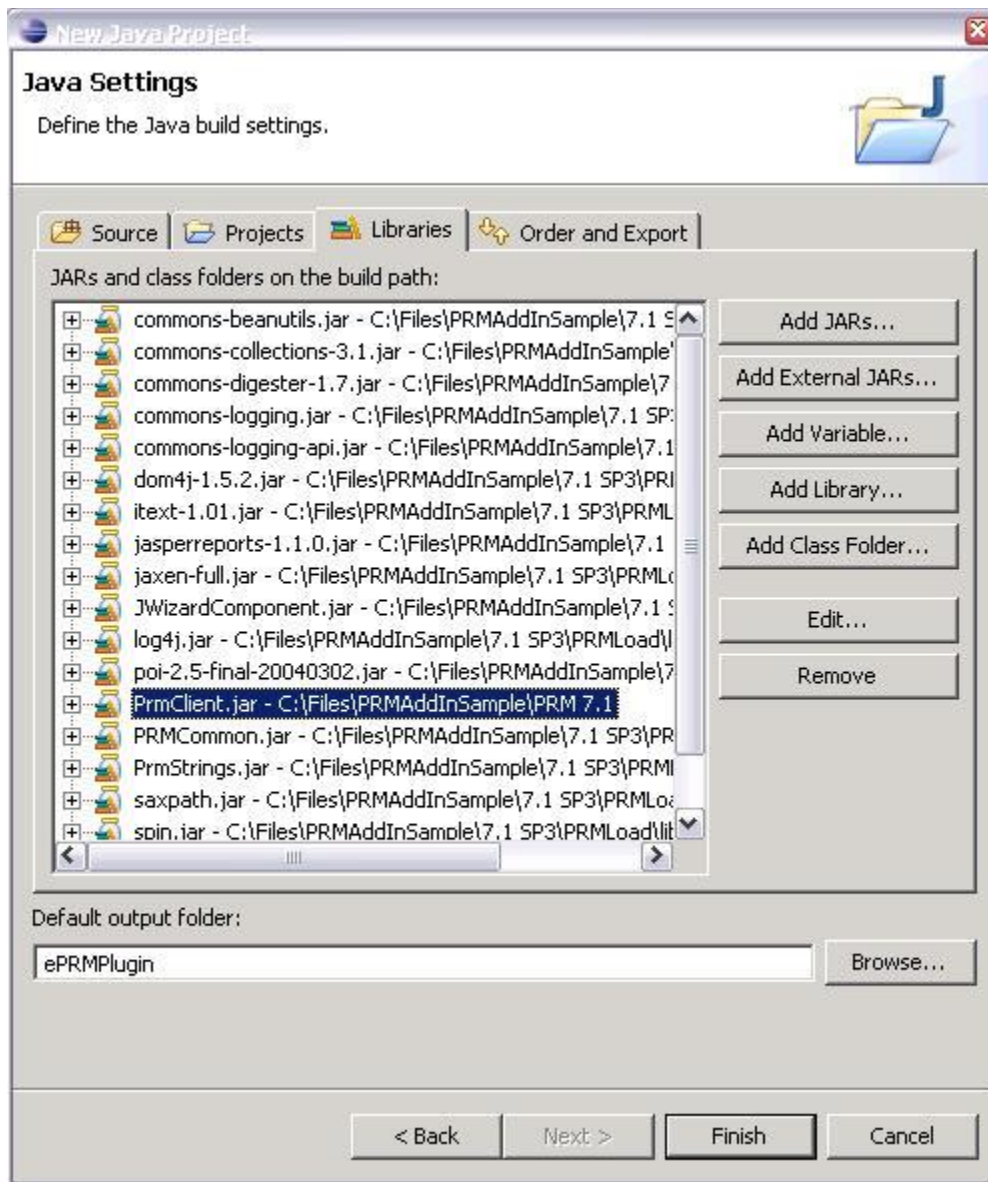
4. Java Project screen



5. Add jar files to Library



6. PrmClient.jar



7. Add java files

- Copy existing java files to the physical location on drive, then refresh Eclipse.
or
- In Eclipse create Package, then Class.

8. Add config.txt

- Copy sample config.txt file from below to the project directory, then refresh.

```
<product>
URL=http://localhost:8080/ui-server/servlet/

<localization>
Country=US
Language=en

<logging>
log4jURL=file:/C:/source/Web/PRMClient/log4j2.xml

<stackTrace>
showStackTraceInErrors=false

<lookAndFeel>
lookAndFeel=com.jgoodies.looks.plastic.Plastic3DLookAndFeel
```

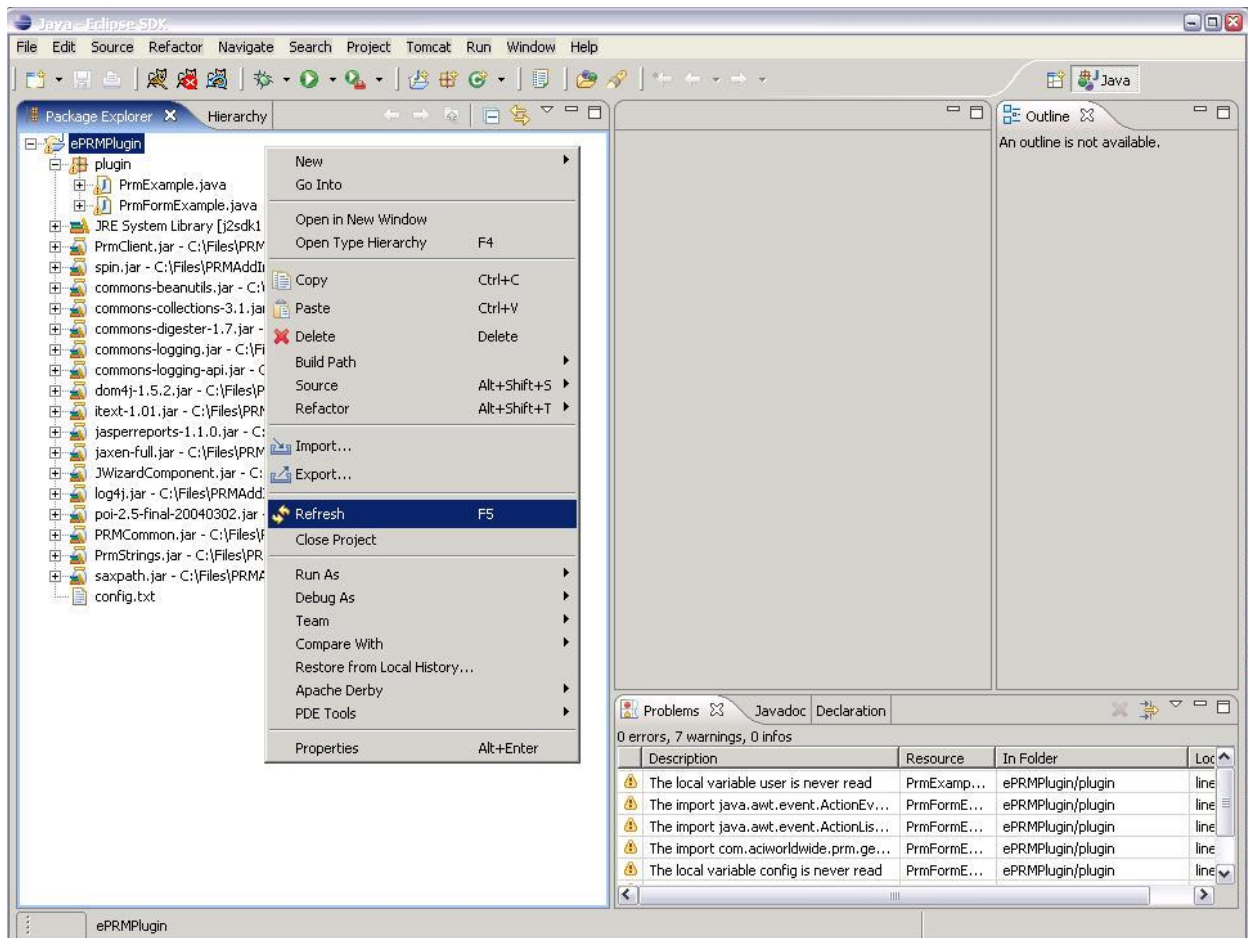
- Open config.txt file, copy URL from a working PRMClient.jnlp file.
- Update path for log4j.properties.

9. Icons location in the PRMAddIn jar

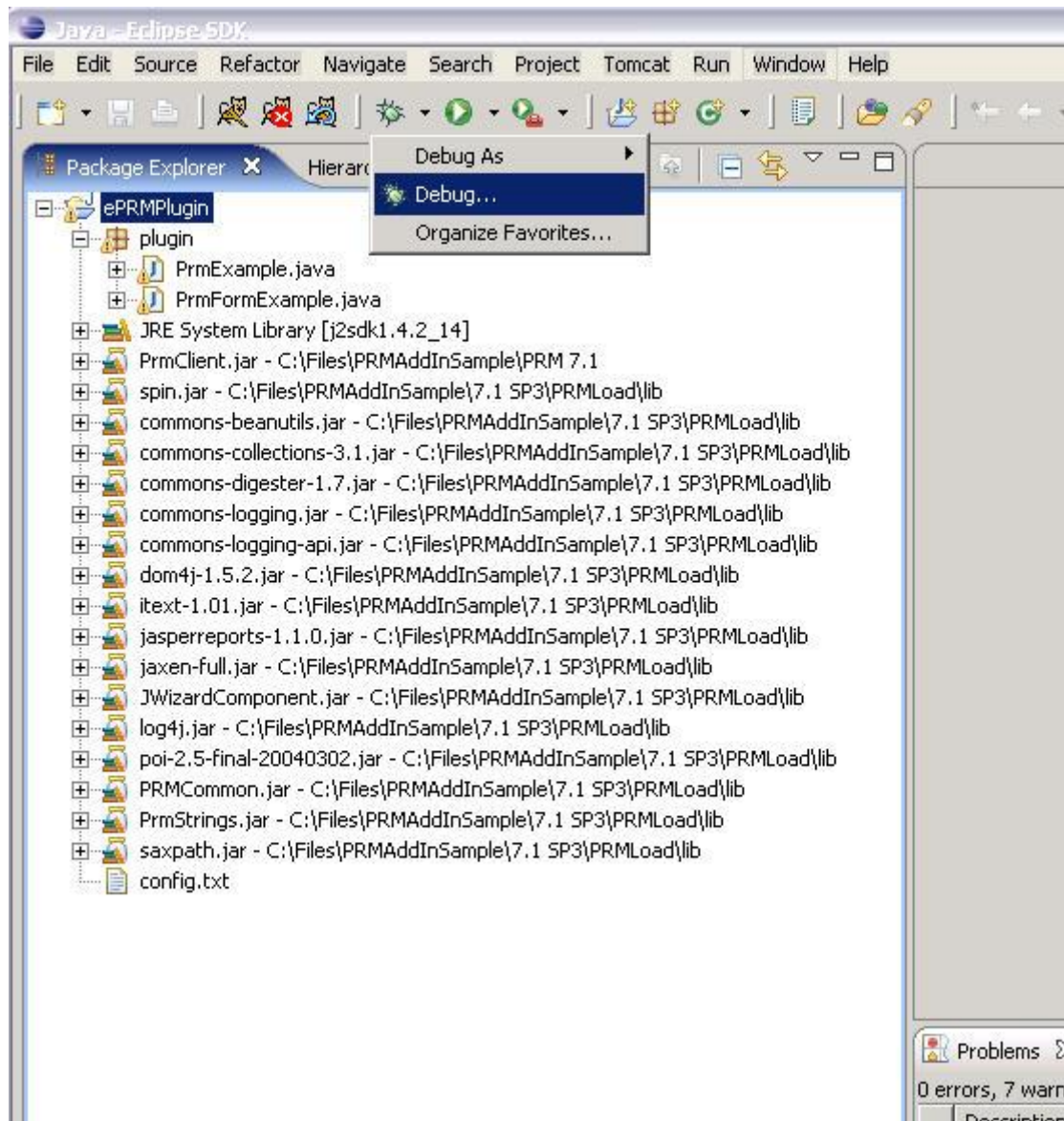
The icons for the buttons that will be configured to appear in the Review Screen toolbar (see section *Configuring the Database | Adding the AddIn to review toolbar button* above) have to be present in \com\aciworldwide\prm\images. This must be done even if the AddIn code the customer has is in a different package of the same jar.

If the icons are not present in the jar at the location \com\aciworldwide\prm\images, the icons will not be loaded. Because of this the AddIn jar file will not be loaded.

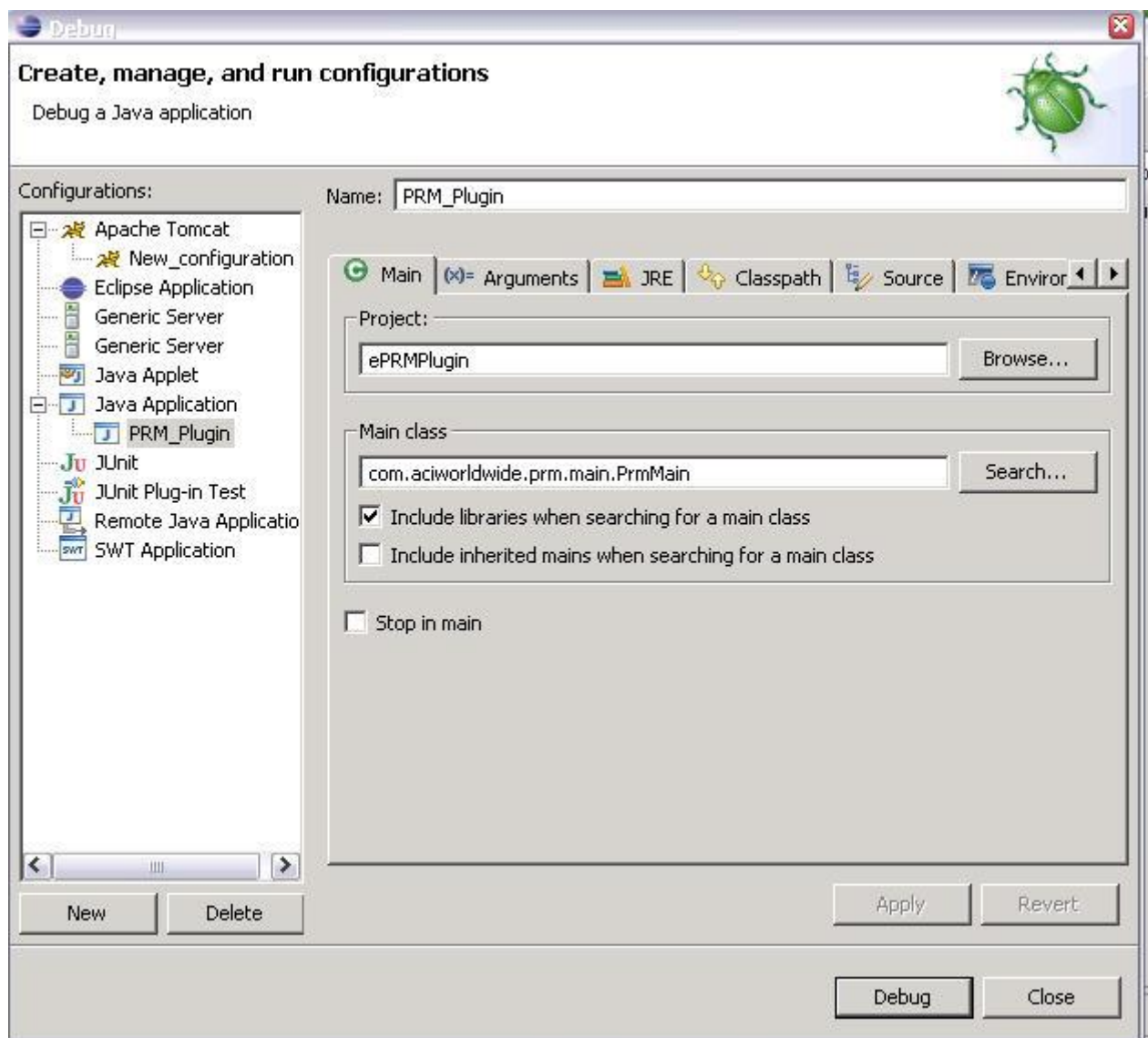
10. Refresh; right click menu in Eclipse



11. Create Debug profile



12. Select PrmMain & check Include libraries...



8.19 Step by Step Example of Configuring the Add-in Functionality

1. DB Configuration

```
Insert INTO L10N_PROPERTY (LOCALE_ID, L10N_PROPERTY_ID, L10N_VALUE)
VALUES ('en_US', 2111222333, 'Test AddIn')
```

- check availability of the L10N_Property_ID first and reserve ids starting from the maximum integer value (2,147,483,647) downwards. In the example the 2111222333 value was used.

```
Insert into PRM_PERMISSION (PERMISSION_ID, DISPLAY_NAME, DESCRIPTION,
PERMISSION_GRP_ID, ASGN_BY_DFLT)
```

```
Values (2111222111, 2111222333, 2111222333, 4, 0)
```

- PERMISSION_ID should also be allocated starting from maxint downwards. In this example, the 2111222111 value was used.

```
INSERT INTO prmroleperms (RoleID, Permission_ID, Action_Level_ID)
```

```
Values (0, 2111222111, 1)
```

```
INSERT INTO prmroleperms (RoleID, Permission_ID, Action_Level_ID)
```

```
Values (1, 2111222111, 1)
```

```
INSERT INTO prmroleperms (RoleID, Permission_ID, Action_Level_ID)
```

```
Values (2, 2111222111, 1)
```

- RoleID 0 - Analyst, 1 - Supervisors, 2 - Administrator

```
INSERT INTO ACTIONSTYPE (LTYPE, NAME, STEXT, AUTO_ACTION, AUTO_ACTION_PROC,
PERMISSION_ID, MULTIPLE_ACTION_LEVELS_ENABLED)
```

```
VALUES (2111222000,'Test AddIn', 2111222333, 0, null, 2111222111, 0)
```

- LTYPE is a numeric id, so it should be allocated in the same fashion (eg 2111222000)

```
INSERT INTO ACTIONSTYPE_ACTION_LEVEL (ACTION_TYPE_ID, ACTION_LEVEL_ID, STEXT,
SHOWSTATE, SHOWSTATUS, TITLEBARSTRING, HASAUTOTEXT, UPDATES_ALLOWED,
AUTO_UPDATES_ALLOWED)
```

```
VALUES (2111222000, 1, 2111222333, 0, 0, 2111222333, 0, 1, 0)
```

```
INSERT INTO ASSGND_ACTIONSTYPE_ACTIONLEVEL (ACTION_TYPE_ID, ACTION_LEVEL_ID)
VALUES (2111222000, 1)
```

```
INSERT INTO TOOLBARBUTTONS (ID, NAME,CAPTION,ICON,HOTKEY,DISPLAYMNEMONIC,
TOOLTIP, ALTTOOLTIP, FORMTYPE, ACTIONTYPE, READONLYENABLED, VISIBLE, ENABLED,
SELECTABLE, DEFAULTORDER, SERVLETNAME, LISTENER, CLASSNAME, TYPE,
PERMISSION_ID)
```

```
VALUES (2111222444, 'Test AddIn', 'AddIn', 'alert.gif', 0, null, '//Test AddIn', 'Add-In', 0, 2111222000, 0,
1,1,1,55, null, null, 'com.MyBank.prm.addin.ui.FraudWatchAddin', 1, 130)
```

- ID is a numeric row identifier, so it should be allocated in the same fashion (eg 2111222444).

2. Check the documentation to configure the servlets in web.xml in PRMServer.

```
<servlet>

    <servlet-name>PRMAddInServlet</servlet-name>

    <servlet-class>com.aciworldwide.prm.servlet.PRMAddInSampleServlet</servlet-class>

</servlet>

<servlet-mapping>

    <servlet-name> PRMAddInServlet </servlet-name>

    <url-pattern>/servlet/PRMAddInSampleServlet</url-pattern>

</servlet-mapping>
```

3. PRM Add-In jars

In order to have AddIn jars loaded into PRM the following conditions have to be fulfilled:

1. These attributes have to be added in the AddIn jars manifest file:
Permissions: all-permissions
Trusted-Library: true
Codebase: *
2. The Jars have to be signed with a CA certificate (see [Signing the Code](#) for instructions)
3. In PRMClient.jnlp, the PRMAddIn.jnlp file has to be referenced <extension href="PRMAddIn.jnlp"/>
4. In PRMAddIn.jnlp file we need to have:
<security>

```
<all-permissions/>

</security>
```

Exactly like in the PRMClient.jnlp

8.19.1 Signing the Code

To sign the code, obtain a valid Java code signing certificate from a valid Certificate Authority. ACI uses Thawte for ACI code signing. Additional information on generating the public/private key and obtaining the code signing certificate can be found on the Thawte web site: <https://www.thawte.com/code-signing/content-signing-certificates/sun-java/>. If desired, generate a self-signed certificate for the purposes of testing the code signing. This requires that the Mixed Code Security Verification be disabled in the JVM, which will reduce the security level, and should only be done for testing purposes.

Use the following command to sign the jars:

```
JARSIGNER -keystore <keystore> -storepass <keystore_password> -keypass <key_password>
-tsa <timestamp_authority> -digestalg <digest_algorithm> <jar_file> <key_alias>
```

Where:

<keystore> = The Java Keystore where the private key and certificate reside. This is created as part of the procedure to obtain the code signing certificate from the Certificate Authority.

<keystore_password> = Password for the key store. Created during the key generation process.

<key_password> = Password for the specific private key. Created during the key generation process.

<timestamp_authority> = URL for the Timestamp authority. This is optional and is used to add a signed timestamp to the signature so that even after the signing certificate expires the signed jar will continue to be considered valid by the JVM. ACI uses <https://timestamp.geotrust.com/tsa> for this value.

<digest_algorithm> = Hash algorithm used during the signing process. ACI uses SHA-256 for this parameter.

<jar_file> = Jar file to be signed.

<key_alias> = Private key/certificate alias name. Created during the key generation process.

The JARSIGNER utility is a part of the JAVA JDK package, which will need to be installed in addition to or in lieu of the standard Java JRE.

For the JVM to accept the application signatures without having to reduce/disable the security level, sign not only the add-in jar, but all of the standard product jars. This is due to the JVM enforcing that all jars in a given application be signed by the same certificate. Each PrmClient jar needs to be edited for the factory signature to be removed and re-signed. On Windows, a shell-script like this may be used:

```
for /r %v in (*.jar) do zip -d %v META-INF\PRM.DSA META-INF\PRM.SF
```

```
for /r %v in (*.jar) do jarsigner -keystore <keystore> -storepass <keystore_password> %v  
<key_alias>
```


9 Localization Utility

The default Localization properties file in PRM 8.3 is called PrmStrings.properties and it can be found in PrmServer \ Strings folder. PrmStrings.properties file contains the titles of all PRM buttons, menus, statuses, and errors. For example, PrmStrings.properties file enables re-titling the options in 'File' menu dropdown, as well as re-titling the 'File' menu option itself.

There are two more properties files that enable Localization, but to a lesser extent. Their names are ReportStrings.properties and PrmImagesStrings.properties.

- ReportStrings.properties file contains the information headings displayed within PRM Reports. For example, you can edit this file to change Rules by Portfolio Report's information headings: Portfolio, Rule, Description, SQL, and Rule Type.
- PrmImagesStrings.properties file contains the titles of PRM button icons. These should not be changed by customers without expressed written approval from the PRM Team.

The PrmServer \ Strings folder is available only on the Server; it does not reside on users' individual machines. New Localization properties files can be created for additional locales and all Localization properties files should be kept in the PrmServer \ Strings folder. For example, a new file PrmStrings_fr_FR.properties can be created and stored in PrmServer \ Strings folder for French-speaking locale(s).

NOTE: Folder PrmServer \ Strings was created expressly to store Localization files. This enabled removing PrmStrings.properties file from the PrmCommon project. Furthermore, PrmStrings.properties file was removed from the PrmStrings.jar file for easier maintenance and configuration updates.

Each PrmStrings.properties localization file for a particular locale, together with the corresponding contents of PRM tables L10N_PROPERTY and L10N_PROPERTY_ADDITIONAL_INFO form all texts need for that locale across PRM application.

So, for the application to be available in French, the user should have PrmStrings_fr_FR.properties, ReportsStrings_fr_FR.properties and the inserts in L10N_PROPERTY, L10N_PROPERTY_ADDITIONAL_INFO tables for fr_FR locale.

The Localization Utility can be used to export localization settings for a particular locale from the database into a flat file (Java Style Properties file). The Utility can also be used to import translated data from the flat file to the database tables.

Only PRM Root Administrator and System Administrator can facilitate and configure translation of localized values.

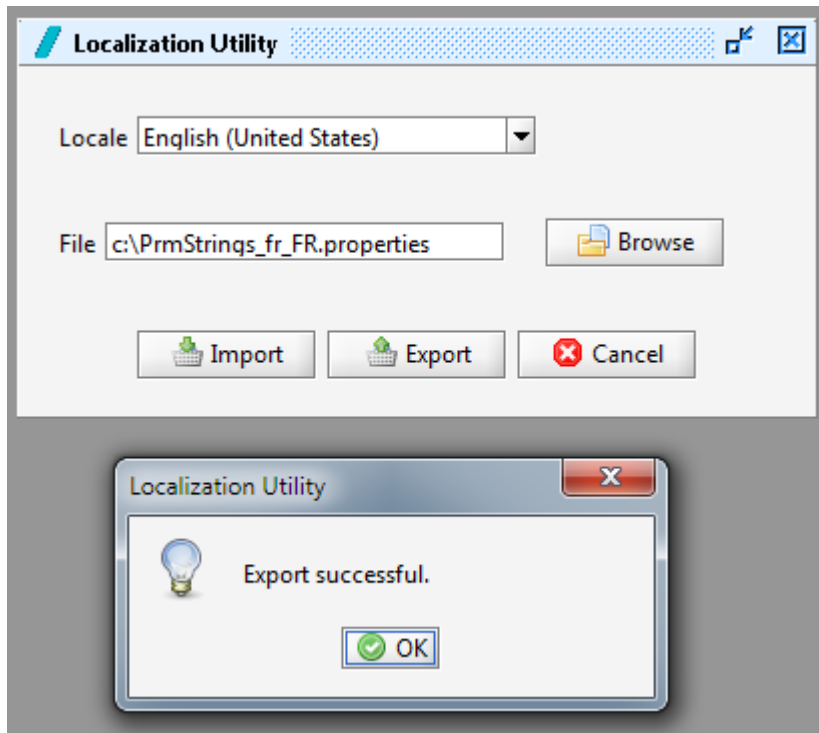
9.1 Add a new locale to the Database Configuration

This task is a prerequisite for importing or exporting a localization property file.

1. Stop the webserver.
2. Add the new Locale to the database by executing the queries shown below. Replace the 'fr_FR' locale used in this example, with your desired Locale.
insert into L10N_PROPERTY values ('fr_FR', 280002,'Francais',38);
insert into AVAILABLE_LOCALE values ('fr_FR',280002);
insert into L10N_PROPERTY_additional_info values('fr_FR', 280002, 'Francais',1);
insert into L10N_PROPERTY values('fr_FR',0,'Francais Resource Not Found',38);
insert into L10N_PROPERTY_additional_info values ('fr_FR',0,'Francais Message Displayed if the Property is not available',1);
insert into L10N_PROPERTY values ('en_US', 280002,'French',38);

3. Start the server
4. Login as root administrator
5. Select Config->Localization Utility.

Select en_US locale, Choose a location. (enter the path ,name and extension for the new file that will be created after export) and click Export.



6. Navigate to the file and translate the content for French language and save the changes. (Do note that there are some labels which contain 1000\$ and 1500\$, that are numbers, and should not be translated).
7. Import the new localization file using Localization Utility .(with root administrator)
8. Change the Locale ID for the user(s) you want to have the new Locale:
`update PRM_USERS set locale_id='<<new_locale>>' where sname='<<user name>>'`
9. Restart the webserver.

Note: In case you want to change the locale for all users, also in PRMClient.jnlp file the language (en/fr) and locale (US/FR) need to be changed.

9.2 Export a translation properties file

1. Open the **Config** dropdown menu and select **Localization Utility** to display the Localization Utility screen. The following fields are displayed:

Field	Description
Locale	The locale name to appear on menus for the translation language.
File	The name of the properties file (.properties file extension) being imported.

2. Select the Locale pull-down menu and click the name of an existing **Locale** that you wish to use to create a new properties file locally. On default, you will see only en_US option.
3. Enter the name that you wish to assign to the new **Properties** file. This file name must include the .properties extension. If necessary, click **Browse** to assist you in setting the fully-qualified path for the new file.
4. Click **Export**.

9.3 Import a Localization Settings file

1. Open the **Config** drop-down menu and select **Localization Utility** to display the Localization Utility screen. The following fields are displayed:

Field	Description
Locale	The locale name to appear on menus for the translation language.
File	The name of the properties file (.properties file extension) being imported.

2. Enter the **Locale**.
 - Enter the name of a new locale that you wish to add. Or,
 - Select the Locale pull-down menu and click the name of an existing Locale that you wish to replace.
3. Enter the location of the **Properties** file to be imported. If necessary, click **Browse** to assist you in setting the fully-qualified path to the file.
4. Click **Import** to add or replace the properties file, then click **OK** to verify the action.

10 XSD Builder Tool

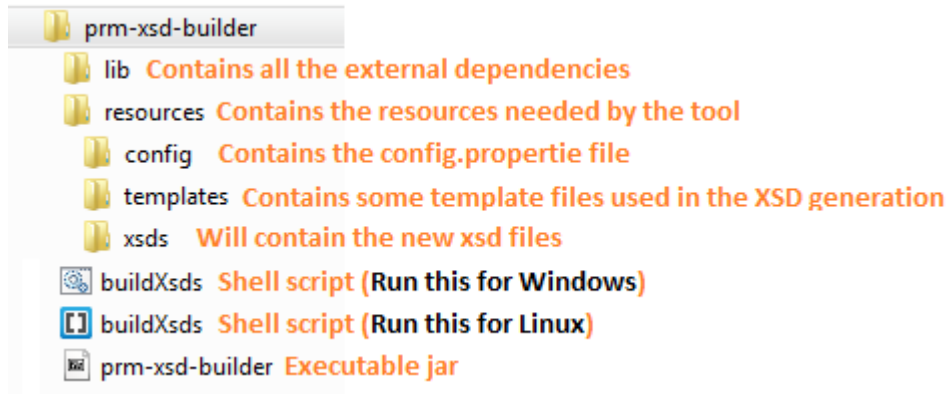
10.1 Prerequisites

- PRM system deployment
- In order to use this tool, the user will need to know the username, password for the prm databases.
- This tool uses the Data Extensibility tables to build the XSD files. Any change to the database (adding columns, changing column dimension) should be done with the Data Extensibility tool.
- Before running this tool the user should set the JAVA_HOME variable

10.2 Introduction to XSD builder tool

XSD builder tool can be used to generate XSD files for all the record sources that can be found in the Data Extensibility tables.

10.3 Unzip the distribution



10.4 Configure the tool

In the config directory the user can find the config.properties file. This contains the following parameters:

- **xfTemplate** - contains the path to the XF.xsd file template
- **xfRecordFormatsTemplate** - contains the path to the XFRecordFormats.xsd file template
- **xfBaseTemplate** - contains the path to the XFBase.xsd files template
- **xfXsdsOutput** – the location where the xsd's will be saved
- **xfXsd2Build** – possible values are:
 - **all** – with this option the tool will create xsd files for all the XF record sources that are defined in the DE_RECORD_SOURCE table.
 - **any XF record source name** – with this option the tool will generate xsd files only for the record source defined by this option. An example with this option should look like this:
xfXsd2Build=CardDemo,AccountDemo,BranchTran
- **stringTypes** – contains the mapping between the xsd string type and the corresponding types from database.
- **integerTypes** - contains the mapping between the xsd integer type and the corresponding types from database.

- **decimalTypes** - contains the mapping between the xsd decimal type and the corresponding types from database.
- **dateTimeTypes** - contains the mapping between the xsd dateTime type and the corresponding types from database.
- **Define exceptions** – the user can remove attributes from the xsd generation by adding the following parameter for each record source `<xf_record_source_name>=<attribute1>,<attribute2>`
- **dbUrl** – url to the PRM database
 - for MSSQL - `jdbc:jtds:sqlserver://<IP>:<PORT>/<DB_NAME>;SendStringParametersAsUnicode=false`
 - for Oracle - `jdbc:oracle:thin:<IP>:<PORT>:<SID>`
- **dbDriver** – database driver
 - for MSSQL - `net.sourceforge.jtds.jdbc.Driver`
 - for Oracle - `oracle.jdbc.OracleDriver`
- **dbUser** – PRM database user
- **dbPwd** – PRM database user password

10.5 Run the tool

Windows

- run the buildXsds.cmd file

Linux

- open the buildXsds.sh file and change the java path
- run the buildXsds.sh file

10.6 Integrate the new XSD files in the PRM transaction generator (Developers only)

1. Generate the xsd's with prm-xsd-builder tool.
2. Copy the xsd files to `.../ag-core/src/main/xml-schema-compiler/xsd/`
3. Run `.../ag-core/src/main/xml-schema compiler /compile_xml_schema.bat` file.
 - a) This will generate Java code from the XSD schema.
 - b) The generated Java classes will be placed in `.../ag-core/src/main/java/com/aciworldwide/risk/gateway/formatter/xml`
4. Run a build for AG project.
5. Run transaction generator with the `-useXsds` option.

11 Configuring Fields to be Masked in the User Interface

By default, the following PRM demographic and transaction fields are configured to be masked in the PRM user interface: SD_PAN, SM_PAN, and SM_USER_PREV_CRD_NUM.

Other demographic and transaction fields can be configured to be displayed masked by making changes to the PRM_UI_FIELD and PRM_FIELD tables, and if a different masking configuration is desired, the PRM_UI_MASKING table.

11.1 Configure how the field should be masked

The PRM_UI_MASKING table contains masking configurations and allows different masking to be used for one or more fields. Optionally, when configuring a field to be masked, you may also configure how the field should be masked, if you wish to apply a different masking configuration than the masking used for the other masked fields.

The PRM_UI_MASKING table contains four columns:

1. MASKING_ID: The unique identifier for the masking configuration
2. MASK_CHAR: The masking character
3. UNMASK_LEFT: The number of leftmost characters to leave unmasked in the user interface
4. UNMASK_RIGHT: The number of rightmost characters to leave unmasked in the user interface

By default, there is one masking configuration used for the SD_PAN, SM_PAN, and SM_USER_PREV_CRD_NUM fields.

As an example, to add a new masking configuration, the PRM_UI_MASKING table can be updated as follows:

```
insert into PRM_UI_MASKING (MASKING_ID, MASK_CHAR, UNMASK_LEFT, UNMASK_RIGHT)
values (2,*,4,2)
```

The system will apply this masking configuration to any fields in PRM_FIELD and PRM_UI_FIELD that are associated with a MASKING_ID value of 2. The system will leave the four leftmost characters and two rightmost characters of the field value unmasked and will mask all other characters of the field value using an asterisk.

11.2 Configure the field to be masked

Using the SM_RETL_ADDR_1 field as an example, the PRM_FIELD and PRM_UI_FIELD tables should be updated as follows:

```
update PRM_FIELD set DEFAULT_MASKING_ID=1 where FIELD_NAME like 'SM_RETL_ADDR_1'
```

```
update PRM_UI_FIELD set MASKING_ID=1 where COMPONENT_ID=5163
```

12 Custom Real-Time Dispositions

12.1 Adding Real-Time Dispositions

New real-time disposition values can be added directly into the database. The table that holds the dispositions is RQDISPOSITION.

To insert new real-time dispositions in the RQDISPOSITION table, perform the following operations:

1. Insert a new row in the L10N_PROPERTY table:

```
insert into L10N_PROPERTY values ('en_US', 305004, 'Disposition text', 43);
```

Where:

- “en_US” is the locale for which the new disposition is being added
- 305004 is the L10N_PROPERTY_Id and the value can be determined by running the query :
`select max(sname) + 1 from RQDISPOSITION`
- “Disposition text” is the value that will be shown in the Rule Editor under the disposition drop down
- 43 is the value returned by the query: `select * From L10N_PROPERTY_RANGE where table_name = 'RQDISPOSITION'`

2. Insert a new row in the L10N_PROPERTY_ADDITIONAL_INFO table:

```
insert into L10N_PROPERTY_ADDITIONAL_INFO values ('en_US', 305004, 'RQDisposition Name', 0);
```

Where:

- “en_US” is the locale for which the new disposition is being added; has to be the same value used in first step insert
- 305004 is the L10N_PROPERTY_Id and needs to have the same value used in the step 1
- 'RQDisposition Name' can be any text

3. Insert a new row in the RQDISPOSITION table:

```
insert into RQDISPOSITION values (3, 305004, 'X', 4);
```

Where:

- 3 is LID of the new disposition and has the value determined by running the query: `select max(LID) + 1 from RQDISPOSITION`
- 305004 is the L10N_PROPERTY_Id and needs to have the same value used in the step 1 and 2
- 'X' is the value that will be returned by PRM when the rule with this new disposition is matched
- 4 is the value returned by the query: `select max(LORDER) + 1 from RQDISPOSITION`

Please note that any new real-time dispositions must be added to the list of acceptable values for the “RealTimeDsps” and “RealTimeRuleDsps” fields from RECORD_SOURCE_FIELD or the AG will reject transactions containing the new values.

To view the currently configured acceptable values, run the following queries:

```
select RECORD_SOURCE_ID, FIELD_NAME, ACCEPTABLE_VALUES_TEXT from  
RECORD_SOURCE_FIELD where field_name = 'RealTimeRuleDsps'
```

```
select RECORD_SOURCE_ID, FIELD_NAME, ACCEPTABLE_VALUES_TEXT from  
RECORD_SOURCE_FIELD where field_name = 'RealTimeDsps'
```

To update the acceptable values, run the following queries on both RT & NRT schemas:

```
update RECORD_SOURCE_FIELD set ACCEPTABLE_VALUES_TEXT = 'null,A,D,R,X' where  
FIELD_NAME = 'RealTimeRuleDsps'
```

```
update RECORD_SOURCE_FIELD set ACCEPTABLE_VALUES_TEXT = 'null,A,D,R,0,N,T,X' where  
FIELD_NAME = 'RealTimeDsps'
```

Where:

- 'null,A,D,R,X' is a string concatenation of the current value (returned by the first query above) which by default is "null,A,D,R" and the new disposition value "X" separated by a comma
- 'null,A,D,R,0,N,T,X' is a string concatenation of the current value (returned by the second query above) which by default is "null,A,D,R,0,N,T," and the new disposition value "X" separated by a comma

12.2 Show custom dispositions in Real-Time Rule Performance widget

For displaying the newly added custom dispositions in the Real-Time Rule Performance widget, there are a couple of steps that are required. If these steps are not taken, the dispositions will still be processed by the report, but they will be processed as "Other dispositions"

In the PRM 9.0 release, the dispositions can be added by updating the report query in the database. Dashboard reports queries are saved in the database, in the TITANIUM_REPORTS table. To retrieve the query that needs to be updated, run the following query:

```
select sql_query from TITANIUM_REPORTS where name = 'rt-rule-performance'
```

The resulting query string is unformatted. For better readability, format it in an SQL formatter. Copy the formatted query string in a text editor, and update it with the following modifications. Save a copy of the original query until you are certain that the query was updated correctly.

In the following example, the Exemption disposition is added with the letter E. Basically, the update consists in replicating one of the default dispositions. You can add as many dispositions as desired. All the required updates necessary for adding new dispositions are highlighted in yellow.

The update will add two new columns to the report: **proc_exemp_tran** – the alerted per mill for the disposition and **total_tran_exemp** – the alerted sum for the dispositions. The new column headers will require an insert in the l10n table, which is described later. In the PRM 9.0 release, only en_US locale is supported. When adding other dispositions, use these templates for defining query vars like they are exemplified for exemp : total_<<disposition>>, total_amt_<<disposition>>, proc_<<disposition>>_tran, total_tran_<<disposition>>, and instead of letter "E", the respective letter for that disposition.

--MSSQL QUERY

```

WITH s
AS (SELECT portfolio_id,
           nd_real_time_rule_fired,
           Count(no_alerts)          total_tran,
           Sum(CASE
               WHEN sd_real_time_rule_disposition = 'A' THEN no_alerts
               ELSE 0
           END)                      total_auth,
           Sum(CASE
               WHEN sd_real_time_rule_disposition = 'D' THEN no_alerts
               ELSE 0
           END)                      total_decl,
           Sum(CASE
               WHEN sd_real_time_rule_disposition = 'R' THEN no_alerts
               ELSE 0
           END)                      total_ref,
           SUM(CASE
               WHEN sd_real_time_rule_disposition = 'E' THEN no_alerts
               ELSE 0
           END)                      total_exemp,
           Sum(CASE
               WHEN sd_real_time_rule_disposition NOT IN
                   ( 'A', 'D', 'R', 'E' )
               THEN
                   no_alerts
               ELSE 0
           END)                      total_others,
           Sum(total_alerted_amt1) total_amt_tran,
           Sum(CASE
               WHEN sd_real_time_rule_disposition = 'A' THEN
                   total_alerted_amt1
               ELSE 0
           END)                      total_amt_auth,
           Sum(CASE
               WHEN sd_real_time_rule_disposition = 'D' THEN
                   total_alerted_amt1
               ELSE 0
           END)                      total_amt_decl,
           Sum(CASE
               WHEN sd_real_time_rule_disposition = 'R' THEN
                   total_alerted_amt1
               ELSE 0
           END)                      total_amt_ref,
           SUM(CASE
               WHEN sd_real_time_rule_disposition = 'E' THEN
                   total_alerted_amt1
               ELSE 0
           END)                      total_amt_exemp,
           Sum(CASE
               WHEN sd_real_time_rule_disposition NOT IN
                   ( 'A', 'D', 'R', 'E' )
               THEN
                   total_alerted_amt1
               ELSE 0
           END)                      total_amt_others
FROM titanium_rt_rule_perf
WHERE dd_date BETWEEN Cast(Dateadd(day, Datediff(day, 0,
                                                Dateadd(day, -1 * :REPORT_INTERVAL_DAYS,
                                                Getdate())
                                                ), 0) AS DATE)
AND Cast (Getdate() AS DATE)

GROUP BY portfolio_id,

```

```

        nd_real_time_rule_fired)
SELECT COALESCE(L10N.l10n_value, l10n1.l10n_value, l10n2.l10n_value) rule_name,
( CASE
    WHEN RQA.ndeployed = 1 THEN
        COALESCE(L10N3.l10n_value, l10n4.l10n_value,
            l10n2.l10n_value)
    ELSE COALESCE(L10N5.l10n_value, l10n6.l10n_value, l10n2.l10n_value)
END )
s.total_tran
total_transactions,
( CASE
    WHEN ( s.total_tran IS NULL
        OR s.total_tran = 0 ) THEN NULL
    ELSE Round(Cast(s.total_auth AS DECIMAL) / Cast(
        s.total_tran AS DECIMAL) *
        100, 4)
END )
proc_auth_tran,
( CASE
    WHEN ( s.total_tran IS NULL
        OR s.total_tran = 0 ) THEN NULL
    ELSE Round(Cast(s.total_decl AS DECIMAL) / Cast(
        s.total_tran AS DECIMAL) *
        100, 4)
END )
proc_declines_tran,
( CASE
    WHEN ( s.total_tran IS NULL
        OR s.total_tran = 0 ) THEN NULL
    ELSE Round(Cast(s.total_ref AS DECIMAL) / Cast(
        s.total_tran AS DECIMAL) *
        100, 4)
END )
proc_ref_tran,
( CASE
    WHEN ( s.total_tran IS NULL
        OR s.total_tran = 0 ) THEN NULL
    ELSE Round(Cast(s.total_exemp AS DECIMAL) / Cast(
        s.total_tran AS DECIMAL) *
        100, 4)
END )
proc_exemp_tran,

( CASE
    WHEN ( s.total_tran IS NULL
        OR s.total_tran = 0 ) THEN NULL
    ELSE Round(Cast(s.total_others AS DECIMAL) / Cast(
        s.total_tran AS DECIMAL) *
        100, 4)
END )
proc_others_tran,
s.total_amt_auth
total_tran_auth,
s.total_amt_decl
total_tran_decl,
s.total_amt_ref
total_tran_ref,
s.total_amt_exemp
total_tran_exemp,
s.total_amt_others
total_tran_others,
s.total_amt_tran
total_tran_amount
FROM ruleque rq
JOIN ruledef rd
    ON ( rd.lid = rq.lruledefid
        AND rd.ltype = 0
        AND RD.lsubtype = 2 )
JOIN rqportassign rqa
    ON ( rqa.lruleid = rq.lid

```

```

        AND RQA.lportfolioid = :PORTFOLIO_ID )
LEFT JOIN prm_dictionary DICT1
    ON ( DICT1.word = 'DEPLOYED' )
LEFT JOIN prm_dictionary DICT2
    ON ( DICT1.word = 'UNDEPLOYED' )
LEFT JOIN l10n_property L10N
    ON ( L10N.l10n_property_id = RQ.sname
        AND L10N.locale_id = 'en_US' )
LEFT JOIN l10n_property L10N1
    ON ( L10N1.l10n_property_id = RQ.sname
        AND L10N1.locale_id = 'en_US' )
LEFT JOIN l10n_property L10N2
    ON ( L10N.l10n_property_id = 0
        AND L10N.locale_id = 'en_US' )
LEFT JOIN l10n_property L10N3
    ON ( L10N3.l10n_property_id = DICT1.l10n_id
        AND L10N3.locale_id = 'en_US' )
LEFT JOIN l10n_property L10N4
    ON ( L10N4.l10n_property_id = DICT1.l10n_id
        AND L10N4.locale_id = 'en_US' )
LEFT JOIN l10n_property L10N5
    ON ( L10N5.l10n_property_id = DICT2.l10n_id
        AND L10N5.locale_id = 'en_US' )
LEFT JOIN l10n_property L10N6
    ON ( L10N6.l10n_property_id = DICT2.l10n_id
        AND L10N6.locale_id = 'en_US' )
LEFT JOIN s
    ON ( s.nd_real_time_rule_fired = RQ.lid
        AND s.portfolio_id = rqa.lportfolioid )

```

--MSSQL QUERY END

Update the query used by the report by running the following statement:

```

DECLARE @new_query VARCHAR(max) = '<<updated-query>>'

UPDATE titanium_reports SET SQL_QUERY = @new_query WHERE NAME = 'rt-rule-performance'

```

Labels for the new headers are added by running the following statement (@field_name is the new disposition column header returned by the report update, and @field_label is the label that will be displayed in UI). **After the updates a PRM server restart is required !**

```

DECLARE @field_name VARCHAR(50) = '<<column-header-returned-by-sql>>';
DECLARE @report_name VARCHAR(50) = 'rt-rule-performance';
DECLARE @field_label NVARCHAR(50) = N'<<column-header-label>>';
DECLARE @field_label_id INT;

EXEC PP_GET_NEXT_L10N_ID_INS
    @L10N_ID=@field_label_id output,
    @TABLE_NAME='TITANIUM_REPORTS_OUTPUT',
    @COLUMN_NAME='FIELD_LABEL_L10N',
    @L10N_VALUE=@field_label,
    @L10N_PROPERTY_NOTE=@field_label,
    @range=1,
    @range_desc='TITANIUM_REPORTS_OUTPUT ranges';

INSERT INTO titanium_reports_output (report_name, field_name, field_label_l10n)
VALUES (@report_name, @field_name, @field_label_id);

```

--ORACLE QUERY

```

WITH s
AS (SELECT nd_real_time_rule_fired,
          Count(no_alerts) total_tran,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'A' THEN no_alerts
            ELSE 0
          END) total_auth,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'D' THEN no_alerts
            ELSE 0
          END) total_decl,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'R' THEN no_alerts
            ELSE 0
          END) total_ref,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'E' THEN no_alerts
            ELSE 0
          END) total_exemp,
          SUM(CASE
            WHEN sd_real_time_rule_disposition NOT IN
              ('A', 'D', 'R', 'E')
            THEN
              no_alerts
            ELSE 0
          END) total_others,
          SUM(total_alerted_amt1) total_amt_tran,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'A' THEN
              total_alerted_amt1
            ELSE 0
          END) total_amt_auth,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'D' THEN
              total_alerted_amt1
            ELSE 0
          END) total_amt_decl,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'R' THEN
              total_alerted_amt1
            ELSE 0
          END) total_amt_ref,
          SUM(CASE
            WHEN sd_real_time_rule_disposition = 'E' THEN
              total_alerted_amt1
            ELSE 0
          END) total_amt_exemp,
          SUM(CASE
            WHEN sd_real_time_rule_disposition NOT IN
              ('A', 'D', 'R', 'E')
            THEN
              total_alerted_amt1
            ELSE 0
          END) total_amt_others
        FROM titanium_rt_rule_perf
        WHERE dd_date BETWEEN Cast(Dateadd(hour, Datediff(hour, 0,
          Dateadd(hour, -1 *
            :REPORT_INTERVAL_DAYS,
              Getdate()))
            , 0) AS DATE) AND Cast ( Getdate() AS DATE)
      GROUP BY nd_real_time_rule_fired)
SELECT Coalesce(L10N.l10n_value, l10n1.l10n_value, l10n2.l10n_value) rule_name,
       ( CASE
         WHEN RQA.ndeployed = 1 THEN
           Coalesce(L10N3.l10n_value, l10n4.l10n_value,
             l10n2.l10n_value)
         ELSE Coalesce(L10N5.l10n_value, l10n6.l10n_value, l10n2.l10n_value)
       END ) status,

```

```

s.total_tran
total_transactions,
( CASE
    WHEN ( s.total_tran IS NULL
          OR s.total_tran = 0 ) THEN NULL
    ELSE Cast(s.total_auth AS DECIMAL) / Cast(s.total_tran AS DECIMAL) *
        100
    END )
proc_auth_tran,
( CASE
    WHEN ( s.total_tran IS NULL
          OR s.total_tran = 0 ) THEN NULL
    ELSE Cast(s.total_decl AS DECIMAL) / Cast(s.total_tran AS DECIMAL) *
        100
    END )
proc_declines_tran,
( CASE
    WHEN ( s.total_tran IS NULL
          OR s.total_tran = 0 ) THEN NULL
    ELSE Cast(s.total_ref AS DECIMAL) / Cast(s.total_tran AS DECIMAL) *
        100
    END )
proc_ref_tran,
( CASE
    WHEN ( s.total_tran IS NULL
          OR s.total_tran = 0 ) THEN NULL
    ELSE Cast(s.total_exemp AS DECIMAL) / Cast(s.total_tran AS DECIMAL) *
        100
    END )
proc_exemp_tran,
( CASE
    WHEN ( s.total_tran IS NULL
          OR s.total_tran = 0 ) THEN NULL
    ELSE Cast(s.total_others AS DECIMAL) / Cast(s.total_tran AS DECIMAL)
        * 100
    END )
proc_others_tran,
s.total_amt_auth
total_tran_auth,
s.total_amt_decl
total_tran_decl,
s.total_amt_ref
total_tran_ref,
s.total_amt_exemp
total_tran_exemp,
s.total_amt_others
total_tran_others,
s.total_amt_tran
total_tran_amount
FROM ruleque rq
join ruledef rd
    ON ( rd.lid = rq.lruledefid
        AND rd.ltype = 0
        AND RD.lsubtype = 2 )
join rportassign rqa
    ON ( rqa.lruleid = rq.lid
        AND RQA.lportfolioid = :PORTFOLIO_ID )
left join prn_dictionary DICT1
    ON ( DICT1.word = 'DEPLOYED' )
left join prn_dictionary DICT2
    ON ( DICT1.word = 'UNDEPLOYED' )
left join l10n_property L10N
    ON ( L10N.l10n_property_id = RQ.sname
        AND L10N.locale_id = 'en_US' )
left join l10n_property L10N1
    ON ( L10N1.l10n_property_id = RQ.sname
        AND L10N1.locale_id = 'en_US' )
left join l10n_property L10N2
    ON ( L10N.l10n_property_id = 0
        AND L10N.locale_id = 'en_US' )
left join l10n_property L10N3

```

```

        ON ( L10N3.l10n_property_id = DICT1.l10n_id
            AND L10N3.locale_id = 'en_US' )
left join l10n_property L10N4
        ON ( L10N4.l10n_property_id = DICT1.l10n_id
            AND L10N4.locale_id = 'en_US' )
left join l10n_property L10N5
        ON ( L10N5.l10n_property_id = DICT2.l10n_id
            AND L10N5.locale_id = 'en_US' )
left join l10n_property L10N6
        ON ( L10N6.l10n_property_id = DICT2.l10n_id
            AND L10N6.locale_id = 'en_US' )
left join s
        ON ( s.nd_real_time_rule_fired = RQ.lid )

```

--ORACLE QUERY END

Update the query used by the report by running the following statement:

```

DECLARE v_new_query VARCHAR(max) := '<<updated-query>>'

UPDATE titanium_reports SET SQL_QUERY = v_new_query WHERE NAME = 'rt-rule-performance'

```

Labels for the new headers are added by running the following statement (v_field_name is the new disposition column header returned by the report update, and v_field_label is the label that will be displayed in UI). **After the updates a PRM server restart is required !**

```

v_report_name VARCHAR2(50) := 'rt-rule-performance';
v_field_name VARCHAR2(50) := '<<column-header-returned-by-sql>>';
v_field_label nvarchar2(50) := N'<<column-header-label>>';
v_field_label_id l10n_property.l10n_property_id%TYPE;
PP_GET_NEXT_L10N_ID_INS ( v_l10n_id => v_field_label_id,
    v_table_name => 'TITANIUM_REPORTS_OUTPUT',
    v_column_name => 'FIELD_LABEL_L10N',
    v_l10n_value => v_field_label,
    v_l10n_property_note => v_field_label,
    v_range => 1,
    v_range_desc => 'TITANIUM_REPORTS_OUTPUT ranges' );
INSERT INTO titanium_reports_output
(
    report_name,
    field_name,
    field_label_l10n
)
VALUES
(
    v_report_name,
    v_field_name,
    v_field_label_id
);

```



© Copyright ACI Worldwide, Inc. 2020

ACI, ACI Worldwide, the ACI logo, ACI Universal Payments, UP, the UP logo and all ACI product/solution names are trademarks or registered trademarks of ACI Worldwide, Inc., or one of its subsidiaries, in the United States, other countries or both. Other parties' trademarks referenced are the property of their respective owners.